

بوت کمپ برنامه نویسی C++ با استاندارد MISRA

توسعه و طراحی الگوریتم های ساختار یافته توسط زبان C++



مجید درهم بخش

مدرس دانشگاه / توسعه دهنده سیستم های نهفته

ایمیل:

Majid.Derhambakhsh@Outlook.com

گیت هاب:

Majid-Derhambakhsh

سخنی از نویسنده

سلام،

این دوره به صورت کاملاً رایگان منتشر شده تا علاقه‌مندان بتوانند یادگیری C و C++ را به صورت اصولی و مهندسی دنبال کنند. در این مسیر، ابتدا مفاهیم و دستورات پایه و پیشرفته C/C++ را بررسی می‌کنیم و سپس به مرور مباحثی مانند اصول کدنویسی، ساختار پروژه، استانداردهای برنامه‌نویسی مانند MISRA و نکات مهندسی نرم‌افزار را به صورت ترکیبی پیش خواهیم برد. همچنین در طول آموزش، مثال‌ها، تمرین‌ها و تحلیل کد نیز انجام می‌شود.

این دوره در ابتدا به صورت حضوری و آنلاین برگزار شده و طبیعتاً بسیاری از توضیحات در جلسات کامل‌تر ارائه شده‌اند. محتوای منتشرشده در اینجا شامل فایل‌های آموزشی، مثال‌ها و منابع دوره است؛ با این حال، تلاش شده تا حد زیادی قابلیت خودآموز داشته باشند. تمامی فایل‌ها، مثال‌ها و کدهای دوره در گیت‌هاب قرار گرفته‌اند تا دسترسی و ادامه یادگیری ساده‌تر باشد. این محتوا با هدف آموزش رایگان منتشر شده است. استفاده شخصی و آموزشی آزاد است؛ اما برای استفاده در مراکز آموزشی، آموزشگاه‌ها، دانشگاه‌ها، دوره‌های سازمانی یا بازنشر محتوا، لطفاً پیش از استفاده هماهنگی و مجوز لازم دریافت شود.

 امیدوارم این آموزش برای شما مفید باشد و در مسیر یادگیری‌تان مؤثر واقع شود. موفق باشید

نکات قبل از آموزش

⚠ توجه مهم

تمام کدها و مثال‌هایی که تا پیش از بخش «مقدمه زبان C/C++» مشاهده می‌کنید، صرفاً جهت آشنایی اولیه با مفاهیم الگوریتمی، ساختار داده و شیوه پیاده‌سازی آن‌ها ارائه شده‌اند.

هدف از نمایش این کدها، ایجاد درک مفهومی و آشنایی با ساختار کلی برنامه‌ها است و انتظار نمی‌رود در این مرحله اقدام به یادگیری، تحلیل عمیق یا حفظ ساختار کدها نمایید.

آموزش اصولی برنامه‌نویسی، مفاهیم زبان، ساختار کدنویسی و تحلیل کامل مثال‌ها از بخش «مقدمه زبان C/C++» به بعد به صورت مرحله به مرحله آغاز خواهد شد.



01

مبانی زبان ها و مفاهیم اولیه

در این بخش به بررسی انواع زبان های برنامه نویسی در سبک های
مختلف میپردازیم

نرم افزار و انواع آنها

در این بخش به تعریف نرم افزار و انواع آنها می پردازیم

نرم افزار چیست؟

- نرم افزار، مجموعه از دستور العمل ها، داده های یا برنامه ها است که هدف خاصی را دنبال می کند، نرم افزار در مقابل سخت افزار است که جنبه های فیزیکی رایانه را توصیف می کند. نرم افزار یک اصطلاح عمومی است که به برنامه ها ، اسکریپت ها و برنامه هایی که بر روی دستگاه اجرا می شوند، اطلاق می شود. می توان آن را به عنوان قسمت متغیر یک کامپیوتر در نظر گرفت ، در حالی که سخت افزار قسمت غیرقابل تغییر است.
- در کل کامپیوتر از دو بخش اصلی تشکیل شده است، سخت افزار و نرم افزار! سخت افزار شامل هر قطعه کامپیوتری مانند موس، کیبورد، مانیتور و... می باشد که می توان آن را مستقیما لمس نمود و نرم افزارها غیر قابل لمس بوده و در حافظه کامپیوتر شما اجرا می شوند مانند آهنگی که در برنامه مدیا پلیر اجرا می کنید (مدیا پلیر یک نرم افزار از نوع مولتی مدیا است) و یا ویندوز ایکس پی یا ویندوز 7 که استفاده می کنید یک نوع نرم افزار از نوع سیستم عامل است.
- سخت افزارها بدون نرم افزار تقریبا غیر قابل استفاده هستند، نرم افزار پل رابط کاربر با سخت افزار است و امکان بهره مندی کامل از سخت افزار را فراهم می نماید.

دسته بندی نرم افزارها



- **نرم افزار پایه (Programming Software)**
- این دسته از نرم افزارها، یکی از انواع معمول، شناخته شده و مورد علاقه کاربران در میان نرم افزار های کامپیوتری است . این نرم افزار در قالب ابزار بوده و به برنامه نویسی در نوشتن برنامه های کامپیوتری کمک می کند.
- **نرم افزارهای کاربردی**
- رایج ترین نوع نرم افزار ، نرم افزار کاربردی یک بسته نرم افزاری رایانه ای است که عملکرد خاصی را برای کاربر یا در برخی موارد برای برنامه دیگری انجام می دهد. یک برنامه می تواند مستقل باشد یا می تواند گروهی از برنامه ها باشد که برنامه را برای کاربر اجرا می کند. نمونه هایی از برنامه های کاربردی مدرن شامل مجموعه های اداری ، نرم افزارهای گرافیکی ، پایگاه های داده و برنامه های مدیریت پایگاه داده، مرورگر های وب، پردازشگر کلمات ، ابزارهای توسعه نرم افزار ، ویرایشگر های تصویر و بستر های ارتباطی است.
- **نرم افزارهای سیستم های نهفته**
- این نرم افزار ها در سیستم هایی با کاربری خاص مانند اسباب بازی، اتومبیل و تجهیزات پزشکی به کار گرفته میشوند.
- **نرم افزارهای سیستم های بلادرنگ**
- در این سیستم ها، عکس العمل بلافاصله صورت می گیرد. نرم افزار هایی بدین منظور بیشتر در مراکز تولیدی نظیر پالایشگاه مورد استفاده دارد که با بروز هر عامل و علامتی، بی درنگ عکس العمل نشان داده و به تنظیم فرآیند تولید بپردازد.
- **نرم افزارهای جانبی (Utilities Software)**
- همچنان که مطابق عرف معمول خدمات نرم افزاری شناخته شده اند، این گونه از نرم افزارها در خصوص مدیریت سخت افزار رایانه و نرم افزارهای رایانه ای کمک می کنند. این فرآیند نرم افزاری، طیف محدودی از وظایف و عملکرد ها را بر عهده دارد. یک پارچه سازه های دیسک سخت (Disk defragmenters)، نرم افزارهای سیستمی جانبی و ویروس یاب ها، برخی از نمونه های متداول نرم افزار های جانبی هستند.
- **نرم افزارهای شبیه سازی و مدل سازی**
- به جهات اقتصادی و ایمنی و صرفه جویی در وقت، برای آموزش و تحقیق در بسیاری از موارد، از این قابلیت در رایانه استفاده می شود. آموزش خلبانی و طراحی بدنه اتومبیل و امثال اینها، مصادیق این دسته اند.



IDE چیست؟



مخفف سه کلمه Integrated Development Environment می باشد، Integrated به معنی یکپارچه، Development به معنی توسعه و Environment به معنی محیط می باشد و در مجموع به معنی محیط یکپارچه توسعه است.

در دنیای فناوری اطلاعات به برنامه نویس توسعه دهنده و به محیطی که یک برنامه نویس در آن برنامه نویسی می کند محیط توسعه یا Development Environment می گویند. بنابراین IDE یک محیط برنامه نویسی است که مجموعه ای از نرم افزارهایی که معمولاً شامل ویرایشگر کد برنامه (Code Editor)، کامپایلر (Compiler)، دیباگر (Debugger)، لینکر (Linker) و در نهایت رابط گرافیکی یا GUI هستند را در یک نرم افزار یکپارچه نموده و در اختیار برنامه نویس قرار داده تا بتوان بر راحتی کدهای برنامه را تولید نمود.

ویرایشگر سورس کد

اولین ویژگی که IDE دارد این است که می توان در آنها کدهای مورد نظر را نوشت و آنها را ویرایش نمود. که به اصطلاح به آنها Source Code Editor می گویند. در واقع این ویرایشگر محیطی است که برنامه نویس کدها را درون آن وارد می کند و کدی که نوشته شده است توسط کامپایلر یا مفسر تبدیل به زبان ماشین و اجرا می شود، این ویرایشگر از نظر کارکرد بسیار شبیه به ویرایشگرهای کدهای HTML متنی است.

کامپایلر (مفسر)

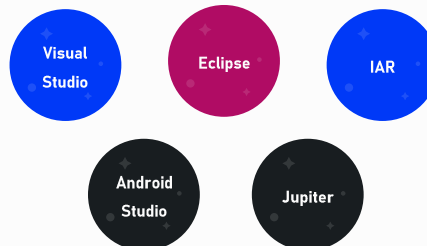
یک کامپایلر که در IDE تعبیه شده است کد نوشته شده در Source Code Editor را تبدیل به برنامه اجرایی می کند و اگر لازم باشد برنامه بصورت خط به خط توسط مفسر یا Interpreter بدون نیاز به کامپایل شدن اجرا می شود.

ابزارهای ساخت اتوماتیک

تمامی فرآیند های مربوط به کامپایل، تفسیر یا Interpret و Debug همگی توسط IDE انجام می شود.

خطایاب ها

IDE ها دارای Debugger هستند تا متوجه شوند کجای کدهای برنامه مشکل دارد.



دسته بندی زبان های برنامه نویسی

زبان های برنامه نویسی در دسته های مختلفی قرار میگیرند، در این بخش به بررسی این دسته ها میپردازیم

سطوح زبان های برنامه نویسی



بررسی زبان های برنامه نویسی بر اساس سطح آنها

زبان های برنامه نویسی را می توان از دیدگاه های مختلف دسته بندی کرد. یکی از مهم ترین دسته بندی ها، دسته بندی بر اساس سطح زبان است. زبان های برنامه نویسی را می توان به دو دسته سطح بالا و سطح پایین تقسیم کرد.

1. **زبان های سطح پایین**، زبان هایی هستند که به زبان ماشین نزدیک تر هستند. زبان ماشین یک زبان دودویی است که مستقیماً توسط سخت افزار کامپیوتر قابل درک است. زبان های سطح پایین برای کنترل مستقیم سخت افزار کامپیوتر استفاده می شوند. به عنوان مثال، زبان اسمبلی یک زبان سطح پایین است که برای توسعه سیستم عامل ها، نرم افزارهای Embedded، و نرم افزارهای پرکاربرد استفاده می شود.
- مثال: زبان اسمبلی

```
mov dx, 0x40  
mov al, 0x1  
out dx, al
```

- مثال: زبان ماشین

```
00000000: b8 01 00 00 00    mov eax, 1
00000005: e8 00 00 00 00    call 0x0000000a
```

2. **زبان های سطح میانی**، زبان هایی هستند که بین زبان های سطح پایین و سطح بالا قرار می گیرند. زبان های میانی معمولاً برای اهداف خاص مانند کامپایل کردن زبان های سطح بالا به زبان ماشین استفاده می شوند. به عنوان مثال، زبان جاوا اسکریپت یک زبان میانی است که برای توسعه وب سایت ها و نرم افزارهای تحت وب استفاده می شود.

- مثال: زبان جاوا اسکریپت

```
const button = document.createElement("button");
button.innerHTML = "Press Button";
button.onclick = function() {
  alert("Button Pressed");
};
document.body.appendChild(button);
```



3. **زبان های سطح بالا**، زبان هایی هستند که به زبان انسان نزدیک تر هستند. زبان های سطح بالا برای توسعه نرم افزارهای کاربردی مانند نرم

افزارهای اداری، نرم افزارهای آموزشی، و نرم افزارهای سرگرمی استفاده می شوند. به عنوان مثال، زبان های پایتون، جاوا و سی شارپ از زبان های سطح بالا هستند.

- مثال: زبان پایتون

```
print(2 + 2)
```

این کد مقدار 4 را چاپ می کند.

- مثال: زبان C#

```
public class HelloWorld
{
    public static void Main(string[] args)
    {
        Console.WriteLine("Hello, world!");
    }
}
```

این کد یک کلاس ساده با یک متد اصلی را تعریف می کند که عبارت **"Hello, world!"** را چاپ می کند.

نحو زبان های برنامه نویسی

01

زبان های کامپایلری

02

زبان های تفسیری

بررسی زبان های برنامه نویسی بر اساس نحو آنها

- **زبان های کامپایلری**، زبان هایی هستند که کدهای نوشته شده به آن ها به زبان ماشین تبدیل می شوند. این زبان ها معمولاً سریع تر از زبان های تفسیری هستند، زیرا کدهای آن ها فقط یک بار کامپایل می شوند و سپس می توان آن ها را به صورت مستقیم اجرا کرد. به عنوان مثال، زبان های سی، سی پلاس پلاس و جاوا از زبان های کامپایلری هستند.



- **زبان های تفسیری**، زبان هایی هستند که کدهای نوشته شده به آن ها به صورت مستقیم اجرا می شوند. این زبان ها معمولاً کندتر از زبان های کامپایلری هستند، زیرا کدهای آن ها هر بار که اجرا می شوند باید تفسیر شوند. به عنوان مثال، زبان های پایتون، جاوا اسکریپت و روبی از زبان های تفسیری هستند.



جهت زبان های برنامه نویسی

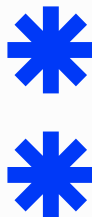
زبان های شی گرا

01



02

زبان های رویه ای



بررسی زبان های برنامه نویسی بر اساس جهت آنها

- **زبان های شی گرا**، زبان هایی هستند که بر اساس مفاهیم شی گرایی مانند کلاس، شی، وراثت، و جابجایی ساخته شده اند. این زبان ها معمولاً برای نوشتن کدهای پیچیده تر مناسب ترند. به عنوان مثال، زبان های پایتون، جاوا و سی شارپ از زبان های شی گرا هستند.
- **زبان های رویه ای**، زبان هایی هستند که بر اساس مفاهیم رویه ای مانند تابع، متغیر، و کنترل جریان ساخته شده اند. این زبان ها معمولاً برای نوشتن کدهای ساده تر مناسب ترند. به عنوان مثال، زبان های سی، سی پلاس پلاس و جاوا اسکریپت از زبان های رویه ای هستند.

اصطلاحات کار با داده ها در حافظه

در هنگام کار با داده ها در حافظه با اصطلاحاتی روبرو می شویم که به معرفی آنها می پردازیم

اصطلاحات پر کاربرد در داده ها

- **بیت (bit):** حافظه از واحدهای کوچکی بنام بیت تشکیل شده است که هر بیت قابلیت نگهداری یک 0 یا 1 را در خود دارد.
 - **بایت (Byte):** به هر 8 بیت یک بایت گفته می شود که واحد اندازه گیری حافظه است.
 - **کلمه (Word):** معمولا هر 2 یا 4 بایت را یک کلمه در نظر می گیرند.
 - **کیلوبیت (Kb):** هر 256 بایت یک کیلوبیت را تشکیل میدهد.
 - **کیلوبایت (KB):** هر 256 بایت یک کیلوبایت را تشکیل میدهد. هر 1024 بایت یک کیلوبایت را تشکیل می دهد. در سیستم دودویی هر کیلوبایت معادل 2^{10} می باشد، واحد های دیگر از این واحد تشکیل میشوند و عبارت اند از:
- MB (Mega Byte) = 1024 KB
 - GB (Giga Byte) = 1024 MB
 - TB (Tera Byte) = 1000 GB

- **کاراکتر:** در کامپیوترها کار با اطلاعات بر مبنای بیت دشوار است از این رو از کاراکترها استفاده می شود که بتوان اطلاعات را به صورت ارقام، حروف و نمادها نمایش داد. برای ذخیره سازی کاراکترها به هریک از آنها یک کد عددی نسبت داده شده است و در حقیقت کد عددی هر کاراکتر در کامپیوتر ذخیره می گردد. در گذشته پر کاربردترین کد مورد استفاده، کد ASCII بود که برای نمایش هر کاراکتر از یک بایت استفاده می کرد. از آنجا که هر بایت می تواند بین 0 تا 255 تغییر کند، بنابراین تا 256 کاراکتر قابل تعریف است. از این بین کدهای بین 0 تا 127 به صورت استاندارد برای علائم و حروف انگلیسی تعریف شده است و کدهای بالاتر از 127 برای هر کشور خالی گذاشته شده است تا بتوانند حروف خاص زبان خود را تعریف کنند. شکل زیر کدهای اسکی برخی از حروف بزرگ انگلیسی را نشان می دهد.

A	B	C	D	E	F	G	H	I	J		R	S	T	U	V	W	X	Y	Z
65	66	67	68	69	70	71	72	73	74		82	83	84	85	86	87	88	89	90

02

الگوریتم و ساختارهای داده

در این بخش به تعریف الگوریتم و انواع ساختارهای داده خواهیم پرداخت

الگوریتم در زندگی روزمره

در این بخش مثال هایی از الگوریتم هایی را خواهیم زد که در زندگی روزمره کاربرد دارند



الگوریتم یک روش یا دستورالعمل است که برای انجام کاری استفاده می شود. مثلاً برای پختن یک کیک، یک الگوریتم داریم که به ما می گوید چه کاری باید انجام دهیم.

الگوریتم ها معمولاً به صورت گام به گام بیان می شوند. به عنوان مثال، الگوریتم پختن کیک ممکن است به این صورت باشد:

1. مواد لازم را آماده کنید
2. فر را روشن کنید و گرم کنید
3. قالب کیک را آماده کنید
4. مواد خشک را با هم مخلوط کنید
5. مواد تر را با هم مخلوط کنید
6. مواد خشک و تر را با هم مخلوط کنید
7. کیک را در فر بگذارید و بپزید

الگوریتم ها دقیق و قابل اعتماد هستند، اما ممکن است برای مسائل ساده بیش از حد پیچیده باشند. به مثال های زیر توجه کنید:





الگوریتم برای پیدا کردن یک شی در یک اتاق:

1. به اطراف اتاق نگاه کنید
2. اگر شی را پیدا کردید، کار تمام است
3. اگر شی را پیدا نکردید، به گوشه دیگری از اتاق بروید
4. مراحل 1 و 2 را تکرار کنید تا شی را پیدا کنید

الگوریتم برای پختن یک تخم مرغ:

1. یک تخم مرغ را در یک قابلمه آب بریزید
2. آب را بجوشانید
3. پس از جوشیدن آب، حرارت را کم کنید و اجازه دهید تخم مرغ به مدت 3-5 دقیقه بجوشد
4. تخم مرغ را از آب خارج کنید و آب سرد روی آن بریزید

پیدا کردن دراهم بخش



الگوریتم برای پیدا کردن یک آدرس:

1. به یک نقشه یا برنامه ناوبری نگاه کنید
2. آدرس را در نقشه یا برنامه ناوبری پیدا کنید
3. مسیر را دنبال کنید

الگوریتم برای خرید کردن:

1. لیست خرید خود را تهیه کنید
2. به فروشگاه بروید
3. اقلام مورد نیاز خود را پیدا کنید
4. اقلام را در سبد خرید خود قرار دهید
5. در هنگام پرداخت، پول یا کارت اعتباری خود را ارائه دهید

سجید درهم بخش





الگوریتم برای یافتن بزرگترین عدد در یک مجموعه اعداد:

1. بزرگترین عدد را به عنوان صفر در نظر بگیرید
2. هر عدد در مجموعه را بررسی کنید
3. اگر عددی بزرگتر از بزرگترین عدد فعلی باشد
4. آن عدد را به عنوان بزرگترین عدد فعلی در نظر بگیرید

الگوریتم برای یافتن کوچکترین عدد در یک مجموعه اعداد:

1. اولین عدد را به عنوان بی نهایت در نظر بگیرید
2. هر عدد در مجموعه را بررسی کنید
3. اگر عددی کوچکتر از کوچکترین عدد فعلی باشد
4. آن عدد را به عنوان کوچکترین عدد فعلی در نظر بگیرید

رایگان درهم بخش

الگوریتم در کامپیوتر و برنامه نویسی

در این بخش به بررسی الگوریتم در دنیای کامپیوتر و الگوهای آن آشنا خواهیم شد



در اصطلاح برنامه نویسی کامپیوتر، یک الگوریتم مجموعه ای از دستورالعمل های کاملاً تعریف شده برای حل یک مسئله خاص است. مجموعه ای از ورودی ها را می گیرد و خروجی مورد نظر را تولید می کند. به عنوان مثال:

الگوریتمی برای جمع دو عدد:

1. دو عدد ورودی بگیرید
2. اعداد را با استفاده از عملگر + جمع کنید
3. نتیجه را نمایش دهید

ویژگی های یک الگوریتم خوب:

- ورودی و خروجی باید دقیق تعریف شود
- هر مرحله الگوریتم باید واضح و بدون ابهام باشد
- الگوریتم ها باید در میان راه های مختلف برای حل یک مسئله مؤثرتر باشند
- یک الگوریتم نباید شامل کد کامپیوتری باشد. در عوض، الگوریتم باید به گونه ای نوشته شود که بتوان از آن در زبان های برنامه نویسی مختلف استفاده کرد





مثال هایی از الگوریتم ها در کامپیوتر:

الگوریتم 1: جمع دو عدد دریافت شده از کاربر

مرحله 1: شروع

مرحله 2: متغیرهای num1، num2 و sum را تعریف کنید

مرحله 3: مقادیر num1 و num2 را دریافت کنید

مرحله 4: num1 و num2 را جمع کنید و نتیجه را در sum ذخیره کنید

$sum \leftarrow num1 + num2$

مرحله 5: نمایش مجموع اعداد

مرحله 6: توقف

الگوریتم 2: پیدا کردن بزرگترین عدد از بین سه عدد

مرحله 1: شروع

مرحله 2: متغیرهای a، b و c را تعریف کنید

مرحله 3: مقادیر a، b و c را دریافت کنید

مرحله 4:

اگر $b < a$

اگر $c < a$

نمایش بده a بزرگترین عدد است

در غیر اینصورت

نمایش بده c بزرگترین عدد است

در غیر اینصورت

اگر $c < b$

نمایش بده b بزرگترین عدد است

در غیر اینصورت

نمایش بده c بزرگترین عدد است

مرحله 5: توقف

الگوریتم ۳: یافتن ریشه های معادله درجه دوم $ax^2 + bx + c = 0$

در ریاضیات برای محاسبه این معادله از رابطه زیر استفاده میشود:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

مرحله ۱: شروع

مرحله ۲: تعریف متغیرهای $a, b, c, D, x_1, x_2, r_p$ و i_p

مرحله ۳: محاسبه عبارت جدا کننده

$$D \leftarrow b^2 - 4ac$$

مرحله ۴:

اگر $D \geq 0$

$$r_1 \leftarrow (-b + \sqrt{D}) / 2a$$

$$r_2 \leftarrow (-b - \sqrt{D}) / 2a$$

نمایش r_1 و r_2 به عنوان ریشه ها

در غیر این صورت

محاسبه بخش حقیقی و موهومی

$$r_p \leftarrow -b / 2a$$

$$i_p \leftarrow \sqrt{-D} / 2a$$

نمایش $r_p + j(i_p)$ و $r_p - j(i_p)$ به عنوان ریشه ها

مرحله ۵: پایان

تمرین های بخش

1. الگوریتمی بنویسید تا مقدار x را در رابطه زیر پیدا کن:

$$x + 5 = 9$$

انواع داده و ساختارها (Data Structure and Types)

در این بخش به بررسی انواع داده و ساختارها در کامپیوتر میپردازیم

ساختار داده چیست؟

"ساختار داده" مثل انباری هست که اطلاعات را به صورت منظم توی کامپیوتر نگه میدارد. به نوعی اطلاعات را توی کامپیوتر دسته بندی میکند تا بشود به راحتی به آن دسترسی پیدا کرد و تغییراتی در آن ایجاد نمود.

بسته به نیاز و پروژه شما، مهم است که ساختار داده مناسب برای پروژه خود را انتخاب کنید. به عنوان مثال، اگر می خواهید داده ها را به صورت متوالی در حافظه ذخیره کنید، می توانید به سراغ ساختار داده Array بروید.

memory locations						
1004	1005	1006	1007	1008	1009	1010
...	2	1	5	3	4	...
	0	1	2	3	4	
index						



انواع ساختار داده

اصولا ساختارهای داده به دو دسته تقسیم می شوند:

- ساختار داده خطی
- ساختار داده غیر خطی

در این بخش به بررسی انواع ساختارها پرداخته و با آن آشنا خواهیم شد

مجموعه داده ها



ساختار داده خطی

در ساختارهای داده خطی، عناصر به ترتیب یکی پس از دیگری مرتب می شوند. از آنجایی که عناصر به ترتیب خاصی چیده شده اند، پیاده سازی آنها آسان است.

با این حال، زمانی که پیچیدگی برنامه افزایش می یابد، ساختار داده های خطی ممکن است به دلیل پیچیدگی های عملیاتی بهترین انتخاب نباشند.

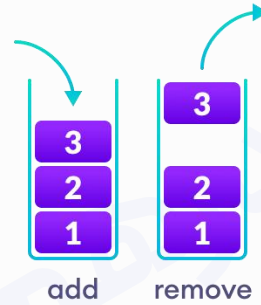
1. ساختار داده آرایه

در یک آرایه، عناصر موجود در حافظه در حافظه پیوسته مرتب شده اند. تمام عناصر یک آرایه از یک نوع هستند. و نوع عناصر قابل ذخیره سازی در قالب آرایه توسط زبان برنامه نویسی تعیین می شود.

2	1	5	3	4
0	1	2	3	4
index				

2. ساختار داده پشته (Stack)

در ساختار داده پشته، عناصر در اصل LIFO ذخیره می شوند. یعنی آخرین عنصر ذخیره شده در یک پشته ابتدا حذف خواهد شد.



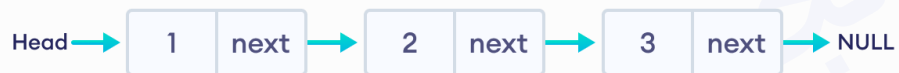
3. ساختار داده صف

برخلاف پشته، ساختار داده صف در اصل FIFO کار می کند، جایی که اولین عنصر ذخیره شده در صف ابتدا حذف می شود. درست مانند صفی از افراد در پیشخوان بلیط که در آن اولین نفر در صف اول بلیط را دریافت می کند.



4. ساختار داده لیست پیوندی

در ساختار داده لیست پیوندی، عناصر داده از طریق یک سری گره به هم متصل می شوند. و هر گره حاوی ارقام داده و آدرس گره بعدی است.



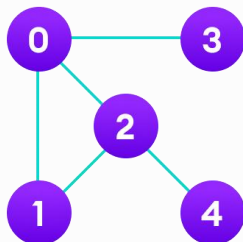
ساختار داده غیرخطی

برخلاف ساختارهای داده خطی، عناصر در ساختارهای داده غیرخطی در هیچ ترتیبی قرار ندارند. در عوض آنها به صورت سلسله مراتبی مرتب شده اند که در آن یک عنصر به یک یا چند عنصر متصل می شود.

ساختارهای داده غیر خطی بیشتر به ساختارهای داده مبتنی بر نمودار و درخت تقسیم می شوند.

1. ساختار داده گراف

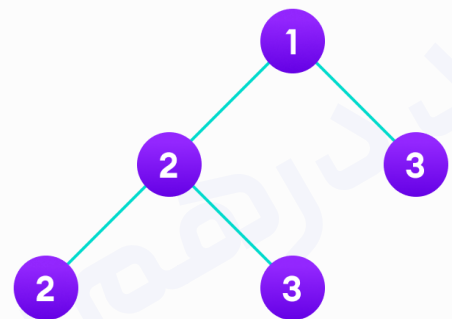
در ساختار داده های گراف، هر گره را راس می نامند و هر راس از طریق یال ها به رئوس دیگر متصل می شود.





2. ساختار داده درختی

مانند یک نمودار، یک درخت نیز مجموعه ای از رئوس و یال ها است. با این حال، در ساختار داده درختی، تنها یک یال می تواند بین دو راس وجود داشته باشد



مجید درویش بخش



مقایسه ساختار داده خطی و غیرخطی

ساختار داده غیرخطی	ساختار داده خطی
اقلام داده به ترتیب غیر ترتیبی (به صورت سلسله مراتبی) مرتب شده اند.	اقلام داده به ترتیب متوالی و یکی پس از دیگری مرتب شده اند.
اقلام داده در لایه های مختلف وجود دارند.	همه موارد در یک لایه وجود دارد.
به اجراهای متعدد نیاز دارد. یعنی اگر از عنصر اول شروع کنیم ممکن است نتوان همه عناصر را در یک پاس طی کرد.	می توان آن را در یک بار پیمایش کرد. یعنی اگر از عنصر اول شروع کنیم، می توانیم تمام عناصر را به صورت متوالی در یک پاس طی کنیم.
ساختارهای مختلف بسته به نیاز از حافظه به روش های کارآمد متفاوتی استفاده می کنند.	استفاده از حافظه کارآمد نیست.
پیچیدگی زمانی ثابت می ماند.	پیچیدگی زمانی با اندازه داده افزایش می یابد.
مثال: درخت، نمودار، نقشه	مثال: آرایه ها، پشته، صف

قضیه اصلی (Master Theorem)

در این بخش به بررسی حل روابط تکراری به کمک قضیه اصلی میپردازیم

قضیه اصلی (Master Theorem)

روش اصلی، فرمولی برای حل روابط بازگشتی به شکل زیر است:

$$T(n) = aT(n/b) + f(n)$$

که در آن:

n = اندازه ورودی

a = تعداد زیر مسئله ها در بازگشت

n/b = اندازه هر زیر مسئله. فرض می شود که همه زیر مسئله ها اندازه یکسانی دارند.

$f(n)$ = هزینه کارهایی که خارج از فراخوانی بازگشتی انجام می شود، که شامل هزینه تقسیم مسئله و هزینه ادغام راه حل ها می شود.

در اینجا، $a \geq 1$ و $b > 1$ ثابت هستند و $f(n)$ یک تابع مثبت مجانبی است.

یک تابع با رشد مثبت مجانبی به این معناست که برای یک مقدار n به اندازه کافی بزرگ، $f(n)$ بزرگتر از صفر خواهد بود.

به زبان ساده تر، هر تابعی که با افزایش n به اندازه کافی زیاد، همیشه مقادیر مثبت داشته باشد، یک تابع با رشد مثبت مجانبی نامیده می شود.

قضیه اصلی

اگر $a \geq 1$ و $b > 1$ ثابت باشند و $f(n)$ یک تابع مثبت مجانبی باشد، پیچیدگی زمانی یک رابطه بازگشتی به صورت زیر بیان می شود:

$$T(n) = aT(n/b) + f(n)$$

که در آن، $T(n)$ دارای حدود مجانبی زیر است:

- اگر $f(n) = O(n^{\log_b a - \epsilon})$ ، آنگاه $T(n) = \Theta(n^{\log_b a})$
 - اگر $f(n) = \Theta(n^{\log_b a} * \log a)$ ، آنگاه $T(n) = \Theta(n^{\log_b a} * \log a)$
 - اگر $f(n) = \Omega(n^{\log_b a - \epsilon})$ ، آنگاه $T(n) = \Theta(f(n))$
- $\epsilon > 0$ یک ثابت است

هر یک از شرایط فوق را می توان به صورت زیر تفسیر کرد:

1. اگر هزینه حل زیر مسئله ها در هر سطح با یک ضریب مشخص افزایش یابد، مقدار $f(n)$ به صورت چند جمله ای کوچکتر از $n^{\log_b a}$ خواهد شد.

بنابراین، پیچیدگی زمانی تحت تأثیر هزینه آخرین سطح قرار می گیرد، یعنی $n^{\log_b a}$

2. اگر هزینه حل زیر مسئله ها در هر سطح تقریباً برابر باشد، مقدار $f(n)$ برابر با $n^{\log_b a}$ خواهد شد. بنابراین، پیچیدگی زمانی برابر با $f(n)$ ضرب در

تعداد کل سطوح خواهد بود، یعنی $n^{\log_b a} * \log a$

3. اگر هزینه حل زیر مسئله ها در هر سطح با یک ضریب مشخص کاهش یابد، مقدار $f(n)$ به صورت چند جمله ای بزرگتر از $n^{\log_b a}$ خواهد شد.

بنابراین، پیچیدگی زمانی تحت تأثیر هزینه $f(n)$ قرار می گیرد

مثال حل شده از قضیه اصلی

$$T(n) = 3T(n/2) + n2$$

$$a = 3$$

$$n/b = n/2$$

$$f(n) = n^2$$

$$\log_b a = \log_2 3 = 1.58 < 2$$

$$\text{ie. } f(n) < n^{\log_b a + \epsilon}$$

$$T(n) = f(n) = \Theta(n^2)$$

محدودیت های قضیه اصلی

قضیه اصلی را نمی توان در موارد زیر استفاده کرد:

- $T(n) = \sin n$ به عنوان مثال، $T(n)$ یکنواخت نیست.
- $f(n) = 2^n$ یک چند جمله ای نیست. به عنوان مثال، $f(n)$.
- $a = 2n$ یک ثابت نیست. به عنوان مثال، a .
- a کمتر از 1 است

03

الگوهای الگوریتم

در این بخش به بررسی انواع الگو الگوریتم های مورد استفاده خواهیم پرداخت

الگوریتم تقسیم و حل (Divide and Conquer)

در این بخش به بررسی الگو الگوریتم تقسیم و حل خواهیم پرداخت



الگوریتم تقسیم و حل (Divide and Conquer)

الگوریتم تقسیم و حل روشی برای حل یک مسئله ی بزرگ است که در آن به صورت زیر عمل میشود:

1. مسئله را به زیر مسئله های کوچک تر تقسیم می کنیم.
 2. زیر مسئله ها را حل می کنیم.
 3. سپس پاسخ های زیر مسئله ها را ترکیب می کنیم تا پاسخ مسئله اصلی را به دست آوریم.
- برای استفاده از الگوریتم تقسیم و حل از روش بازگشتی استفاده می شود. برای یادگیری توابع بازگشتی به سرفصل این درس مراجعه کنید.

الگوریتم های تقسیم و حل چگونه کار می کنند؟

در اینجا مراحل استفاده از این الگوریتم ارائه شده است:

1. تقسیم: موضوع اصلی را با استفاده از بازگشت به زیرمجموعه های کوچکتر تقسیم کنید.
2. پاسخ: زیرمجموعه های کوچک را به صورت بازگشتی حل کنید. اگر زیر مسئله به اندازه کافی کوچک باشد، آن را مستقیماً حل کنید.
3. ترکیب: راه حل های زیر مسئله هایی که بخشی از روند بازگشتی هستند را می توانید تا بخش اصلی حل کنید.



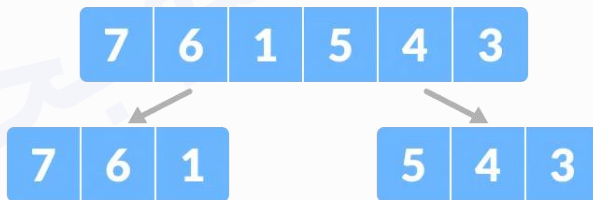
بیابید این مفهوم را با کمک یک مثال درک کنید:

در اینجا، ما یک آرایه را با استفاده از یک تقسیم بندی و تفسیر (یعنی مرتب سازی ادغامی) مرتب می کنیم.

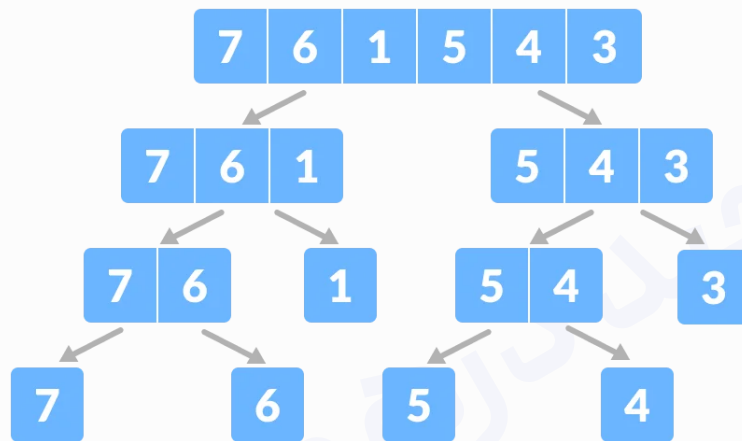
1. فرض کنیم آرایه داده شده به این صورت باشد:



2. آرایه را به دو نیمه تقسیم کنید.

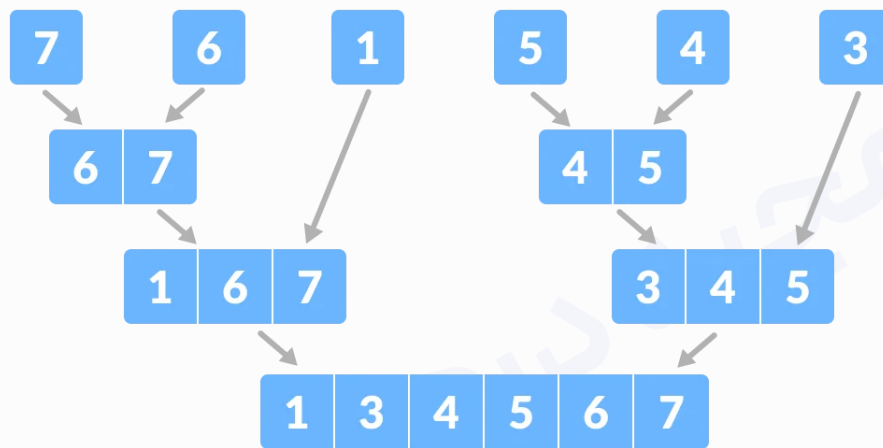


مجدداً، هر قسمت فرعی را به صورت بازگشتی به دو نیمه تقسیم کنید تا زمانی که عناصر جداگانه بدست آورید.



3. اکنون عناصر جداگانه را به صورت مرتب ترکیب کنید. در اینجا، مراحل را حل و ترکیب مشاهده میکنید.

مجموعه دانش



پیچیدگی زمانی

پیچیدگی الگوریتم تقسیم و حل با استفاده از قضیه اصلی (Master Theorem) محاسبه می شود.

$$T(n) = aT(n/b) + f(n)$$

n : اندازه ورودی

a : تعداد مشکلات فرعی در بازگشت

n/b : اندازه هر زیر مسئله. فرض بر این است که همه مسائل فرعی دارای اندازه یکسانی هستند.



$f(n)$: هزینه کار انجام شده خارج از فراخوان بازگشتی، که شامل هزینه تقسیم مسئله و هزینه ادغام راه حل ها می شود.

اجازه دهید برای یافتن پیچیدگی زمانی یک مسئله بازگشتی مثالی بزنیم.

برای مرتب سازی ادغام، معادله را می توان به صورت زیر نوشت:

$$\begin{aligned}T(n) &= aT(n/b) + f(n) \\ &= 2T(n/2) + O(n)\end{aligned}$$

جایی که،

$a = 2$ (هر بار یک مسئله به 2 مشکل فرعی تقسیم می شود)

$n/b = n/2$ (اندازه هر مسئله فرعی نیمی از ورودی است)

$f(n)$ = زمان صرف شده برای تقسیم مسئله و ادغام مسائل فرعی

$T(n/2) = O(n \log n)$ (برای درک این موضوع میتوانید به سرفصل قضیه اصلی مراجعه کنید).

اکنون،

$$\begin{aligned}T(n) &= 2T(n \log n) + O(n) \\ &\approx O(n \log n)\end{aligned}$$

تقسیم و حل (Divide and Conquer) در مقابل رویکرد پویا (Dynamic Approach)

روش تقسیم و حل یک مسئله را به زیر مسئله کوچکتر تقسیم می کند. این زیر مسئله ها به صورت بازگشتی حل می شوند. نتیجه هر زیر مسئله برای ارجاع در آینده ذخیره نمی شود، در حالی که در یک رویکرد پویا، نتیجه هر زیر مسئله برای ارجاع در آینده ذخیره می شود.

هنگامی که یک زیر مسئله چندین بار حل نمی شود، از روش تقسیم و حل استفاده کنید. هنگامی که قرار است از نتیجه یک زیر مسئله چندین بار در آینده استفاده شود، از رویکرد پویا استفاده کنید.

بیایید با یک مثال این را بفهمیم. اگر ما در تلاش برای یافتن سری فیبوناچی هستیم، پس...

رویکرد تقسیم و حل:

fib(n)

اگر $n < 2$ بود، 1 را برگردانید

در غیر این صورت، $f(n-1) + f(n-2)$ را برگردانید

رویکرد پویا:

`[] = mem`

`fib(n)`

اگر n در `mem` بود: مقدار `mem[n]` را برگردان

در غیر اینصورت،

اگر $n < 2$ `f = 1`

در غیر اینصورت `f = f(n - 1) + f(n - 2)`

`mem[n] = f`

`f` را برگردان

در یک رویکرد پویا، `mem` نتیجه هر زیر مسئله را ذخیره می کند.

مزایای الگوریتم تقسیم و حل

- پیچیدگی ضرب دو ماتریس با استفاده از روش ساده $O(n^3)$ است، در حالی که استفاده از رویکرد تقسیم و حل (یعنی ضرب ماتریس استراسن) $O(n^{2.8074})$ است. این رویکرد مشکلات دیگر مانند برج هانوی را نیز ساده می کند.



- این رویکرد برای سیستم های چند پردازشی مناسب است.
- این روش باعث استفاده موثر از حافظه نهان می شود.

کاربرد های الگوریتم تقسیم و حل

- جستجوی باینری (Binary Search)
- ترکیب مرتب سازی (Merge Sort)
- مرتب سازی سریع (Quick Sort)
- ضرب ماتریس استراسن (Strassen's Matrix multiplication)
- الگوریتم کاراتسوبا (Karatsuba Algorithm)

مجموعه دراهم بخش



الگوریتم جست و جوی عمقی (Depth first Search)

در این بخش به بررسی الگو الگوریتم جست و جوی عمقی خواهیم پرداخت



الگوریتم جست و جوی عمقی (Depth first Search)

Depth first Search یا Depth first traversal یک الگوریتم بازگشتی برای جستجوی تمام رئوس یک نمودار یا ساختار داده درختی است. که در آن پیمایش به معنای بازدید از تمام گره های یک گراف است.

اجرای استاندارد DFS هر رأس گراف را در یکی از دو دسته قرار می دهد:

1. دیده شده

2. دیده نشده

هدف از الگوریتم این است که هر راس را به عنوان دیده شده علامت گذاری کند و در عین حال از چرخه ها اجتناب کند.

الگوریتم DFS به صورت زیر عمل می کند:

1. با قرار دادن هر یک از رئوس نمودار در بالای پشته شروع کنید.

2. آیتم بالای پشته را بردارید و آن را به لیست بازدید شده اضافه کنید.

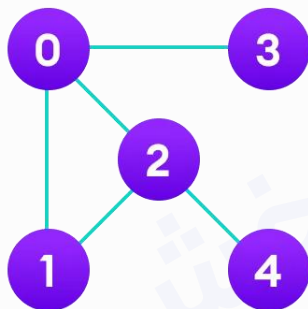


3. لیستی از گره های مجاور آن راس ایجاد کنید. مواردی را که در لیست بازدید شده نیستند به بالای پشته اضافه کنید.

4. به تکرار مراحل 2 و 3 ادامه دهید تا پشته خالی شود.

مثال جست و جوی عمقی

بیا ببینیم الگوریتم Depth First Search با یک مثال چگونه کار می کند. ما از یک گراف بدون جهت با 5 راس استفاده می کنیم.

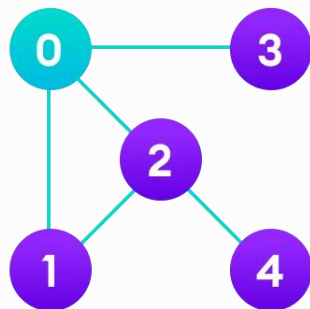
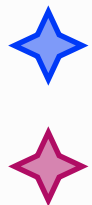


Visited



Stack

از راس 0 شروع می کنیم، الگوریتم DFS با قرار دادن آن در لیست دیده شده و قرار دادن تمام رئوس مجاور آن در پشته شروع می شود.



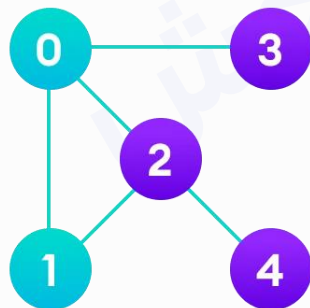
0				
---	--	--	--	--

Visited

1	2	3		
---	---	---	--	--

Stack

در مرحله بعد، از عنصر بالای پشته یعنی 1 بازدید می کنیم و به گره های مجاور آن می رویم. از آنجایی که 0 قبلاً بازدید شده است، به جای آن از 2 بازدید می کنیم.



0	1			
---	---	--	--	--

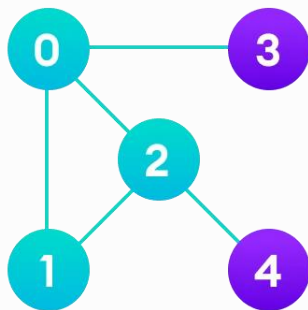
Visited

2	3			
---	---	--	--	--

Stack



راس 2 دارای یک راس مجاور بازدید نشده در 4 است، بنابراین ما آن را به بالای پشته اضافه می کنیم و از آن بازدید می کنیم.

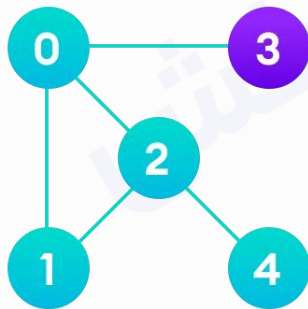


0	1	2		
---	---	---	--	--

Visited

4	3			
---	---	--	--	--

Stack



0	1	2	4	
---	---	---	---	--

Visited

3				
---	--	--	--	--

Stack

پس از بازدید از آخرین عنصر 3، هیچ گره مجاور بازدید نشده ای ندارد، بنابراین اولین پیمایش عمق گراف را تکمیل کردیم.



شبه کد جست و جوی عمقی

شبه کد برای جست و جوی عمقی در زیر نشان داده شده است. در تابع `init()`، توجه داشته باشید که تابع DFS را روی هر گره اجرا می کنیم. این به این دلیل است که گراف ممکن است دو بخش مجزا و جدا از هم داشته باشد، بنابراین برای اطمینان از اینکه هر راس را پوشش می دهیم، می توانیم الگوریتم DFS را روی هر گره نیز اجرا کنیم.



```
DFS(G, u)
```

```
  u.visited = true
```

```
  for each  $v \in G.Adj[u]$ 
```

```
    if  $v.visited == false$ 
```

```
      DFS(G, v)
```

```
init() {
```

```
  For each  $u \in G$ 
```

```
    u.visited = false
```

```
  For each  $u \in G$ 
```

```
    DFS(G, u)
```

```
}
```

نوشتن کد جست و جوی عمقی

کد برای الگوریتم جستجوی عمقی با یک مثال در زیر نشان داده شده است. کد ساده شده است به طوری که ما می توانیم به جای سایر جزئیات، روی الگوریتم تمرکز کنیم.



نوشتن کد جست و جوی عمقی به زبان C

- کدهای اولیه و تعریف ساختار گره

کد زیر یک ساختار داده ساده به نام «node» را تعریف می کند. این ساختار داده دارای دو فیلد است:

vertex: یک عدد صحیح که نشان دهنده رأس گره است.

next: یک اشاره گر به گره بعدی در لیست.

C

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int vertex;
    struct node* next;
};

struct node* createNode(int v);
```

• تعریف ساختار گراف

این ساختار برای نمایش کل گراف استفاده می شود. این ساختار سه فیلد دارد:

`numVertices`: تعداد رأس های گراف.

`visited`: آرایه ای برای مشخص کردن اینکه کدام رأس ها بازدید شده اند.

`adjLists`: آرایه ای از لیست های پیوندی، که هر لیست مجاورت یک رأس را نشان می دهد.

C

```
struct Graph {
    int numVertices;
    int* visited;

    // We need int** to store a two dimensional array.
    // Similary, we need struct node** to store an array of Linked lists
    struct node** adjLists;
};
```

• تعریف تابع DFS

این تابع اصلی الگوریتم جستجوی عمق اول است. این تابع ورودی یک رأس را می‌گیرد و به صورت بازگشتی تمام گره‌های مجاور آن را که بازدید نشده‌اند، بازدید می‌کند.

C

```
void DFS(struct Graph* graph, int vertex) {
    struct node* adjList = graph->adjLists[vertex];
    struct node* temp = adjList;

    graph->visited[vertex] = 1;
    printf("Visited %d \n", vertex);

    while (temp != NULL) {
        int connectedVertex = temp->vertex;

        if (graph->visited[connectedVertex] == 0) {
            DFS(graph, connectedVertex);
        }
        temp = temp->next;
    }
}
```

• تعریف تابع createNode

این تابع یک گره جدید در لیست مجاورت ایجاد می کند. این تابع ورودی شماره رأسی را که گره باید نشان دهد را می گیرد.

C

```
struct node* createNode(int v) {  
    struct node* newNode = malloc(sizeof(struct node));  
    newNode->vertex = v;  
    newNode->next = NULL;  
    return newNode;  
}
```

• تعریف تابع createGraph

این تابع یک گراف جدید با تعداد رأس های مشخص شده ایجاد می کند. این تابع ورودی تعداد رأس های گراف را می گیرد.

(صفحه بعد)

```
struct Graph* createGraph(int vertices) {  
    struct Graph* graph = malloc(sizeof(struct Graph));  
    graph->numVertices = vertices;  
  
    graph->adjLists = malloc(vertices * sizeof(struct node*));  
  
    graph->visited = malloc(vertices * sizeof(int));  
  
    int i;  
    for (i = 0; i < vertices; i++) {  
        graph->adjLists[i] = NULL;  
        graph->visited[i] = 0;  
    }  
    return graph;  
}
```

- **تعریف تابع addEdge**

این تابع یک یال بین دو رأس در گراف اضافه می کند. این تابع ورودی شماره رأس های شروع و مقصد یال را می گیرد.

(صفحه بعد)



c

```
void addEdge(struct Graph* graph, int src, int dest) {  
    // Add edge from src to dest  
    struct node* newNode = createNode(dest);  
    newNode->next = graph->adjLists[src];  
    graph->adjLists[src] = newNode;  
  
    // Add edge from dest to src  
    newNode = createNode(src);  
    newNode->next = graph->adjLists[dest];  
    graph->adjLists[dest] = newNode;  
}
```

- تعریف تابع printGraph

این تابع گراف را چاپ می کند. این تابع ورودی گراف را می گیرد.

(صفحه بعد)

C

```
void printGraph(struct Graph* graph) {  
    int v;  
    for (v = 0; v < graph->numVertices; v++) {  
        struct node* temp = graph->adjLists[v];  
        printf("\n Adjacency list of vertex %d\n ", v);  
        while (temp) {  
            printf("%d -> ", temp->vertex);  
            temp = temp->next;  
        }  
        printf("\n");  
    }  
}
```

- اجرای دستورات درون تابع main جهت بررسی عملکرد کد نوشته شده

(صفحه بعد)



c

```
int main() {  
    struct Graph* graph = createGraph(4);  
    addEdge(graph, 0, 1);  
    addEdge(graph, 0, 2);  
    addEdge(graph, 1, 2);  
    addEdge(graph, 2, 3);  
  
    printGraph(graph);  
  
    DFS(graph, 2);  
  
    return 0;  
}
```

نوشتن کد جست و جوی عمقی به زبان C++

- کدهای اولیه و تعریف کلاس گراف

این کلاس یک گراف را نشان می دهد. این کلاس سه فیلد دارد:

numVertices: تعداد رأس های گراف.

adjLists: آرایه ای از لیست های مجاورت، که هر لیست مجاورت یک رأس را نشان می دهد.

visited: آرایه ای برای نشان دادن اینکه کدام رأس ها بازدید شده اند.

(صفحه بعد)



```
1
```

```
2#include <iostream>
3#include <list>
4using namespace std;
5
6class Graph {
7    int numVertices;
8    list<int> *adjLists;
9    bool *visited;
10
11    public:
12    Graph(int V);
13    void addEdge(int src, int dest);
14    void DFS(int vertex);
15};
```

• تعریف تابع Graph

این تابع سازنده کلاس Graph است. این تابع یک گراف جدید با تعداد رأس های مشخص شده ایجاد می کند.

(صفحه بعد)



C++

```
Graph::Graph(int vertices) {  
    numVertices = vertices;  
    adjLists = new list<int>[vertices];  
    visited = new bool[vertices];  
}
```

• تعریف تابع addEdge

این تابع یک یال بین دو رأس مشخص شده اضافه می کند. این تابع رأس src را به لیست مجاورت رأس dest اضافه می کند.

C++

```
void Graph::addEdge(int src, int dest) {  
    adjLists[src].push_front(dest);  
}
```

• تعریف تابع DFS

این تابع الگوریتم DFS را از یک رأس شروع مشخص شده اجرا می کند. این تابع مراحل زیر را انجام می دهد:

1. رأس شروع را به عنوان بازدید شده علامت گذاری می کند.
2. رأس شروع را چاپ می کند.
3. به صورت بازگشتی، الگوریتم DFS را برای تمام رأس های مجاور رأس شروع که بازدید نشده اند، اجرا می کند.

C++

```
void Graph::DFS(int vertex) {  
    visited[vertex] = true;  
    list<int> adjList = adjLists[vertex];  
  
    cout << vertex << " ";  
  
    list<int>::iterator i;  
    for (i = adjList.begin(); i != adjList.end(); ++i)  
        if (!visited[*i])  
            DFS(*i);  
}
```

- اجرای دستورات درون تابع main جهت بررسی عملکرد کد نوشته شده

C++

```
int main() {  
    Graph g(4);  
    g.addEdge(0, 1);  
    g.addEdge(0, 2);  
    g.addEdge(1, 2);  
    g.addEdge(2, 3);  
  
    g.DFS(2);  
  
    return 0;  
}
```



نوشتن کد جست و جوی عمقی به زبان Python

- تحلیل کد ها به عهده دانشجو است

(صفحه بعد)

مجید درهم بخش



Python

```
# DFS algorithm
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)

    print(start)

    for next in graph[start] - visited:
        dfs(graph, next, visited)
    return visited

graph = {'0': set(['1', '2']),
        '1': set(['0', '3', '4']),
        '2': set(['0']),
        '3': set(['1']),
        '4': set(['2', '3'])}

dfs(graph, '0')
```




پیچیدگی الگوریتم جست و جوی عمقی

پیچیدگی زمانی الگوریتم DFS به صورت $O(V + E)$ نشان داده می شود، که در آن V تعداد رأس ها و E تعداد یال ها است.

پیچیدگی فضایی الگوریتم $O(V)$ است.

کاربرد های الگوریتم جست و جوی عمقی

- برای یافتن مسیر
- برای آزمایش اینکه نمودار دو طرفه است
- برای یافتن اجزای به شدت متصل یک نمودار
- برای تشخیص چرخه در یک نمودار

مجموعه دراهم بخش

03

کار با گیت (Git) و گیت هاب (GitHub)

در این بخش به بررسی سرویس نسخه زنی گیت و نحوه کار با آن
خواهیم پرداخت

04

مقدمه زبان C++

در این بخش به دستورات پایه و کار با آنها پرداخته و پروژه های مبتنی بر
این زبان را بررسی خواهیم نمود

یادگیری زبان C/C++

C++ یک زبان برنامه نویسی چند منظوره، با هدف کلی، میان سطحی است که معمولاً کامپایل شده، دارای تایپ ایستا و فرم آزاد است. این زبان بر اساس C توسعه یافته و در سال 1979 توسط **بیارن استروستروپ (Bjarne Stroustrup)** ایجاد شده است.

بسیاری از سیستم های عامل، درایور های سیستم، مرورگرها و بازی های امروزی از C++ به عنوان زبان اصلی خود استفاده می کنند که آن را به یکی از محبوب ترین زبان های مورد استفاده تبدیل می کند.

ویژگی های برنامه نویسی C++

سرعت:

از آنجایی که C++ نسخه توسعه یافته ی زبان C است، بخش C آن در سطح بسیار پایینی قرار دارد. این امر باعث افزایش قابل توجهی در سرعت می شود که زبان های سطح بالا مانند پایتون و جاوا نمی توانند آن را ارائه دهند.

نوع داده به صورت ایستا تعیین شده:



C++ یک زبان برنامه نویسی با نوع دهی ایستا است. به این معنی که به کامپایلر اجازه نمی دهد فرض هایی درباره نوع داده بگیرد. برای مثال، عدد 10 با عبارت "10" متفاوت است و باید به صراحت اعلام شود.

از آنجایی که این موارد در زمان کامپایل تعیین می شوند، به کامپایلر کمک می کند تا قبل از اجرای برنامه خطاها و باگ ها را شناسایی کند.

زبان برنامه نویسی چند الگویی (Multi-paradigm)

C++ حداقل از ۷ سبک برنامه نویسی مختلف پشتیبانی می کند و به توسعه دهندگان آزادی انتخاب بر اساس نیازشان می دهد. برخلاف جاوا، برای حل هر مشکلی مجبور نیستید از شی استفاده کنید مگر اینکه ضروری باشد.

برنامه نویسی شی گرا (Object-oriented)

C++ از برنامه نویسی شی گرا پشتیبانی می کند که ساختار ماژولار واضحی برای برنامه ها فراهم می کند. با استفاده از آن در C++، می توانید مشکلات پیچیده را با ایجاد اشیا به مجموعه های کوچکتر تقسیم کنید.

کتابخانه استاندارد (کتابخانه الگوی استاندارد - STL)

استفاده از C++ با استفاده از کتابخانه استاندارد موجود در آن قابل گسترش بیشتر است.



این کتابخانه ها حاوی الگوریتم های کارآمدی هستند که می توانید به طور گسترده در هنگام کدنویسی پروژه های خود استفاده کنید.
این کار مقدار زیادی از تلاش برنامه نویسی را که در غیر این صورت برای اختراع مجدد چرخ هدر می رفت، صرفه جویی می کند.

چرا باید برنامه نویسی C++ را یاد بگیرید؟

1. بی همتا بودن

از آنجایی که C++ در توسعه بازی های مدرن، سیستم های عامل، مرورگرها و موارد دیگر استفاده می شود، می توان گفت که C++ بی همتا است.
بسیاری از شرکت های بزرگ مانند Adobe، Amazon، Facebook و Autodesk از C++ در محصولات خود استفاده می کنند.

2. برای یادگیری معماری داخلی کامپیوتر

با توجه به اینکه C++ یک زبان سطح میانی است، شما کدهایی را خواهید نوشت که مستقیماً با سخت افزار داخلی رایانه تعامل دارند. به همین دلیل، شما درک خواهید کرد که حافظه رایانه چگونه کار می کند و چگونه می توان اطلاعات را ذخیره و بازیابی کرد. این کار مطمئناً دانش شما را در مورد معماری کامپیوتر گسترش می دهد.

3. جامعه فعال و پروژه های متن باز:

C++ دارای یک جامعه حمایتی بزرگ است. انجمن های آنلاین فعال زیادی وجود دارند که اگر در هر کجای فرآیند یادگیری گیر کردید، می توانند به شما کمک کنند.

اگر C++ را بلد باشید، می توانید در پروژه های متن باز بزرگی که بر صدها میلیون نفر تأثیر می گذارد، مشارکت کنید. برخی از پروژه های متن باز بزرگتر که در آنها از برنامه نویسی C++ استفاده می شود، TensorFlow، Electron، OpenCV و غیره هستند.

4. آشنایی با برنامه نویسی:

بسیاری از زبان های برنامه نویسی محبوب بر اساس C (و C++)، که به عنوان مجموعه ای فرعی از برنامه نویسی C با ویژگی های شی گرا در نظر گرفته می شود) ساخته شده اند. بنابراین، اگر C و C++ را بلد باشید، مشکلی برای یادگیری زبان دیگری نخواهید داشت.

5. فرصت های شغلی:

توسعه دهندگان C++ می توانند به طور متوسط 100000 دلار حقوق با فرصت های شغلی surplus انتظار داشته باشند. الزامات مشاغل بیشتر برای توسعه بازی، موتورهای رندر و ایجاد برنامه های کاربردی ویندوز است.



چگونه در C++ بهتر شویم؟

از طریق عمل کردن یاد بگیرید

هر روشی که برای یادگیری C++ انتخاب می کنید، در نهایت به نوشتن برنامه برای تمرین آنچه خوانده اید، ختم می شود. تنها تمرین مداوم شما را به یک برنامه نویس بهتر تبدیل می کند.

به سادگی کد دیگران را کپی نکنید و اجرا نکنید. کمی وقت بگذارید و درباره کاری که کد انجام می دهد فکر کنید. آن را در سیستم خود بازسازی کنید و ببینید چه خطاهایی رخ می دهد و مهمتر از همه، از آنها درس بگیرید.

استانداردهای C++ را دنبال کنید

هنگامی که شروع به برنامه نویسی می کنید، پیروی از یک استاندارد خوب C++ بسیار مهم است. حتی اگر در ابتدای کار آزادی عمل کمتری به شما بدهند، اما با گسترش برنامه و پیشرفت در C++، مفید خواهند بود.

نگاهی به [سوالات متداول استاندارد کد نویسی ISO C++](#)، [استاندارد MISRA C](#)، [استاندارد MISRA C++](#) بیندازید و همچنین [دستورالعمل های هسته](#)

[C++](#) را دنبال کنید.



کد C++ نوشته شده توسط دیگران را بخوانید

به پروژه های منبع باز گیت هاب بپیوندید و کد دیگران را بخوانید. ممکن است در ابتدا با دیدن همه کدهای پروژه احساس غرق شدن کنید. می توانید از Code Whittling استفاده کنید تا با موارد کوچک شروع کنید و در هر لحظه فقط روی یک مورد تمرکز کنید.

شما نه تنها با سبک های کد نویسی دیگران آشنا می شوید، بلکه همچنین درک می کنید که آنها چگونه فکر می کنند.

به جامعه C++ بپیوندید

از دیگران کمک بگیرید. جوامع C++ عالی زیادی وجود دارند که به شما کمک می کنند مشکلات واقعی را حل کنید و مهمتر از همه، به یک توسعه دهنده بهتر تبدیل شوید.

برخی از آنها عبارتند از:

1. StackOverflow - محبوب ترین سایت سوال و جواب برنامه نویسی در وب
2. Codechef - سوالات تمرینی، چالش ها و جامعه بزرگی از برنامه نویسان
3. CodeProject - برای کسانی که کد نویسی می کنند، با مقالات عمیق و جامعه عظیمی از کد نویسان

سخن پایانی:

یادگیری هر زبانی زمان می برد و این مورد برای C++ بیشتر صدق می کند. از آنجایی که این یک زبان سطح بالا نیست، یادگیری C++ در ابتدا می تواند طاقت فرسا باشد و باید برای گذاشتن ساعت های مطالعه هوشمندانه برای یادگیری اصول اولیه آماده باشید.

با این حال، این تجربه شما را در مسیر تبدیل شدن به یک توسعه دهنده عالی بسیار کمک خواهد کرد.

ساختار یک برنامه ساده C/C++

در این بخش به بررسی ساختار ساده برنامه ای به زبان C/C++ خواهیم پرداخت

ساختار یک برنامه ساده C/C++

ساده ترین ساختار کد C++ شامل موارد زیر است:

1. پیش پردازنده ها:

- شامل دستورات `#include` برای کتابخانه های مورد نیاز مانند `iostream`
- تعریف ماکرو ها

2. تابع `main`:

- نقطه شروع برنامه
- شامل دستورات برنامه

3. دستورات:

- شامل عبارات شرطی، حلقه ها، متغیرها، توابع و ...

4. کامنت ها:

- توضیحات برای خوانایی کد

مثال:

C++

```
#include <iostream>

int main() {
    // print "Hello, World!"
    std::cout << "Hello, World!" << std::endl;

    return 0;
}
```

توضیح:

- `#include <iostream>` کتابخانه `iostream` را برای استفاده از `std::cout` وارد می کند.
- `int main()` تابع اصلی برنامه است.
- `std::cout << "Hello, World!" << std::endl;` عبارت `"Hello, World!"` را چاپ می کند.

- `return 0;` نشان می دهد که برنامه با موفقیت اجرا شده است.

نکات:

- هر خط کد باید با `;` به پایان برسد.
- `//` برای نوشتن کامنت در یک خط استفاده می شود.
- `/* ... */` برای نوشتن کامنت در چند خط استفاده می شود.
- به بزرگی و کوچکی حروف در کد دقت کنید.

کلمات کلیدی و شناسه هادر C/C++

در این بخش به بررسی کلمات کلیدی و شناسه ها در زبان C/C++ خواهیم پرداخت



کلمات کلیدی و شناسه ها در C/C++

مجموعه کاراکتر

مجموعه کاراکتر مجموعه ای از الفبا ها، حروف و برخی کاراکتر های ویژه است که در زبان C معتبر هستند.

حروف الفبا (Alphabets)

- حروف بزرگ: A B C X Y Z
 - حروف کوچک: a b c x y z
- C++ الفبای حروف کوچک و بزرگ را به عنوان متغیرها و توابع می پذیرد.

اعداد (Digits)

- 0 1 2 3 4 5 6 7 8 9

کاراکترهای خاص (Special Characters)

مجموعه کاراکتر مجموعه ای از الفبا ها، حروف و برخی کاراکتر های ویژه است که در زبان C معتبر هستند.

کاراکتر های خاص در زبان C/C++				
,	<	>	.	-
(	)	;	\$	:
%	[	]	#	?
'	&	{	}	"
^	!	*	/	
-	\	~	+	

کاراکتر فضای خالی (Space)

فضای خالی، خط جدید، صفحه افقی، بازگشت و جلو رفتن فرم.

کلمات کلیدی (Keywords)

کلمات کلیدی، واژه های از پیش تعریف شده و رزرو شده ای هستند که در برنامه نویسی معانی خاصی برای کامپایلر دارند. کلمات کلیدی بخشی از ساختار زبان برنامه نویسی هستند و نمی توان از آن ها به عنوان شناسه استفاده کرد. برای مثال:

```
int money;
```

در اینجا، int یک کلمه کلیدی است که نشان می دهد money یک متغیر از نوع int (عدد صحیح) است. از آنجایی که C یک زبان حساس به حروف بزرگ و کوچک است، تمام کلمات کلیدی باید با حروف کوچک نوشته شوند. در اینجا لیستی از تمام کلمات کلیدی مجاز در ANSI C آمده است.

کلمات کلیدی زبان C/C++			
alignas	decltype	namespace	struct
alignof	default	new	switch
and	delete	noexcept	template



and_eq	do	not	this
asm	double	not_eq	thread_local
auto	dynamic_cast	nullptr	throw
bitand	else	operator	true
bitor	enum	or	try
bool	explicit	or_eq	typedef
break	export	private	typeid
case	extern	protected	typename
catch	false	public	union
char	float	register	unsigned



char16_t	for	reinterpret_cast	using
char32_t	friend	return	virtual
class	goto	short	void
compl	if	signed	volatile
const	inline	sizeof	wchar_t
constexpr	int	static	while
const_cast	long	static_assert	xor
continue	mutable	static_cast	xor_eq

نکته: از آنجایی که C++ یک زبان حساس به حروف بزرگ و کوچک است، همه کلمات کلیدی باید با حروف کوچک نوشته شوند.



تمام این کلمات کلیدی، نحوه استفاده و کاربرد آنها در بخش های مربوطه مورد بحث قرار خواهند گرفت. با این حال، اگر بدون عمیق شدن، فقط می خواهید مرور کلی ای بر این کلمات داشته باشید، می توانید به سرفصل [فهرست تمام کلمات کلیدی در C/C++](#) مراجعه کنید.

شناسه ها (Identifiers) در زبان C/C++

شناساگر (Identifier) به نامی اشاره می کند که به موجودیت هایی مانند متغیرها، توابع، ساختارها و غیره داده می شود. شناساگر ها باید منحصر به فرد باشند. آن ها برای دادن یک نام منحصر به فرد به یک موجودیت ایجاد می شوند تا بتوان آن را در طول اجرای برنامه شناسایی کرد. برای مثال:

```
int money;  
double accountBalance;
```

در اینجا، money و accountBalance شناسه ها هستند.

همچنین به خاطر داشته باشید که نام های شناسه باید با کلمات کلیدی متفاوت باشند. شما نمی توانید از int به عنوان شناسه استفاده کنید زیرا int یک کلمه کلیدی است.

قوانین نامگذاری شناسه ها

1. یک شناسه معتبر می تواند شامل حروف (بزرگ و کوچک)، ارقام و آندرلاین (_) باشد.
2. حرف اول یک شناسه باید یا یک حرف یا آندرلاین (_) باشد.
3. شما نمی توانید از کلمات کلیدی مانند `while`، `int` و غیره به عنوان شناسه استفاده کنید.
4. هیچ محدودیتی برای طول یک شناسه وجود ندارد. با این حال، اگر شناسه بیش از 31 کاراکتر باشد، ممکن است در برخی کامپایلر ها با مشکل مواجه شوید.

مطابق با قانون قبلی، شما برای هر شناسه ای میتوانید هر نامی را که دوست دارید در نظر بگیرید. با این حال، پیشنهاد میشود که تا حد امکان از اسامی معنادار برای شناسه ها استفاده کنید که مفهوم آنها را به خوبی مشخص کند.

مثال هایی از شناسه های خوب و بد

(صفحه بعد)



شناسه اشتباه	شناسه بد	شناسه خوب
Total points	T_points	totalPoint
1list	list_1	list1
float	n_float	floatNumber

رایگان برای آموزش شخصی - کپی و بازنشر بدون مجوز ممنوع



متغیرها، داده ثابت و ثوابت در C++

در این بخش به بررسی کلی متغیرها، داده ثابت و نوع ثابت در زبان C/C++ خواهیم پرداخت، در آینده این بخش ها با **جزئیات** بررسی خواهند شد.

متغیرها، داده ثابت و ثوابت در C++

متغیرها در C++

در برنامه نویسی، یک متغیر مانند یک مخزن (محل ذخیره سازی) برای نگهداری داده ها عمل می کند.

برای مشخص کردن این محل ذخیره سازی، باید به هر متغیر یک نام منحصر به فرد (شناسه) اختصاص داد. برای مثال:

```
int age = 14;
```

در اینجا، age یک متغیر از نوع داده ی int (عدد صحیح) است و ما مقدار عددی 14 را به آن اختصاص داده ایم.

توجه: نوع داده ی int نشان می دهد که این متغیر فقط می تواند اعداد صحیح را در خود نگه دارد. به همین ترتیب، اگر نیاز به ذخیره سازی اعداد اعشاری و نماد داشته باشیم، می توانیم از نوع داده ی double استفاده کنیم.

در آموزش های بعدی، به طور کامل در مورد تمام انواع داده ها صحبت خواهیم کرد.

مقادیر یک متغیر قابل تغییر هستند، به همین دلیل به آن ها متغیر می گویند.



```
int age = 14;    // age is 14
age = 17;        // age is 17
```

نحوه اعلام متغیر ها

اعلام متغیر به طور معمول به صورت زیر است:

```
// اعلام یک متغیر:
type variable_name;

// اعلام چند متغیر:
type variable1_name, variable2_name, variable3_name;
```

نام یک متغیر می تواند از حروف الفبا (اعم از بزرگ و کوچک)، اعداد و کاراکتر آندرلاین "_" تشکیل شده باشد. با این حال، نام نباید با یک عدد شروع شود.

int age = 14;

← value (مقدار)

↑ ↑

data type (نوع داده) variable_name (نام متغیر)



RAM

در شکل بالا:

datatype: نوع داده ای که می تواند در این متغیر ذخیره شود.

variable_name: نامی که به متغیر داده شده است.

value: مقدار اولیه ذخیره شده در متغیر است.

مثال:

```
// Declaring float variable
float simpleInterest;

// Declaring integer variable
int time, speed;

// Declaring character variable
char var;
```

ما میتوانیم مقدار متغیر ها را در هنگام اعلام آن ها تنظیم نماییم، به مثال زیر توجه نمایید:

```
int a = 50, b = 100; //declaring 2 variable of integer type
float f = 50.8; //declaring 1 variable of float type
char c = 'Z'; //declaring 1 variable of char type
```

قوانین نامگذاری متغیر ها

1. نام یک متغیر فقط می تواند شامل حروف الفبا، اعداد و آندرلاین (_) باشد.
2. نام یک متغیر نمی تواند با یک عدد شروع شود.
3. ترجیح داده می شود که نام های متغیر با یک حرف کوچک شروع شوند. برای مثال، استفاده از name بهتر از Name است.
4. نام یک متغیر نمی تواند یک کلمه کلیدی باشد. برای مثال، int یک کلمه کلیدی برای معرفی اعداد صحیح است.
5. نام یک متغیر می تواند با آندرلاین (_) شروع شود. با این حال، این کار توصیه نمی شود.

نکته: ما باید سعی کنیم نام های معناداری برای متغیرها انتخاب کنیم. برای مثال، first_name یک نام متغیر بهتر از fn است.

تفاوت بین تعریف (definition) و اعلام (declaration) متغیر

اعلام متغیر به بخشی گفته می شود که در آن یک متغیر قبل از اولین استفاده، اعلام یا معرفی می شود. تعریف متغیر بخشی است که به متغیر یک موقعیت حافظه و یک مقدار اختصاص داده می شود. در بیشتر مواقع، اعلام و تعریف متغیر با هم انجام می شوند.

برای درک بهتر، برنامه C++ زیر را ببینید:

C++

```
int main()
{
    // this is declaration of variable a
    int a;

    // this is initialisation of a
    a = 10;

    // this is definition = declaration + initialization
    int b = 20;
```



C++



```
// declaration and definition
// of variable 'a123'
char a123 = 'a';

// This is also both declaration and definition
// as 'c' is allocated memory and
// assigned some garbage value.
float c;

// multiple declarations and definitions
int _c, _d45, e;

// Let us print a variable
cout << a123 << endl;

return 0;
}
```



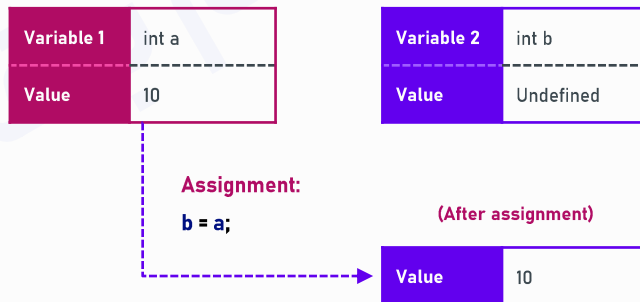
کپی کردن متغیر ها در یکدیگر

ما میتوانیم مقدار متغیری را در متغیر دیگر کپی نماییم، این عمل با عملگر انتصاب (=) در برنامه انجام میشود، به مثال های زیر توجه نمایید:

مثال 1:

```
int a = 10;  
int b = a;
```

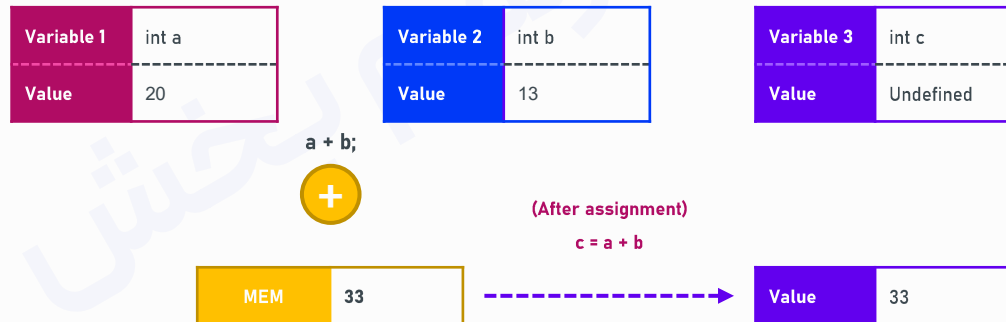
در این مثال مقدار متغیر a در متغیر b کپی می شود، به شکل زیر توجه کنید:



مثال 2:

```
int a = 20;  
int b = 13;  
  
int c = a + b;
```

در این مثال مقدار متغیر a و b با یکدیگر در حافظه ای پنهان جمع شده و در متغیر c جمع میشوند، به شکل زیر توجه کنید:



در آینده مثال های بیشتری از کپی کردن انواع داده ها را بررسی خواهیم کرد.



داده ثابت (literal) در C++

لیترال ها (داده ثابت) داده هایی هستند که برای نمایش مقادیر ثابت استفاده می شوند. آن ها می توانند به طور مستقیم در کد مورد استفاده قرار گیرند. برای مثال: 1، 2.5، 'c' و غیره.

در اینجا، 2.5 و 'c' لیترال هستند. چرا؟ چون شما نمی توانید مقادیر متفاوتی به این عبارات اختصاص دهید.

در اینجا لیستی از انواع مختلف لیترال ها در برنامه نویسی C++ آمده است.

1. اعداد صحیح (Integer)

یک عدد صحیح یک مقدار عددی (مرتبط با اعداد) است که هیچ بخش اعشاری یا نمایی ندارد. در برنامه نویسی C سه نوع عدد صحیح وجود دارد:

- ددهی (پایه 10)
- هشت تایی (پایه 8)
- شانزده دهی (پایه 16)

مثال:

Decimal: 0, -9, 22

Octal: 021, 077, 033

Hexadecimal: 0x7f, 0x2a, 0x521

توجه داشته باشید که در برنامه نویسی C++، اعداد هشت تایی با یک صفر (0) و اعداد شانزده دهی با 0x شروع می شوند.

2. اعداد اعشاری (Floating-point)

یک عدد اعشاری، یک مفهوم عددی است که یا به صورت کسری یا به صورت نمایی نمایش داده می شود. برای مثال:

-2.0

0.0000234

-0.22E-5

نکته: E-5 برابر با 10^{-5} است

3. کاراکترها (Characters)

یک کاراکتر لیترال با قرار دادن یک کاراکتر واحد در داخل کوتیشن (' ') ایجاد می شود. به عنوان مثال: 'a'، 'm'، 'F'، '2'، '}' و غیره.

4. توالی خروج (Escape Sequences)

بعضی اوقات، نیاز است تا از کاراکتر هایی که نمی توانند تایپ شوند یا در برنامه نویسی C++ معنی خاصی دارند استفاده کنیم. به عنوان مثال، Newline (Enter)، Tab، علامت سوال و غیره

به منظور استفاده از این کاراکتر ها، از توالی خروج استفاده می شود.

توالی خروج	کاراکتر
\b	Backspace
\f	Form feed
\n	خط جدید (Newline)



Return	\r
Horizontal tab	\t
Vertical tab	\v
Backslash	\\
Single quotation mark	\'
Double quotation mark	\"
Question mark	\?
Null Character	\0



5. رشته های لفظی

رشته ی لفظی، دنباله ای از کاراکترها است که در بین دو علامت دابل کوتیشن (") قرار گرفته است. به عنوان مثال:

رشته ثابت	"good"
رشته ثابت خالی	""
رشته ثابت با 6 فضای خالی	" "
رشته ثابت با کاراکتر x	"x"
نمایش متن به همراه رفتن به خط بعد	"Earth is round\n"

ما در آموزش رشته ++C، در مورد رشته ها به طور کامل یاد خواهیم گرفت.

مقادیر ثابت در C++

در C++ می توانیم متغیرهایی ایجاد کنیم که مقدار آن ها قابل تغییر نباشد. برای این کار، از کلمه کلیدی `const` استفاده می کنیم. در اینجا یک مثال آورده شده است:

```
const int LIGHT_SPEED = 299792458;  
LIGHT_SPEED = 2500 // Error! LIGHT_SPEED is a constant.
```

در اینجا، ما از کلمه کلیدی `const` برای تعریف یک ثابت به نام `LIGHT_SPEED` استفاده کرده ایم. اگر سعی کنیم مقدار `LIGHT_SPEED` را تغییر دهیم، با خطا مواجه خواهیم شد.

روش های تعریف ثابت در C++

ما می توانیم با استفاده از سه روش زیر در C++ ثابت ها را ایجاد نماییم:

- استفاده از کلیدواژه `const`
- استفاده از کلیدواژه `constexpr`
- استفاده از پیش پردازنده `#define`



تعریف متغیرهای ثابت با استفاده از `const`

برای تعریف یک ثابت در زبان برنامه نویسی سی از کلیدواژه `const` استفاده می کنیم. این کلیدواژه که همچنین به عنوان یک شناسه نوع ثابت شناخته می شود، در ابتدای تعریف متغیر قرار می گیرد تا آن متغیر را به عنوان یک ثابت اعلام کند.

تعریف متغیر ثابت به طور معمول به صورت زیر است:

```
const data_type var_name = value;
```

const int var = 5; ← value (مقدار)

کلمه کلیدی ↑
data type (نوع داده) ↑
variable_name (نام متغیر) ↑

مثال:

C++

```
int main()
{
    // defining integer constant using const keyword
    const int int_const = 25;

    // defining character constant using const keyword
    const char char_const = 'A';

    // defining float constant using const keyword
    const float float_const = 15.66;

    return 0;
}
```

نکته قابل توجه این است که باید متغیرهای ثابت را در زمان اعلام آن ها مقداردهی کنیم. در غیر این صورت، متغیر مقداری بی معنی (garbage value) را ذخیره می کند و ما دیگر نمی توانیم آن را تغییر دهیم. تصویر زیر نمونه هایی از تعاریف نادرست و صحیح متغیرهای ثابت را نشان می

نحوه تعریف ثابت ها با استفاده از const

```
const int var;
```

×

```
const int var;
```

```
var = 5;
```

×

```
const int var = 5;
```

✓

انواع ثابت ها در C++

نوع ثابت با نوع داده متغیرها یکسان است. در زیر لیستی از انواع ثابت ها آمده است:

- ثابت عدد صحیح (integer)
- ثابت کاراکتر (character)
- ثابت اعشاری float
- ثابت اعشاری double



- ثابت آرایه (array)

- ثابت ساختار (structure)

برای تعریف یک ثابت، تنها کافیسست کلیدواژه const را در ابتدای اعلام متغیر اضافه کنید.

نکته: در آینده روش های تعریف ثابت ها و استفاده از آن ها در بخش های مختلف با جزئیات بررسی خواهند شد

مجموعه دارهفم بخش



انواع داده (Data Types) در C++

در این بخش به بررسی انواع داده در زبان C/C++ خواهیم پرداخت

انواع داده (Data Types) در C++

در زبان C++، تمامی متغیرها هنگام تعریف، از یک نوع داده مشخص برای محدود کردن نوع اطلاعاتی که می توانند ذخیره کنند، استفاده می کنند. بنابراین، می توان گفت نوع داده برای راهنمایی متغیرها درباره نوع داده ای که می توانند نگه دارند، به کار می رود. هر بار که یک متغیر در C++ تعریف می شود، کامپایلر بر اساس نوع داده ای که برای آن اعلام شده، میزان مشخصی حافظه برای آن متغیر اختصاص می دهد. هر نوع داده نیازمند مقدار متفاوتی حافظه است. برای مثال:

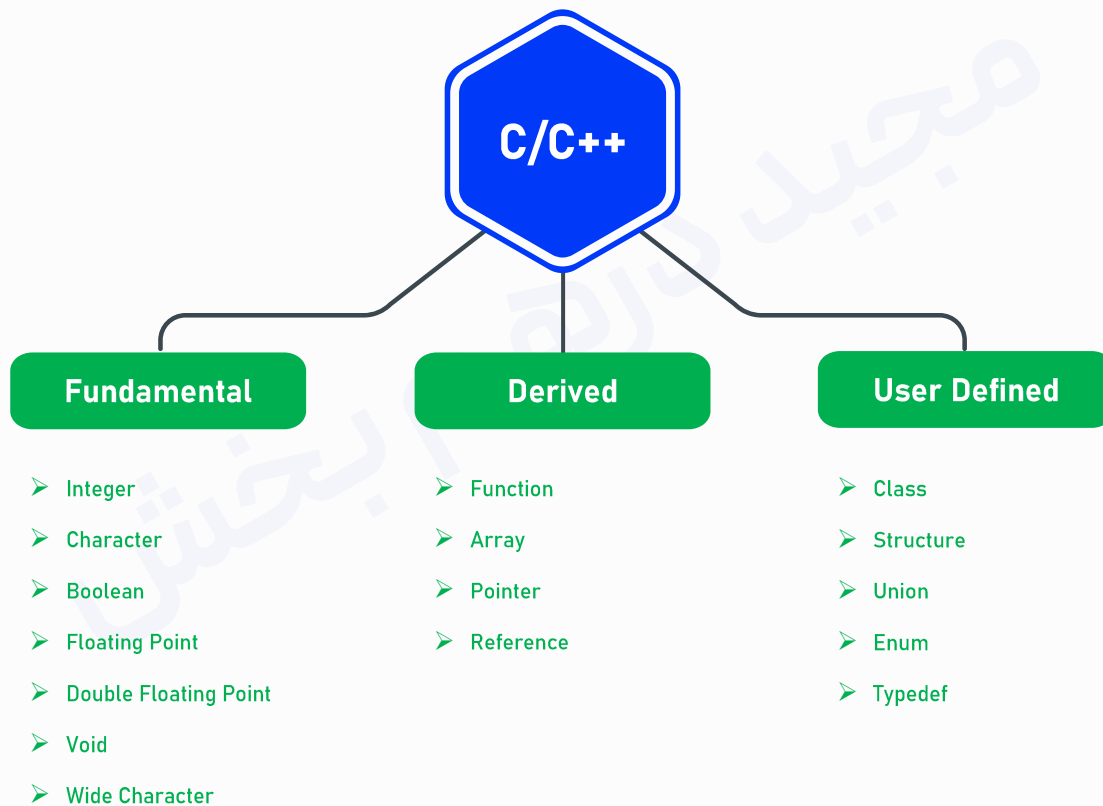
```
int age = 13;
```

در اینجا، age یک متغیر از نوع int است. یعنی این متغیر فقط می تواند اعداد صحیح را ذخیره کند که اندازه آن ها 2 یا 4 بایت است.

زبان C++ از انواع داده زیر پشتیبانی می کند:

1. نوع داده اولیه (Primary) یا داخلی (Built-in) یا بنیادی (Fundamental)
2. انواع داده های مشتق شده (Derived)
3. انواع داده های تعریف شده توسط کاربر (User-defined)

انواع داده در C/C++



مثال دوم:

```
float score = 18.25;
```

در اینجا، score یک متغیر از نوع float است. یعنی این متغیر می تواند اعداد اعشاری را ذخیره کند که اندازه آن ها 4 بایت است.

انواع داده های بنیادی (Fundamental) C++

جدول زیر انواع داده های بنیادی، معانی آنها و اندازه هایشان (بر حسب بایت) را نشان می دهد:

نوع داده	معنی	سایز (بایت)
int	Integer	2 or 4
float	Floating-point	4
double	Double Floating-point	8



char	Character	1
wchar_t	Wide Character	2
bool	Boolean	1
void	Empty	0

اکنون، اجازه دهید در مورد این انواع داده های بنیادی با جزئیات بیشتری صحبت کنیم.

1. عدد صحیح (int) در C++

کلمه کلیدی "int" برای نشان دادن اعداد صحیح استفاده می شود.

اندازه آن معمولا 4 بایت است. به این معنی که می تواند مقادیری از -2147483648 تا 2147483647 را ذخیره کند.

برای مثال،



```
int salary = 85000;
```

2. عدد اعشاری float و double در C++

- Float و double برای ذخیره سازی اعداد نقطه شناور (اعشاری و توانی) استفاده می شوند.
- اندازه ی float برابر با 4 بایت و اندازه ی double برابر با 8 بایت است. بنابراین، double دو برابر دقت float را دارد. برای بررسی دقیق تر به سر فصل float و double در C++ مراجعه کنید.
- برای مثال،

```
float area = 64.74;  
double volume = 134.64534;
```

- همانطور که در بالا ذکر شد، این دو نوع داده همچنین برای اعداد توانی استفاده می شوند. به عنوان مثال،

```
double distance = 45E12    // 45E12 is equal to 45*10^12
```




3. کاراکتر (char) در C++

- کلمه کلیدی char برای کاراکترها استفاده می شود.
- اندازه آن 1 بایت است.
- کاراکترها در C++ داخل تک کوتیشن ' ' قرار می گیرند.
- برای مثال:

```
char test = 'h';
```

توجه: در زبان C++، به جای خود کاراکتر، یک مقدار عددی در متغیر char ذخیره می شود. برای بررسی جزئیات به سرفصل [کاراکترها در C++](#) مراجعه کنید.

4. کاراکتر عریض (wchar_t) در C++

- نوع داده کاراکتر عریض یا wchar_t شبیه به نوع داده char است، با این تفاوت که اندازه آن به جای 1 بایت، 2 بایت است.
- از آن برای نمایش کاراکترهایی استفاده می شود که برای ذخیره سازی به حافظه بیشتری نسبت به یک کاراکتر معمولی char نیاز دارند.
- برای مثال:



```
wchar_t test = L'ד' //storing Hebrew character;
```

به حرف L قبل از علامت کوتیشن توجه کنید.

نکته: دو نوع کاراکتر با اندازه ثابت دیگر به نام های char16_t و char32_t نیز وجود دارند که در C++11 معرفی شده اند.

5. بولین (bool) در C++

- نوع داده bool فقط دو مقدار ممکن را می تواند داشته باشد: true (درست) یا false (غلط).
- بولین ها در عبارات شرطی و حلقه ها استفاده می شوند (که در فصل های بعدی خواهیم آموخت).
- برای مثال:

```
bool cond = false;
```

نکته: این نوع داده تنها در C++ به صورت پایه وجود دارد و در زبان C یا باید از نسخه های جدید زبان C استفاده نمود و یا از داده های اصلاح شده

آن

6. void در C++

- کلمه کلیدی void نشان دهنده عدم وجود داده است. به معنای "هیچ" یا "بدون مقدار" می باشد.
- زمانی که توابع و اشاره گرها را یاد بگیریم، از void استفاده خواهیم کرد.

نکته: نمی توانیم متغیرهایی از نوع void تعریف کنیم.

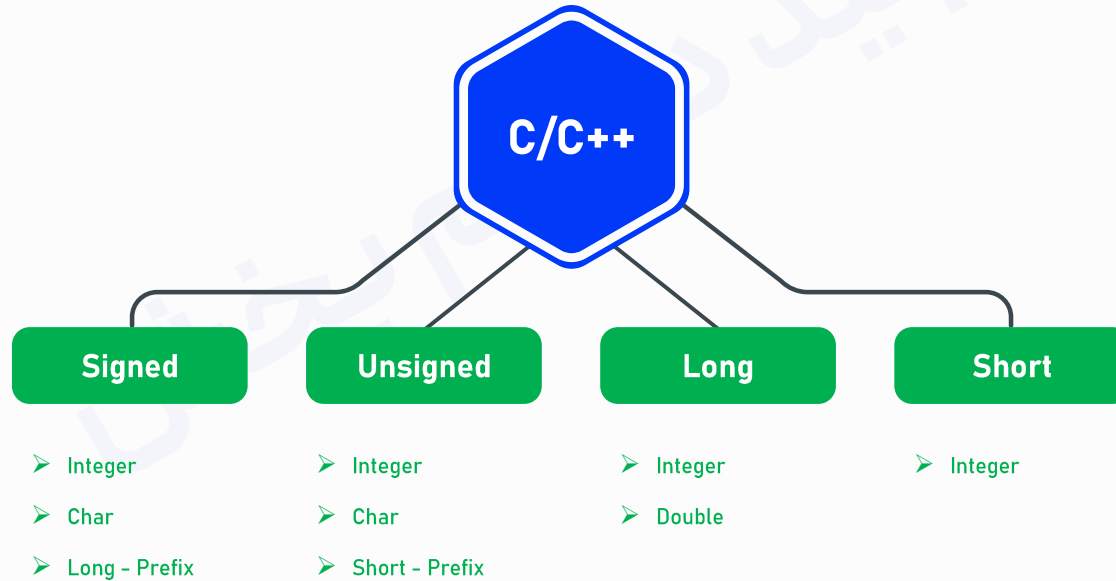
اصلاح کننده های نوع C++

ما می توانیم برخی از انواع داده های بنیادی را با استفاده از اصلاح کننده های نوع اصلاح کنیم. 4 نوع اصلاح کننده در C++ وجود دارد. آن ها عبارت اند از:

- signed
- unsigned
- short
- long

می توانیم با استفاده از تعدیل کننده های بالا، انواع داده های زیر را تغییر دهیم:

- int
- double
- char



لیست داده های اصلاح شده C++

نوع داده	سایز (بایت)	نوع استفاده
signed int	4	برای اعداد صحیح استفاده می شود (معادل int)
unsigned int	4	فقط می تواند اعداد صحیح مثبت را ذخیره کند
short	2	برای اعداد صحیح کوچک استفاده می شود (محدوده -32768 تا 32767)
unsigned short	2	برای اعداد صحیح مثبت کوچک استفاده می شود (محدوده 0 تا 65535)
long	حداقل 4	برای اعداد صحیح بزرگ (معادل long int) استفاده می شود
unsigned long	4	برای اعداد صحیح مثبت بزرگ یا 0 (معادل unsigned long int) استفاده می شود
long long	8	برای اعداد صحیح بسیار بزرگ (معادل long long int) استفاده می شود.



unsigned long long	8	برای اعداد صحیح مثبت بسیار بزرگ یا 0 (معادل unsigned long long) استفاده می شود (int)
long double	12	برای اعداد اعشاری بزرگ استفاده می شود
signed char	1	برای کاراکترها استفاده می شود (محدوده تضمینی -127 تا 127)
unsigned char	1	برای کاراکترها استفاده می شود (محدوده 0 تا 255)

بیا ببینیم چند نمونه را.

```
long b = 4523232;  
long int c = 2345342;  
long double d = 233434.56343;  
short d = 3434233; // Error! out of range  
unsigned int a = -5; // Error! can only store positive numbers or 0
```

انواع داده های مشتق شده (Derived Data Types)

انواع داده هایی که از انواع داده های بنیادی مشتق می شوند، انواع مشتق شده نام دارند. برای مثال، آرایه ها، اشاره گرها، انواع تابع، ساختارها و غیره از انواع داده های مشتق شده هستند.

- نوع شمارشی (Enumerated)
- نوع مختلط (Complex)

در آموزش های بعدی، درباره این نوع داده های مشتق شده بیشتر خواهیم آموخت.

بدست آوردن سایز داده و متغیرها

بدست آوردن سایز متغیر با استفاده از `sizeof()`:

در زبان برنامه نویسی C/C++، اندازه انواع داده ها به معماری سیستم وابسته است، بنابراین نمی توان یک اندازه کلی برای آنها تعریف کرد. به همین دلیل، زبان C/C++ عملگر `sizeof()` را برای بررسی اندازه انواع داده ها در اختیار برنامه نویس قرار می دهد. ساختار این دستور به صورت زیر است:

- **روش اول:** قرار دادن نوع داده درون پرانتز ساختار `sizeof()`



`sizeof` (data type)

در این روش نوع داده درون پرانتز قرار میگیرد، به مثال زیر توجه کنید:

```
cout << "No of Bytes taken up by char is " << sizeof(char) << endl;
cout << "No of Bytes taken up by int is " << sizeof(int) << endl;
cout << "No of Bytes taken up by float is " << sizeof(float) << endl;
cout << "No of Bytes taken up by double is " << sizeof(double) << endl;
cout << "No of Bytes taken up by long is " << sizeof(long) << endl;
```

خروجی برنامه:

```
No of Bytes taken up by char is 1
No of Bytes taken up by int is 4
No of Bytes taken up by float is 4
No of Bytes taken up by double is 8
No of Bytes taken up by long is 8
```


• روش دوم: قرار دادن نام متغیر درون پرانتز ساختار sizeof()

sizeof (expression)

در این روش نام داده درون پرانتز قرار میگیرد، به مثال زیر توجه کنید:

```
int a;  
float b;  
char g;  
cout << "No of Bytes taken up by a is " << sizeof(a) << endl;  
cout << "No of Bytes taken up by b is " << sizeof(b) << endl;  
cout << "No of Bytes taken up by g is " << sizeof(g) << endl;
```

خروجی برنامه:

```
No of Bytes taken up by a is 4  
No of Bytes taken up by b is 4  
No of Bytes taken up by g is 1
```



در آینده درباره این دستور بیشتر صحبت خواهیم کرد.

روش های بدست آوردن محدوده متغیر ها

- روش اول: محاسبه دستی محدوده متغیر ها

در این روش ما از دو رابطه زیر برای بدست آوردن محدوده متغیر ها استفاده خواهیم نمود:

1. متغیر های نوع signed:

$$\text{Min Value} = -2^{n-1}$$

$$\text{Max Value} = 2^{n-1} - 1$$

نکته: در این رابطه n تعداد بیت های متغیر است، که میتوان از رابطه $8 * \text{sizeof}()$ آن را بدست آورد

مثال: محاسبه محدوده متغیر short int

$$\text{Min Value} = -2^{16-1} = -32768$$

$$\text{Max Value} = 2^{16-1} - 1 = 32767$$

2. متغیرهای نوع unsigned:

Min Value = 0

Max Value = $2^n - 1$

نکته: در این رابطه n تعداد بیت های متغیر است، که میتوان از رابطه $8 * \text{sizeof}()$ آن را بدست آورد

مثال: محاسبه محدوده متغیر unsigned short int

Min Value = 0

Max Value = $2^{16} - 1 = 65535$

• **روش دوم:** استفاده از فایل هدر `<limits.h>`

فایل هدر `<limits.h>` طوری تعریف شده که محدوده انواع داده های پایه را مشخص کند. مقادیر مثبت بدون علامت (unsigned) حداقل مقدار صفر را دارند. بنابراین، هیچ ثابت ماکروئی برای حداقل مقدار unsigned تعریف نشده است.

لیست ماکروهای این فایل هدر در جدول زیر آمده است:

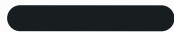


نام	تعریف
CHAR_MIN	حداقل مقدار نوع char
CHAR_MAX	حداکثر مقدار نوع char
SCHAR_MIN	حداقل مقدار نوع signed char
SCHAR_MAX	حداکثر مقدار نوع signed char
UCHAR_MAX	حداکثر مقدار نوع unsigned char
CHAR_BIT	تعداد بیت های نوع char
MB_LEN_MAX	تعداد بایت های نوع char چنتایی
SHRT_MIN	حداقل مقدار نوع short int





SHRT_MAX	حداکثر مقدار نوع short int
USHRT_MAX	حداکثر مقدار نوع unsigned short int
INT_MIN	حداقل مقدار نوع int
INT_MAX	حداکثر مقدار نوع int
UINT_MAX	حداکثر مقدار نوع unsigned int
LONG_MIN	حداقل مقدار نوع long int
LONG_MAX	حداکثر مقدار نوع long int
ULONG_MAX	حداکثر مقدار نوع unsigned long int
LLONG_MIN	حداقل مقدار نوع long long int





LLONG_MAX	حداکثر مقدار نوع long long int
ULLONG_MAX	حداکثر مقدار نوع unsigned long long int

مقدار واقعی به سیستم خاص و پیاده سازی کتابخانه بستگی دارد، اما باید محدودیت های این نوع داده ها در پلتفرم هدف را منعکس کند. LLONG_MIN, LLONG_MAX و ULLONG_MAX برای کتابخانه هایی که با استاندارد C سال 1999 یا بالاتر (که فقط استاندارد C++ از سال 2011 به بعد را شامل می شود: C++11) مطابقت دارند، تعریف شده اند.

مثال:

```
cout << "Size of char : " << sizeof(char) << " byte" << endl;
cout << "char minimum value: " << CHAR_MIN << endl;
cout << "char maximum value: " << CHAR_MAX << endl;
```

Size of char : 1 byte
char minimum value: -128
char maximum value: 127

تمرین های بخش انواع داده

1. متغیر هایی با نام و نوع داده مناسب (با در نظر گرفتن سایز) برای موارد زیر اعلام نمایید:

- معدل
- دما
- رطوبت
- تعداد اعضا
- حجم صدا

1. متغیر هایی با نام و نوع داده مناسب (با در نظر گرفتن سایز) برای موارد زیر تعریف (اعلام و مقدار دهی) نمایید:

- معدل
- دما
- رطوبت
- تعداد اعضا
- حجم صدا

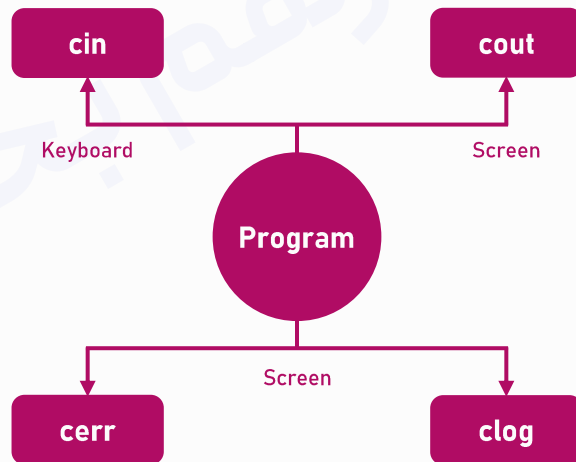
ورودی / خروجی پایه (Basic I/O) در C++

در این بخش به بررسی کلی ورودی / خروجی پایه در زبان C/C++ خواهیم پرداخت، همچنین در سرفصلی جداگانه دستورات و فایل های هدر موجود با جزئیات بیشتری بررسی خواهند شد

ورودی / خروجی پایه (Basic I/O) در C++

C++ دارای کتابخانه هایی است که روش های مختلفی برای کار با ورودی و خروجی را در اختیار ما قرار می دهد. در C++، ورودی و خروجی به صورت توالی ای از بایت ها یا به زبان عامیانه تر، **جریان (stream)** انجام می شود.

- **جریان ورودی (Input Stream):** اگر جهت جریان بایت ها از دستگاه (مثلا کیبورد) به حافظه اصلی باشد، این فرایند ورودی نامیده می شود.
- **جریان خروجی (Output Stream):** اگر جهت جریان بایت ها برعکس باشد، یعنی از حافظه اصلی به دستگاه (صفحه نمایش)، این فرایند خروجی نامیده می شود.





در C++، فایل های هدر زیر برای عملیات ورودی / خروجی استفاده می شوند:

- 
- 
1. **iostream**: این فایل اختصار "standard input-output stream" است و شامل تعریف آبجکت هایی مانند `cin`، `cout`، `cerr` و غیره می باشد.
 2. **iomanip**: این فایل اختصار "input-output manipulators" است. روش های اعلام شده در این فایل برای دستکاری جریان ها استفاده می شوند.
این فایل شامل تعریف `setw`، `setprecision` و غیره می باشد.
 3. **fstream**: این فایل هدر عمدتاً جریان فایل را توصیف می کند. از این فایل برای مدیریت داده هایی که از یک فایل به عنوان ورودی خوانده می شوند یا داده هایی که به عنوان خروجی در فایل نوشته می شوند، استفاده می شود.
 4. **bits/stdc++**: این فایل هدر شامل کل کتابخانه استاندارد است. در مسابقات برنامه نویسی، استفاده از این فایل ایده خوبی است، زمانی که می خواهید زمان تلف شده در کارهای معمولی را کاهش دهید؛ به خصوص زمانی که رتبه شما به زمان حساس است.

در C++، پس از فایل های هدر، اغلب از عبارت `using namespace std` استفاده می کنیم. دلیل آن این است که همه تعاریف کتابخانه استاندارد داخل فضای نام `std` قرار دارند. از آنجایی که توابع کتابخانه در محدوده عمومی تعریف نشده اند، بنابراین برای استفاده از آنها از فضای نام `std` استفاده می کنیم. این کار باعث می شود که لازم نباشد در هر خط عبارت `std::` را بنویسیم (مثلاً `std::cout` و غیره).



دو آبجکت `cin` و `cout` از کلاس `iostream` به طور مکرر به ترتیب برای چاپ خروجی و گرفتن ورودی استفاده می شوند. این دو، اساسی ترین روش های گرفتن ورودی و چاپ خروجی در C++ هستند. برای استفاده از `cin` و `cout` در C++ باید فایل هدر `iostream` را در برنامه وارد کنید.

به مثال زیر توجه کنید:

```
#include <iostream>
using namespace std;

int main() {
    return 0;
}
```

در این مثال ما با استفاده از دستور `#include <iostream>` فایل هدر `iostream` را به برنامه خود اضافه کرده ایم.

در ادامه به بررسی کلی دستورات موجود در این فایل هدر می پردازیم، در آینده این فایل هدر با جزئیات بررسی خواهد شد.

خروجی C++

در C++، دستور cout خروجی فرمت بندی شده را به دستگاه های خروجی استاندارد، مانند صفحه نمایش ارسال می کند. ما از شی cout همراه با عملگر << برای نمایش خروجی استفاده می کنیم.

جهت داده از حافظه به خروجی است

```
cout << var_name;
```

دستور خروجی

متغیر برای نمایش

در این دستور جهت داده از حافظه (متغیر) به خروجی (نمایشگر) است پس برای انتقال داده از عملگر << استفاده میشود

به مثال های زیر توجه کنید:

مثال 1: خروجی رشته

```
#include <iostream>
using namespace std;

int main() {
    // prints the string enclosed in double quotes
    cout << "This is C++ Programming";
    return 0;
}
```

خروجی

This is C++ Programming

این برنامه چطور کار می کند؟

ما ابتدا فایل هدر `iostream` را وارد می کنیم که به ما امکان نمایش خروجی برنامه را می دهد.

شی `cout` در درون فضای نام `std` تعریف شده است. برای استفاده از فضای نام `std`، ما از دستور `using namespace std` استفاده کرده ایم.



هر برنامه C++ با تابع `main()` شروع می شود. اجرای کد از ابتدای تابع `main()` آغاز می گردد.

`cout` شیئی است که رشته ی درون دابل کوتیشن " " را چاپ می کند. بعد از آن از عملگر << استفاده شده است.

`return 0;` وضعیت خروج" تابع `main()` است. برنامه با این دستور به پایان می رسد، اما این دستور الزامی نیست.

توجه: اگر دستور `using namespace std;` را وارد نکنیم، به جای `cout` باید از `std::cout` استفاده کنیم.

این روش ترجیح داده می شود زیرا استفاده از فضای نام `std` می تواند مشکلات بالقوه ای ایجاد کند.

با این حال، ما در آموزش های خود از فضای نام `std` استفاده کرده ایم تا کدها قابل خواندن تر باشند.

به کد زیر دقت کنید:

```
#include <iostream>

int main() {
    // prints the string enclosed in double quotes
    std::cout << "This is C++ Programming";
    return 0;
}
```

مثال 2: خروجی اعداد و کاراکترها

برای چاپ متغیرهای عددی و کاراکتری، از شی همانند cout استفاده می کنیم، اما بدون استفاده از کوتیشن.

```
#include <iostream>
using namespace std;

int main() {
    int num1 = 70;
    double num2 = 256.783;
    char ch = 'A';

    cout << num1 << endl;    // print integer
    cout << num2 << endl;    // print double
    cout << "character: " << ch << endl;    // print char
    return 0;
}
```

خروجی



70

256.783

character: A

عبارت کنترل گر endl برای درج یک خط جدید استفاده می شود. به همین دلیل است که هر خروجی در یک خط جدید نمایش داده می شود. اگر بخواهیم متغیرها، رشته ها و غیره مختلف را در یک دستورالعمل چاپ کنیم، می توان از عملگر << چندین بار استفاده کرد. به عنوان مثال:

```
cout << "character: " << ch << endl;
```

دستور cout دارای توابع عضوی بوده که میتواند نحوه نمایش خروجی را کنترل نماید، این توابع عبارت اند از:

- `cout.write(char str, int n)`: به تعداد n کاراکتر از رشته str را در خروجی چاپ می کند.
- `cout.put(char &ch)`: کاراکتر ذخیره شده در متغیر ch را چاپ می کند.
- `cout.precision(int n)`: هنگام استفاده از مقادیر اعشاری، دقت نمایش اعشار را به n رقم تنظیم می کند.

مثال 3: نمایش با استفاده از `cout.write` و `cout.put`:

```
#include <iostream>
using namespace std;

// Driver Code
int main()
{
    char ch = 'A';

    // Print first 5 characters
    cout.write("Hello Programmer", 5);

    // Print the character ch
    cout.put(ch);
    return 0;
}
```

HelloA

مثال 4: تنظیم نمایش عدد اعشاری تا 5 و 7 رقم با استفاده از `cout.precision`:

```
#include <iostream>
using namespace std;

// Driver Code
int main()
{
    double pi = 3.14159783;

    // Set precision to 5
    cout.precision(5);
```



```
// Print pi
cout << pi << endl;

// Set precision to 7
cout.precision(7);

// Print pi
cout << pi << endl;

return 0;

}
```

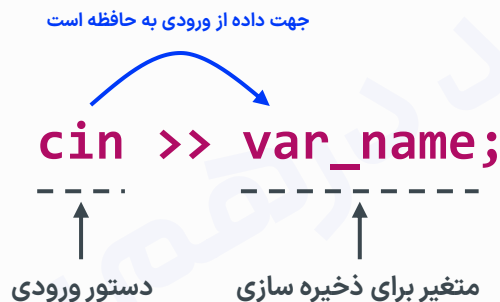
خروجی

3.1416

3.141598

ورودی C++

در زبان C++، شی `cin` برای دریافت ورودی به صورت قالب بندی شده از وسایل ورودی استاندارد مثل کیبورد استفاده می شود. ما برای خواندن ورودی از شی `cin` همراه با عملگر `>>` استفاده می کنیم.



در این دستور جهت داده از ورودی (کیبورد) به حافظه (متغیر است) پس برای انتقال داده از عملگر `>>` استفاده میشود

به مثال های زیر توجه کنید:

مثال 5: ورودی / خروجی عدد صحیح (int)

```
#include <iostream>

using namespace std;

int main() {

    int num;

    cout << "Enter an integer: ";

    cin >> num;    // Taking input

    cout << "The number is: " << num;
    return 0;
}
```

Enter an integer: 70

The number is: 70

در این برنامه ما از کد زیر

```
cin >> num;
```

برای دریافت ورودی از کاربر استفاده کرده ایم. ورودی در متغیر num ذخیره می شود. ما برای دریافت ورودی از cin از عملگر >> استفاده می کنیم.

نکته: اگر از عبارت `using namespace std;` استفاده نکنیم، به جای cin باید از `std::cin` استفاده کنیم.

مثال 6: محاسبه مساحت یک دایره (float)

برای محاسبه مساحت دایره از رابطه $a = 3.14 * r * r$ استفاده میشود که در آن r به معنای شعاع دایره بوده و از کاربر دریافت میشود



```
#include <iostream>

using namespace std;

int main()
{
    float r = 0;

    cout << "Enter r: ";
    cin >> r;

    float area = 3.14 * r * r;

    cout << "Area: " << area;
}
```

خروجی





Enter r: 5

Area: 78.5

مثال 7: محاسبه BMI بزرگسالان (float)

برای محاسبه شاخص توده بدنی از رابطه زیر استفاده میشود

$$BMI = \frac{\text{وزن}}{\text{قد} \times \text{قد}}$$

```
#include <iostream>

using namespace std;

int main()
{
    float weight = 0;
    float height = 0;
```



```
cout << "Enter weight: ";  
cin >> weight;  
cout << "Enter height: ";  
cin >> height;  
  
float bmi = weight / (height * height);  
  
cout << "BMI: " << bmi;  
  
}
```

خروجی

```
Enter weight: 57  
Enter height: 1.69  
BMI: 19.9572844
```



```
#include <iostream>
using namespace std;

int main() {
    char a;
    int num;

    cout << "Enter a character and an integer: ";
    cin >> a >> num;

    cout << "Character: " << a << endl;
    cout << "Number: " << num;

    return 0;
}
```

Enter a character and an integer: F

23

Character: F

Number: 23

تمرین های بخش ورودی / خروجی

1. کد های نوشته شده در جزوه و کلاس را مجددا نوشته و خروجی آن را بررسی کنید

2. برنامه ای بنویسید تا خروجی زیر را چاپ کند:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

3. برنامه ای بنویسید تا دو عدد را از ورودی دریافت نموده و در دو متغیر با نام های num1 و num2 ذخیره نموده و جمع آن ها را در خروجی چاپ کند:

نکته: برای عملیات جمع از عملگر + استفاده می شود

4. برنامه ای بنویسید تا با دریافت طول و عرض یک مستطیل مساحت آن را محاسبه و در خروجی چاپ نماید:

نکته: رابطه مساحت مستطیل به صورت $area = length * width$ است

5. برنامه ای بنویسید تا شاخص توده بدنی (BMI) را محاسبه و تا 3 رقم اعشار آن را نمایش دهد

6. کد های نوشته شده را بدون استفاده از دستور `using namespace std` بازنویسی کنید

تبدیل نوع (Type Conversion) در C++

در این بخش به بررسی تبدیل انواع داده به یکدیگر در زبان C/C++ خواهیم پرداخت



تبدیل نوع (Type Conversion) در C++

در C++ امکان تبدیل داده از یک نوع به نوع دیگر وجود دارد. این فرایند تبدیل نوع داده نامیده می شود.

دو نوع تبدیل نوع داده در C++ وجود دارد:

1. تبدیل نوع ضمنی (Implicit Conversion)

2. تبدیل نوع صریح (Explicit Conversion)

تبدیل نوع ضمنی (Implicit Conversion)

این نوع تبدیل داده، بطور خودکار توسط کامپایلر انجام می شود. همچنین به عنوان تبدیل خودکار نیز شناخته می شود.

بیایید به دو مثال از تبدیل ضمنی داده نگاهی بیندازیم:

مثال 1: تبدیل از int به double

(صفحه بعد)



```
//Working of Implicit type-conversion
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    // assigning an int value to num_int
```

```
    int num_int = 9;
```

```
    // declaring a double type variable
```

```
    double num_double;
```

```
    // implicit conversion
```

```
    // assigning int value to a double variable
```

```
    num_double = num_int;
```

```
    cout << "num_int = " << num_int << endl;
```

```
    cout << "num_double = " << num_double << endl;
```

```
    return 0;
```

```
}
```


خروجی:

```
num_int = 9
num_double = 9
```

در برنامه ای که نوشتیم، یک مقدار از نوع `<int>` را به یک متغیر از نوع `<double>` اختصاص دادیم:

```
num_double = num_int;
```

در اینجا، مقدار `<int>` قبل از انتساب به متغیر `<num_double>` توسط کامپایلر به طور خودکار به `<double>` تبدیل می شود. این یک نمونه از تبدیل ضمنی نوع (implicit type conversion) است.

مثال 2: تبدیل اتوماتیک از `double` به `int`

(صفحه بعد)



```
//Working of Implicit type-conversion
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int num_int;
```

```
    double num_double = 9.99;
```

```
    // implicit conversion
```

```
    // assigning a double value to an int variable
```

```
    num_int = num_double;
```

```
    cout << "num_int = " << num_int << endl;
```

```
    cout << "num_double = " << num_double << endl;
```

```
    return 0;
```

```
}
```

خروجی برنامه:

خروجی:

```
num_int = 9
num_double = 9.99
```

در برنامه، یک مقدار اعشاری (double) را به یک متغیر عددی صحیح (int) اختصاص داده ایم.

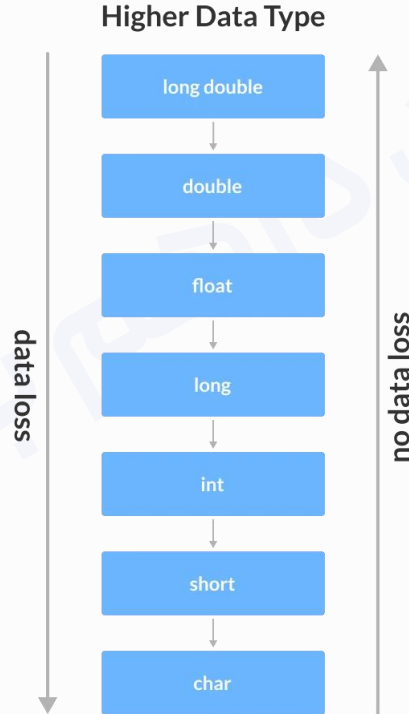
```
num_int = num_double;
```

در اینجا، قبل از اینکه مقدار اعشاری به متغیر num_int اختصاص داده شود، کامپایلر به طور خودکار آن را به نوع int تبدیل می کند. این نیز نمونه ای از تبدیل نوع ضمنی است.

توجه: از آنجایی که int نمی تواند بخش اعشاری داشته باشد، ارقام بعد از نقطه اعشار در مثال بالا حذف می شوند.

از دست رفتن اطلاعات در هنگام تبدیل (تبدیل محدود کننده)

همانطور که در مثال بالا دیدیم، تبدیل از یک نوع داده به نوع دیگر مستعد از دست دادن داده است. این زمانی اتفاق می افتد که داده های نوع بزرگتر به داده های نوع کوچکتر تبدیل می شوند.



Lower Data Type



همانطور که گفته شد در هنگام تبدیل نوع ممکن است داده هایی از دست بروند، اما این داده ها کدام هستند؟

1. در هنگام تبدیل نوع عدد صحیح بیت های اضافی حذف می شوند
2. در هنگام تبدیل اعداد اعشاری به اعداد صحیح مقدار اعشار آن ها حذف میشود

• تبدیل نوع اعداد صحیح سائز بزرگ به اعداد صحیح با سائز کوچک:

در هنگام تبدیل نوع داده ها بیت های داده از سمت راست به چپ در نوع جدید کپی میشوند، در نتیجه بیت های اضافی نوع سائز بزرگ که در نوع سائز کوچک جا نمیشوند حذف میشود.

مثال: فرض کنید می خواهیم عدد 259 را در یک متغیر 8 بیتی ذخیره کنیم

- مرحله اول: عدد 259 را به سیستم دودویی تبدیل می کنیم.

0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

بنابراین، نمایش دودویی عدد 259 برابر با 10000011 خواهد بود.

- مرحله دوم: 8 بیت سمت چپ عدد دودویی را حذف می کنیم.



بنابراین، 8 بیت باقیمانده 00000111 خواهد بود.

- مرحله سوم: عدد دودویی باقیمانده را در متغیر 8 بیتی کپی می کنیم

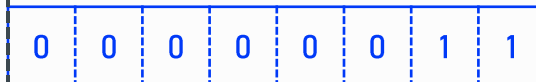
بیت ها از این جایگاه به بعد حذف می شوند

short int



سایز: 16 بیت
مقدار: 259

char



سایز: 8 بیت
مقدار: 3

- تبدیل نوع اعداد اعشاری به اعداد صحیح:

در هنگام تبدیل نوع اعشاری به نوع صحیح به دلیل ایمن بودن نوع داده زبان C/C++ مقدار اعشار آن حذف شده و در نوع صحیح قرار نمی گیرد.



مثال: فرض کنید می خواهیم عدد 14.99 را در یک متغیر عدد صحیح کپی کنیم

14 . 99
-- --
↑ ↑
صحیح اعشار

در این مرحله مقدار اعشار از متغیر حذف میشود (این کار توسط کامپایلر انجام میشود)

14 . ~~99~~
-- --
↑ ↑
صحیح اعشار

خروجی نهایی عدد صحیح به صورت زیر است:

14
--
↑
صحیح

تبدیل نوع صریح (Explicit Conversion)

زمانی که کاربر به صورت دستی داده ها را از یک نوع به نوع دیگر تغییر می دهد، این عمل به عنوان تبدیل صریح (explicit conversion) شناخته می شود. این نوع تبدیل نیز با عنوان قالب بندی نوع (type casting) شناخته می شود.

سه روش اصلی برای استفاده از تبدیل صریح در C++ وجود دارد:

- تبدیل نوع به سبک C (C-style) (همچنین با عنوان قالب بندی نشانه شناخته می شود)
- تبدیل نوع به سبک توابع (Function-style) (همچنین با عنوان تبدیل نوع به سبک C++ قدیمی شناخته می شود)
- عملگر های تبدیل نوع

تبدیل نوع به سبک C (C-style)

همانطور که از نامش پیداست، این نوع تبدیل نوع در زبان برنامه نویسی C رایج است. همچنین با نام تبدیل نوع نشانه گذاری (cast notation) شناخته می شود.

نحو (syntax) این روش به صورت زیر است:

(data_type) expression;

(data_type) expression;

نوع داده خروجی

عبارت برای تبدیل

برای مثال:

```
// initializing int variable
int num_int = 26;

// declaring double variable
double num_double;

// converting from int to double
num_double = (double)num_int;
```

مثال 3:

```
// C++ program to demonstrate
// explicit type casting

#include <iostream>
using namespace std;

int main()
{
    double x = 1.2;

    // Explicit conversion from double to int
    int sum = (int)x + 1;

    cout << "Sum = " << sum;

    return 0;
}
```

خروجی:

Sum = 2

تبدیل نوع به سبک توابع (Function-style)

ما همچنین می توانیم برای تبدیل داده از یک نوع به نوع دیگر از تبدیل نوع شبیه تابع استفاده کنیم.

ساختار نحوی این سبک به صورت زیر است:

```
data_type(expression);
```

data_type(expression);

نوع داده خروجی

عبارت برای تبدیل

برای مثال:

```
// initializing int variable
int num_int = 26;

// declaring double variable
double num_double;

// converting from int to double
num_double = double(num_int);
```

مثال 4: تبدیل نوع

(صفحه بعد)



```
#include <iostream>
using namespace std;

int main() {
    // initializing a double variable
    double num_double = 3.56;
    cout << "num_double = " << num_double << endl;

    // C-style conversion from double to int
    int num_int1 = (int)num_double;
    cout << "num_int1 = " << num_int1 << endl;

    // function-style conversion from double to int
    int num_int2 = int(num_double);
    cout << "num_int2 = " << num_int2 << endl;

    return 0;
}
```

خروجی:

(صفحه بعد)



```
num_double = 3.56  
num_int1   = 3  
num_int2   = 3
```

برای تبدیل نوع داده، از هر دو روش تبدیل به سبک C و تبدیل به سبک تابع استفاده کردیم و نتایج را نمایش دادیم. از آنجایی که هر دو روش کار یکسانی را انجام می دهند، خروجی یکسانی نیز ارائه می دهند.

عملگر های تبدیل نوع (Type Conversion Operators)

علاوه بر این دو تبدیل نوع، C++ همچنین دارای چهار عملگر برای تبدیل نوع است. این عملگرها به عنوان عملگر های تبدیل نوع شناخته می شوند و عبارتند از:

- static_cast
- dynamic_cast
- const_cast
- reinterpret_cast

برای مثال:

```
#include <iostream>
using namespace std;

int main()
{
    float f = 3.5;

    // using cast operator
    int b = static_cast<int>(f);

    cout << b;
}
```

خروجی:

3



در آموزش های بعدی، در مورد این تبدیل ها بیشتر خواهیم آموخت.

مجید درهم بخش



عملگرها (Operators) در C++

در این بخش به بررسی انواع عملگرها و کاربرد آنها در زبان C/C++ خواهیم پرداخت

عملگرها (Operators) در C++

عملگرها نمادهایی هستند که عملیاتی را بر روی متغیرها و مقادیر انجام می دهند. برای مثال، "+" یک عملگر برای جمع است، در حالی که "-" یک عملگر برای تفریق است.

عملگرها در C++ به 6 نوع دسته بندی می شوند:

1. عملگرهای محاسباتی
2. عملگرهای انتساب
3. عملگرهای رابطه ای
4. عملگرهای منطقی
5. عملگرهای بیتی
6. دیگر عملگرها

1. عملگرهای محاسباتی (Arithmetic)

عملگرهای محاسباتی برای انجام عملیات های حسابی روی متغیرها و داده ها استفاده می شوند. برای مثال،

`a + b;`

در اینجا، عملگر "+" برای جمع دو متغیر a و b استفاده می شود. به همین ترتیب، عملگرهای محاسباتی دیگری نیز در C++ وجود دارند. برای مثال،

عملیات	عملگر
جمع	+
تفریق	-
ضرب	*
تقسیم	/
باقیمانده پس از تقسیم	%

```
#include <iostream>

using namespace std;

int main() {
    int a, b;
    a = 7;
    b = 2;

    // printing the sum of a and b
    cout << "a + b = " << (a + b) << endl;

    // printing the difference of a and b
    cout << "a - b = " << (a - b) << endl;

    // printing the product of a and b
    cout << "a * b = " << (a * b) << endl;

    // printing the division of a by b
    cout << "a / b = " << (a / b) << endl;
```



```
// printing the modulo of a by b
cout << "a % b = " << (a % b) << endl;
```

```
return 0;
}
```

خروجی:

```
a + b = 9
a - b = 5
a * b = 14
a / b = 3
a % b = 1
```

اینجا ، عملگر های "+", "-", "*" و "/" به ترتیب جمع ، تفریق و ضرب را محاسبه می کنند، همانطور که می توانستیم انتظار داشته باشیم.

• عملگر تقسیم /

توجه کنید به عملیات (a / b) در برنامه ما. عملگر / عملگر تقسیم است.



همانطور که از مثال بالا می توانیم ببینیم، اگر یک عدد صحیح توسط یک عدد صحیح دیگر تقسیم شود، خارج قسمت را خواهیم گرفت. اما اگر یکی از مقسوم یا مقسوم علیه یک عدد اعشاری باشد، نتیجه را به صورت اعشاری خواهیم گرفت.

In C++,

```
7/2 is 3
```

```
7.0 / 2 is 3.5
```

```
7 / 2.0 is 3.5
```

```
7.0 / 2.0 is 3.5
```

• عملگر باقیمانده %

عملگر باقیمانده "%" باقیمانده تقسیم را محاسبه می کند. وقتی $a = 9$ توسط $b = 4$ تقسیم می شود، باقیمانده برابر با 1 است.

توجه: عملگر "%" فقط می تواند با اعداد صحیح استفاده شود.



عملگر های افزایش و کاهش

C++ همچنین عملگر های افزایش و کاهش را نیز ارائه می دهد: به ترتیب ++ و -- .

- ++ مقدار عملوند را یک واحد افزایش می دهد
- -- مقدار عملوند را یک واحد کاهش می دهد

برای مثال:

```
int num = 5;  
  
// increment operator  
++num; // 6
```

اینجا، کد num++; مقدار num را 1 واحد افزایش می دهد.

مثال 2: عملگر های افزایش و کاهش

(صفحه بعد)



```
// Working of increment and decrement operators

#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 100, result_a, result_b;

    // incrementing a by 1 and storing the result in result_a
    a++;
    result_a = a;
    cout << "result_a = " << result_a << endl;

    // decrementing b by 1 and storing the result in result_b
    b--;
    result_b = b;
    cout << "result_b = " << result_b << endl;

    return 0;
}
```


خروجی:

```
result_a = 11
result_b = 99
```

در برنامه بالا، ما از عملگر های ++ و -- به عنوان **پسوند** استفاده کرده ایم (**a++ و b--**) اما ما همچنین می توانیم از این عملگرها به عنوان **پیشوند** نیز استفاده کنیم (**++a و --b**).

برای یادگیری بیشتر به سرفصل عملگر های افزایش ++ و کاهش -- به صورت پیشوند و پسوند مراجعه کنید.

2. عملگر های انتساب (Assignment)

در C++، عملگر های انتساب برای انتساب مقادیر به متغیرها استفاده می شوند. برای مثال،

```
// assign 5 to a
a = 5;
```

در اینجا، ما مقدار 5 را به متغیر a نسبت داده ایم.

عملگر	مثال	معادل با
=	$a = b;$	$a = b;$
+=	$a += b;$	$a = a + b;$
-=	$a -= b;$	$a = a - b;$
*=	$a *= b;$	$a = a * b;$
/=	$a /= b;$	$a = a / b;$
%=	$a \% = b;$	$a = a \% b;$

مثال 3: عملگرهای انتساب



```
#include <iostream>
using namespace std;

int main() {
    int a, b;

    // 2 is assigned to a
    a = 2;

    // 7 is assigned to b
    b = 7;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
    cout << "\nAfter a += b;" << endl;

    // assigning the sum of a and b to a
    a += b; // a = a + b
    cout << "a = " << a << endl;

    return 0;
}
```

خروجی:

a = 2

b = 7

After a += b;

a = 9

3. عملگر های رابطه ای (Relational)

یک عملگر رابطه ای برای بررسی رابطه بین دو عملوند استفاده می شود. برای مثال،

```
// checks if a is greater than b
```

```
a > b;
```

اینجا، < یک عملگر رابطه ای است. این عملگر بررسی می کند که آیا a از b بزرگتر است یا خیر.

اگر رابطه درست باشد، 1 را بر می گرداند در حالی که اگر رابطه نادرست باشد، 0 را بر می گرداند.

مثال	عملیات	عملگر
$3 == 5$ gives us false	برابر بودن با	<code>==</code>
$3 != 5$ gives us true	برابر نبودن با	<code>!=</code>
$3 > 5$ gives us false	بزرگتر از	<code>></code>
$3 < 5$ gives us true	کوچکتر از	<code><</code>
$3 >= 5$ give us false	بزرگتر یا مساوی با	<code>>=</code>
$3 <= 5$ gives us true	کوچکتر یا مساوی با	<code><=</code>

مثال 4: عملگر های رابطه ای



```
#include <iostream>

using namespace std;

int main() {
    int a, b;
    a = 3;
    b = 5;
    bool result;

    result = (a == b);    // false
    cout << "3 == 5 is " << result << endl;

    result = (a != b);    // true
    cout << "3 != 5 is " << result << endl;

    result = a > b;        // false
    cout << "3 > 5 is " << result << endl;

    result = a < b;        // true
    cout << "3 < 5 is " << result << endl;
```



```
result = a >= b; // false
cout << "3 >= 5 is " << result << endl;
```

```
result = a <= b; // true
cout << "3 <= 5 is " << result << endl;
```

```
return 0;
```

```
}
```

خروجی:

```
3 == 5 is 0
```

```
3 != 5 is 1
```

```
3 > 5 is 0
```

```
3 < 5 is 1
```

```
3 >= 5 is 0
```

```
3 <= 5 is 1
```

توجه: عملگر های رابطه ای برای تصمیم گیری و حلقه ها استفاده می شوند.

4. عملگر های منطقی (Logical)

عملگر های منطقی برای بررسی اینکه یک عبارت درست (true) یا نادرست (false) است استفاده می شوند. اگر عبارت درست باشد، عملگر 1 را بر می گرداند و اگر نادرست باشد، 0 را بر می گرداند.

عملیات	مثال	عملگر
اند (AND) منطقی. زمانی True است که تمام عملوند ها True باشند.	expression1 && expression2	&&
ار (OR) منطقی. زمانی True است که یکی یا تمام عملوند ها True باشند.	expression1 expression2	
نات (NOT) منطقی. زمانی True است که عملوند false باشد، (عملوند را برعکس میکند)	!expression	!



در زبان برنامه نویسی C++، عملگر های منطقی معمولا در تصمیم گیری استفاده می شوند. برای درک بهتر عملگر های منطقی، بیایید به مثال های زیر نگاهی بیندازیم.

Suppose,

`a = 5`

`b = 8`

Then,

`(a > 3) && (b > 5)` evaluates to `true`

`(a > 3) && (b < 5)` evaluates to `false`

`(a > 3) || (b > 5)` evaluates to `true`

`(a > 3) || (b < 5)` evaluates to `true`

`(a < 3) || (b < 5)` evaluates to `false`

`!(a < 3)` evaluates to `true`

`!(a > 3)` evaluates to `false`

```
#include <iostream>
using namespace std;

int main() {
    bool result;

    result = (3 != 5) && (3 < 5);    // true
    cout << "(3 != 5) && (3 < 5) is " << result << endl;

    result = (3 == 5) && (3 < 5);    // false
    cout << "(3 == 5) && (3 < 5) is " << result << endl;

    result = (3 == 5) && (3 > 5);    // false
    cout << "(3 == 5) && (3 > 5) is " << result << endl;

    result = (3 != 5) || (3 < 5);    // true
    cout << "(3 != 5) || (3 < 5) is " << result << endl;

    result = (3 != 5) || (3 > 5);    // true
    cout << "(3 != 5) || (3 > 5) is " << result << endl;
```



```
result = (3 == 5) || (3 > 5);    // false
cout << "(3 == 5) || (3 > 5) is " << result << endl;
```

```
result = !(5 == 2);    // true
cout << "!(5 == 2) is " << result << endl;
```

```
result = !(5 == 5);    // false
cout << "!(5 == 5) is " << result << endl;
```

```
return 0;
```

خروجی:

```
(3 != 5) && (3 < 5) is 1
(3 == 5) && (3 < 5) is 0
(3 == 5) && (3 > 5) is 0
(3 != 5) || (3 < 5) is 1
(3 != 5) || (3 > 5) is 1
(3 == 5) || (3 > 5) is 0
```



`!(5 == 2) is 1`

`!(5 == 5) is 0`

توضیح کد عملگر های منطقی

- `(3 < 5) && (3 != 5)` برابر با 1 است زیرا نتیجه هر دو عبارت `(3 != 5)` و `(3 < 5)` برابر با **true** هستند (1 هستند).
- `(3 < 5) && (3 == 5)` برابر با 0 است زیرا نتیجه عبارت `(3 == 5)` برابر با **false** است (0 است).
- `(3 > 5) && (3 == 5)` برابر با 0 است زیرا نتیجه هر دو عبارت `(3 == 5)` و `(3 > 5)` برابر با **false** هستند (0 هستند).
- `(3 < 5) || (3 != 5)` برابر با 1 است زیرا نتیجه هر دو عبارت `(3 != 5)` و `(3 < 5)` برابر با **true** هستند (1 هستند).
- `(3 > 5) || (3 != 5)` برابر با 1 است زیرا نتیجه عبارت `(3 != 5)` برابر با **true** است (1 است).
- `(3 > 5) || (3 == 5)` برابر با 0 است زیرا نتیجه هر دو عبارت `(3 == 5)` و `(3 > 5)` برابر با **false** هستند (0 هستند).
- `!(5 == 2)` برابر با 1 است زیرا نتیجه عبارت `(5 == 2)` برابر با **false** است (0 است).
- `!(5 == 5)` برابر با 0 است زیرا نتیجه عبارت `(5 == 5)` برابر با **true** است (1 است).

5. عملگر های بیتی (Bitwise)

در زبان C++، عملگر های بیتی برای انجام عملیات روی تک تک بیت های داده استفاده می شوند. این عملگرها فقط با انواع داده char (کاراکتر) و int (عدد صحیح) قابل استفاده هستند.

توضیحات	عملگر
اند (AND) بیتی	&
ار (OR) بیتی	
ار انحصاری (XOR) بیتی	^
مکمل بیتی	~
شیفت به چپ بیتی	<<
شیفت به راست بیتی	>>

برای بررسی بیشتری به سرفصل [عملگر های بیتی \(Bitwise\) در C++](#) مراجعه کنید.

5. دیگر عملگر ها

در اینجا لیستی از برخی عملگر های رایج دیگر موجود در C++ آورده شده است. ما در آموزش های بعدی بیشتر در مورد آنها یاد خواهیم گرفت.

مثال	توضیحات	عملگر
<code>sizeof(int); // 4</code>	سایز نوع داده را برمیگرداند	<code>sizeof</code>
<code>string result = (5 > 0) ? "even" : "odd"; // "even"</code>	مقداری را با توجه به نتیجه شرط بر می گرداند	<code>?:</code>
<code>&num; // address of num</code>	آدرس حافظه عملوند را نشان می دهد	<code>&</code>
<code>s1.marks = 92;</code>	برای دسترسی به عضو های متغیر های struct یا اشیا class ها	<code>.</code>
<code>ptr->marks = 92;</code>	استفاده به همراه اشاره گر ها برای دسترسی به متغیر های struct یا class ها	<code>-></code>



<<	نمایش مقدار در خروجی	cout << 5;
>>	دریافت مقدار ورودی	cin >> num;

عملگر سه گانه (Ternary)

ما از عملگر سه گانه در زبان C/C++ برای اجرای یک کد در صورت درست بودن شرط و اجرای کد دیگری در صورت نادرست بودن شرط استفاده می کنیم. برای مثال:

```
(age >= 18) ? cout << "Can Vote" : cout << "Cannot Vote";
```

در اینجا، اگر سن (age) بزرگ تر یا مساوی 18 باشد، عبارت "Can Vote" چاپ می شود. در غیر این صورت، عبارت "Cannot Vote" چاپ می شود.

ساختار عملگر سه گانه

ساختار عملگر سه گانه به صورت زیر است:

testCondition ? expression1 : expression 2;

testCondition ? expression1 : expression2;

↑
شرط برای بررسی

↑
عبارت اول

↑
عبارت دوم

شرط تست (testCondition) یک عبارت منطقی است که در نهایت نتیجه آن **درست** یا **نادرست** می شود. اگر شرط:

- درست باشد: دستور **expression1** (قبل از دو نقطه) اجرا می شود.
- نادرست باشد: دستور **expression2** (بعد از دو نقطه) اجرا می شود.

عملگر سه گانه یک عملگر است که سه عملوند (condition, expression1 and expression2) را می گیرد. به همین دلیل به آن عملگر سه گانه گفته می شود.

به مثال زیر توجه کنید:


```
#include <stdio.h>

int main() {
    int age;

    // take input from users
    printf("Enter your age: ");
    scanf("%d", &age);

    // ternary operator to find if a person can vote or not
    (age >= 18) ? cout << "You can vote" : cout << "You cannot vote";

    return 0;
}
```



Enter your age: 12

You cannot vote

در مثال بالا، از یک عملگر سه گانه استفاده شده است که بر اساس مقدار ورودی بررسی می کند که آیا یک کاربر می تواند رای بدهد یا خیر. در اینجا:

- (age >= 18) - شرطی است که بررسی می کند آیا مقدار ورودی برابر یا بیشتر از 18 است.

- "You can vote" << cout - عبارتی است که در صورت درست بودن شرط اجرا می شود.

- "You cannot vote" << cout - عبارتی است که در صورت نادرست بودن شرط اجرا می شود.

در این مثال، کاربر مقدار 12 را وارد می کند، بنابراین شرط نادرست می شود. در نتیجه، خروجی "You cannot vote" (شما نمی توانید رای دهید) را در خروجی دریافت می کنیم.

خروجی 2

Enter your age: 24

You can vote



این بار مقدار ورودی 24 است که بیشتر از 18 است. بنابراین، خروجی "شما می توانید رای دهید" را در خروجی دریافت می کنیم.

در آینده درباره این عملگر بیشتر صحبت خواهیم کرد.

مجید درهم بخش



تمرین های بخش عملگرها

1. کد های نوشته شده در جزوه و کلاس را مجددا نوشته و خروجی آن را بررسی کنید
2. برنامه ای بنویسید تا قیمت و تعداد دو محصول را از کاربر دریافت نموده، جمع مبلغ سبد خرید را محاسبه و در خروجی نمایش دهد.

Input-> Price of Product 1: 2300

Input-> Number of Product 1: 3

Input-> Price of Product 2: 8000

Input-> Number of Product 2: 1

Output-> Cart Sum: 14900

3. برنامه ای بنویسید تا زمان را به صورت <ثانیه : دقیقه : ساعت> (HH:MM:SS) از کاربر دریافت نموده و آن را به ثانیه تبدیل و با پیام مناسب در خروجی نمایش دهد.

Input-> Hour: 10

Input-> Minute: 6

Input-> Second: 17

Output-> Converted Time: 39617

4. برنامه ای بنویسید تا تاریخ تولد کاربر را به صورت <روز : ماه : سال> (YYYY:MM:DD) از کاربر دریافت نموده و سن کاربر را به صورت زیر نمایش دهد:

- به صورت YYYY:MM:DD

- تعداد روز های زندگی

- تعداد ساعت زندگی

برای مثال:

Input-> Year: 1386

Input-> Month: 5

Input-> Day: 20

Output-> 16 Year, 6 Month, 3 Day

Output-> 6029 Day

Output-> 144696 Hour

5. برنامه ای بنویسید تا یک مقدار هگزادسیمال یک رقمی را به صورت کاراکتر از کاربر دریافت نموده و آن را به مبنای ده تبدیل و نمایش دهد. برای مثال:

Input-> Hex: C

Output-> Decimal: 12



6. برنامه ای بنویسید تا یک عدد را در دو مرحله به عنوان رمز از کاربر دریافت نماید، در صورتی که رمز وارد شده در مرحله اول و دوم با یکدیگر برابر بود پیغام "Success" و در صورتی که رمز وارد شده در مرحله اول و دوم برابر نبود پیام "Failed" را در خروجی نمایش دهد، برای مثال:

Input-> Enter Password: 2354

Input-> Verify Password: 2354

Output-> Success

نکته 1: نوع متغیر را `unsigned long int` در نظر بگیرید.

نکته 2: رمز با 0 شروع نمی شود.

7. برنامه ای بنویسید تا حجم یک فایل (مگابایت) را از کاربر دریافت نموده و واحد نمایش آن را به Byte, KB, GB داده و در خروجی نمایش دهد، برای مثال:

Input-> Enter File Size: 10.5

Output-> File Size (B): 10500000

Output-> File Size (KB): 10500

Output-> File Size (B): 0.0105

نکته: نوع ورودی اعشاری در نظر گرفته شود.



8. برنامه ای بنویسید تا دو عدد را از کاربر دریافت نماید، سپس مقدار دو عدد ذخیره در متغیر را با یکدیگر جابجا نموده و در خروجی نمایش دهد، برای مثال:

Input-> A: 3

Input-> B: 8

Output-> A: 8

Output-> B: 3

نکته 1: باید مقدار دو متغیر را جابجا و در خروجی نمایش دهید (مقدار نمایش داده شده با نام متغیر باید یکسان باشد)

9. برنامه ای بنویسید تا یک کاراکتر را از کاربر دریافت نماید، نوع کاراکتر را بررسی و با توجه به اینکه کاراکتر وارد شده از حروف الفبا هست یا اعداد پیغام مناسب را نمایش دهید، برای مثال:

مثال 1:

Input-> Enter Value: G

Output-> Value Type: alphabet

مثال 2:

Input-> Enter Value: 3

Output-> Value Type: number



10. برنامه ای بنویسید تا دمای 7 روز هفته را از کاربر دریافت نماید، سپس میانگین دمای هوا در 7 روز گذشته را محاسبه و در خروجی نمایش دهید، برای مثال:

Input-> Saturday: 27.5

Input-> Sunday: 25

Input-> Monday: 25

Input-> Tuesday: 26

Input-> Wednesday: 24

Input-> Thursday: 25

Input-> Friday: 24.5

Output-> Mean of Temperature: 25.28



توضیحات (Comments) در C++

در این بخش به بررسی توضیحات و کاربرد های آن در زبان C/C++ خواهیم پرداخت

توضیحات (Comments) در C++

کامنت ها یا توضیحات در C++ راهنمایی هایی هستند که یک برنامه نویس می تواند برای خواندن و درک آسان تر کد خود اضافه کند. این کامنت کاملاً توسط کامپایلر های C++ نادیده گرفته شده و اجرا نمی شوند.

دو روش برای اضافه کردن کامنت ها به برنامه وجود دارد:

// - کامنت یک خطی

/* */ - کامنت چند خطی

کامنت یک خطی

در C++، هر خطی که به // شروع شده باشد یک کامنت است، برای مثال:

(صفحه بعد)



```
// declaring a variable
int a;
```



```
// initializing the variable 'a' with the value 2
a = 2;
```

در اینجا، ما از دو کامنت یک خطی استفاده کرده ایم:

• `// declaring a variable`

• `// initializing the variable 'a' with the value 2`

همچنین ما میتوانیم کامنت یک خطی را به صورت زیر استفاده نماییم:

```
int a;    // declaring a variable
```

کامنت چند خطی

در C++، هر خطی که بین `/*` و `*/` قرار بگیرد یک کامنت است، برای مثال:

(صفحه بعد)



```
/* declaring a variable  
to store salary to employees  
*/  
int salary = 2000;
```

این نحو می تواند برای نوشتن کامنت های تک خطی و چند خطی استفاده شود.

استفاده از کامنت برای خطایابی (Debugging)

کامنت ها همچنین می توانند برای غیرفعال کردن کد و جلوگیری از اجرای آن استفاده شوند. به عنوان مثال:

```
#include <iostream>  
using namespace std;  
  
int main() {  
    cout << "some code";  
    cout << "'error code";
```



```
cout << "some other code";
```

```
return 0;
```

```
}
```

اگر در هنگام اجرای برنامه با خطایی مواجه شویم، به جای حذف کد مشکل ساز، می توانیم با استفاده از کامنت آن را غیرفعال کنیم تا اجرا نشود. این روش می تواند به عنوان یک ابزار اشکال زدایی ارزشمند عمل کند.

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    cout << "some code";
```

```
    // cout << "error code";
```

```
    cout << "some other code";
```

```
    return 0;
```

```
}
```



نکته حرفه ای: میانبر برای کامنت گذاری را به خاطر بسپارید، این کار بسیار مفید است. در اکثر ویرایشگر های کد، میانبر برای ویندوز / + Ctrl و برای مک / + Cmd است.

چرا از کامنت ها استفاده می کنیم؟

اگر در کد خود کامنت بگذاریم، در آینده درک آن برای خودمان آسان تر می شود. همچنین، برای توسعه دهندگان دیگر نیز فهم کد راحت تر خواهد بود.

نکته: کامنت ها نباید جایگزینی برای توضیح کد ضعیف نوشته شده به زبان انگلیسی باشند. همیشه باید کدهایی با ساختار مناسب و خود توضیح بنویسیم و سپس از کامنت ها استفاده کنیم.

به عنوان یک قاعده کلی، از کامنت ها برای توضیح چرا کاری را انجام داده اید، به جای چگونه آن را انجام داده اید استفاده کنید، تا نشان دهید چرا کارتان خوب است.

روش های مقداردهی متغیر ها در C++

در این بخش به بررسی روش های مقداردهی به انواع متغیر ها در زبان C/C++ خواهیم پرداخت

روش های مقداردهی متغیر هادر C++

در C++ 17، روش های جدیدی برای اعلام متغیر معرفی شده است. قبلا اختصاص و اعلام با استفاده از علامت "=" انجام می شد. اما در C++ 17، دو روش دیگر معرفی شدند، به طور کلی سه روش مقدار دهی متغیر ها عبارت اند از:

مقدار دهی با استفاده از کپی کردن (Copy Initializer)

در این روش همانند قبل عمل مقدار دهی توسط عملگر انتصاب (=) انجام می شود، این روش در بیشتر زبان های برنامه نویسی وجود داشته و به عنوان روش پایه در زبان C استفاده می شود. ساختار این روش به صورت زیر است:

مقدار دهی توسط عملگر انتصاب

```
data_type var_name = value;
```

نوع داده نام متغیر مقدار

برای مثال:

```
int my_var = 5;
```

در این مثال مقدار 5 در متغیر my_var کپی می شود

مقدار دهی با استفاده از سازنده (Constructor Initializer)

در این روش، مقدار متغیر در داخل پرانتز قرار می گیرد. این روش به دو صورت زیر انجام می شود



در این حالت همانطور که مشخص است بعد از نام متغیر مقدار متغیر درون پرانتز قرار میگیرد

حالت دوم

data_type var_name = (value);

نوع داده نام متغیر مقدار

مقدار دهی توسط پرانتز

در این حالت همانطور که مشخص است بعد از نام متغیر مقدار متغیر درون پرانتز قرار گرفته و با استفاده از عملگر انتصاب درون متغیر قرار داده می شود. لازم به ذکر است در این حالت، مقدار متغیر همیشه آخرین مقدار در پرانتز را بر می گرداند.

برای مثال:

```
int a(5);  
int b = (10);
```

مثال 1:

```
#include <iostream>

using namespace std;

// Driver Code
int main()
{
    // Declaring the variable in parentheses
    // Method 1
    int a(5);
    cout << "a = " << a;

    // Declaring the variable in parentheses
    // Method 2
    int b = (10);
    cout << endl << "b = " << b;

    return 0;
}
```

خروجی:

```
a = 5  
b = 10
```

توضیح:

در C++ 17، می توانید متغیرها را با قرار دادن مقادیر درون پرانتز نیز تعریف کنید. تفاوت بین مقداردهی سازنده (constructor initialization) و روش قدیمی و معمولی این است که مقداردهی سازنده همیشه آخرین مقدار داخل پرانتز را بر می گرداند، صرف نظر از بزرگی یا علامت آن. برای مثال:

```
a = (2, 21, -50, 1)
```

مقدار ذخیره شده در a برابر با 1 خواهد بود.

مقدار دهی یکنواخت (Uniform_INITIALIZER)

در این روش، مقدار متغیر داخل آکولاد به جای پرانتز قرار می گیرد. با این کار، مقدار را می توان به دو صورت زیر پاس داد:



حالت اول

مقدار دهی توسط آکولاد

`data_type`

نوع داده

`var_name`

نام متغیر

`{ value }`

مقدار

در این حالت همانطور که مشخص است بعد از نام متغیر مقدار متغیر درون آکولاد قرار میگیرد

حالت دوم

The diagram illustrates the syntax for declaring a variable in C++: `data_type var_name = { value };`. It uses dashed lines and arrows to identify the components:

- `data_type` is labeled "نوع داده" (Data Type).
- `var_name` is labeled "نام متغیر" (Variable Name).
- `{ value }` is labeled "مقدار" (Value/Quantity).

 A bracket above the curly braces is labeled "مقدار دهی توسط آکولاد" (Assignment by Brackets).

در این حالت همانطور که مشخص است بعد از نام متغیر مقدار متغیر درون آکولاد قرار گرفته و با استفاده از عملگر انتصاب درون متغیر قرار داده می شود.

مثال 2:

```
#include <iostream>

using namespace std;

int main()
{

    // Declaring the variable in curly braces
    // Method 1
    int a{ 3 };
    cout << "a = " << a;

    // Declaring the variable in curly braces
    // Method 2
    int b = { 7 };
    cout << endl << "b = " << b;

    return 0;
}
```

خروجی:

```
a = 3  
b = 7
```

توضیح:

بر خلاف مقداردهی سازنده (Constructor initialization)، این روش اختصاص مقدار، تنها می تواند یک مقدار را در آکولاد ها بپذیرد. قرار دادن چندین مقدار در آکولاد ها منجر به یک خطای کامپایل خواهد شد. برای مثال:

```
int b = {7, 12};
```

خروجی:

```
prog.cpp: In function 'int main()':  
prog.cpp:9:12: error: scalar object 'b' requires one element in initializer  
    int b={7, 12};
```



05

کنترل روند برنامه در ++C

در این بخش به دستورات کنترلی در زبان ++C و نحوه کار با آنها خواهیم پرداخت

ساختار if...else و if...else...های تو در تو در C++

در این بخش به بررسی انواع ساختار if در زبان C/C++ خواهیم پرداخت

ساختار if...else و if...else...if های تو در تو در C++

در برنامه نویسی کامپیوتری، ما از دستور شرطی اگر ... در غیر این صورت (if...else) برای اجرای یک بلوک کد تحت شرایط خاص و یک بلوک کد دیگر تحت شرایط متفاوت استفاده می کنیم.

به عنوان مثال، می توانیم بر اساس نمرات به دست آمده توسط یک دانش آموز، به او نمره (A, B, C) اختصاص دهیم.

- اگر درصد نمره بیش از 90 باشد، نمره A را اختصاص دهید
- اگر درصد نمره بیش از 75 باشد، نمره B را اختصاص دهید
- اگر درصد نمره بیش از 65 باشد، نمره C را اختصاص دهید

در زبان برنامه نویسی C++، سه نوع ساختار if...else وجود دارد:

1. ساختار if

2. ساختار if...else

3. ساختار if...else...else if

ساختار if در C++

نحو ساختار if به صورت زیر است:

```
if (condition)
{
    // body of if statement
}
```

دستور if شرط داخل پرانتز () را ارزیابی می کند.

- اگر شرط به صورت "درست" (true) ارزیابی شود، کد داخل بدنه if اجرا می شود.
- اگر شرط به صورت "غلط" (false) ارزیابی شود، کد داخل بدنه if رد می شود.

نکته: کد داخل { } بدنه دستور if است.



Condition is true

```
int number = 5;
```

```
if (number > 0) {  
    // code  
}
```

```
// code after if
```

Condition is false

```
int number = 5;
```

```
if (number < 0) {  
    // code  
}
```

```
// code after if
```

مثال 1: ساختار if در C++

```
// Program to print positive number entered by the user
// If the user enters a negative number, it is skipped

#include <iostream>
using namespace std;

int main() {
    int number;

    cout << "Enter an integer: ";
    cin >> number;

    // checks if the number is positive
    if (number > 0) {
        cout << "You entered a positive integer: " << number << endl;
    }

    cout << "This statement is always executed.";

    return 0;
}
```

خروجی 1

Enter an integer: 5
You entered a positive number: 5
This statement is always executed.

وقتی کاربر عدد 5 را وارد می کند، شرط "`number > 0`" به عنوان "درست" ارزیابی می شود و دستور داخل بدنه `if` اجرا می شود.

خروجی 2

Enter a number: -5
This statement is always executed.

هنگامی که کاربر عدد -5 را وارد می کند، شرط "`number > 0`" به عنوان "نادرست" ارزیابی می شود و دستورات داخل بدنه `if` اجرا نمی شوند.

مثال 2: بررسی زوج یا فرد بودن عدد در C++

```
#include <iostream>
using namespace std;

int main() {
    int number;

    cout << "Enter an number: ";
    cin >> number;

    // checks if the number is positive
    if ((number % 2) == 0) {
        cout << "You entered a even number: " << number << endl;
    }

    // checks if the number is positive
    if ((number % 2) == 1) {
        cout << "You entered a odd number: " << number << endl;
    }

    return 0;
}
```

خروجی 1

Enter an integer: 4

You entered a even number: 4

وقتی کاربر عدد 4 را وارد می کند، شرط " $(number \% 2) == 0$ " به عنوان "درست" ارزیابی می شود و دستور داخل بدنه **if** اول اجرا می شود.

خروجی 2

Enter an integer: 7

You entered a odd number: 7

وقتی کاربر عدد 7 را وارد می کند، شرط " $(number \% 2) == 1$ " به عنوان "درست" ارزیابی می شود و دستور داخل بدنه **if** دوم اجرا می شود.

ساختار if..else در C++

عبارت **if** می تواند یک بخش **else** اختیاری داشته باشد. نحو آن به صورت زیر است:

```
if (condition)
{
    // block of code if condition is true
}
else
{
    // block of code if condition is false
}
```

عبارت شرطی **if..else** شرط داخل پرانتز را ارزیابی می کند.

به شکل زیر دقت کنید:

Condition is true

```
int number = 5;  
  
if (number > 0) {  
    // code  
}  
else {  
    // code  
}  
// code after if...else
```

Condition is false

```
int number = 5;  
  
if (number < 0) {  
    // code  
}  
else {  
    // code  
}  
// code after if...else
```



اگر نتیجه شرط درست (true) باشد،

- کد داخل بدنه if اجرا می شود
- کد داخل بدنه else اجرا نمی شود

اگر نتیجه شرط غلط (false) باشد،

- کد داخل بدنه else اجرا می شود
- کد داخل بدنه if اجرا نمی شود

مثال 3: ساختار if...else در C++

```
// Program to check whether an integer is positive or negative
// This program considers 0 as a positive number

#include <iostream>
using namespace std;

int main() {
```



```
int number;

cout << "Enter an integer: ";
cin >> number;

if (number >= 0) {
    cout << "You entered a positive integer: " << number << endl;
}
else {
    cout << "You entered a negative integer: " << number << endl;
}

cout << "This line is always printed.";

return 0;
}
```

خروجی 1

(صفحه بعد)



```
Enter an integer: 4
```

```
You entered a positive integer: 4.
```

```
This line is always printed.
```

در برنامه بالا، شرط ما این است که "عدد بزرگتر یا مساوی صفر" باشد. اگر عددی بزرگتر یا مساوی صفر وارد کنیم، شرط "درست" ارزیابی می شود.

در اینجا، ما عدد "4" را وارد می کنیم. بنابراین، شرط "درست" است. در نتیجه، دستور داخل بدنه "if" اجرا می شود.

خروجی 2

```
Enter an integer: -4
```

```
You entered a negative integer: -4.
```

```
This line is always printed.
```

در اینجا، ما "-4" را وارد می کنیم. بنابراین، شرط "غلط" است. از این رو، عبارت داخل بدنه **else** اجرا می شود.

ساختار if...else...else if در C++

عبارت شرطی if...else برای اجرا کردن یک بلوک کد از میان دو گزینه استفاده می شود. با این حال، اگر نیاز به انتخاب بین بیش از دو گزینه باشد، از عبارت شرطی if...else...else if استفاده می کنیم. به عبارت دیگر، به ما اجازه می دهد از بین بیشتر از دو انتخاب، یک انتخاب را بر اساس شرایط مختلف انجام دهیم.

ساختار if...else...else if به صورت زیر است:

```
if (condition1)
{
    // code block 1
}
else if (condition2)
{
    // code block 2
}
else
{
    // code block 3
}
```

در اینجا،

- اگر شرط 1 درست باشد، بلوک کد 1 اجرا می شود.
- اگر شرط 1 نادرست باشد، آنگاه شرط 2 ارزیابی می شود.
- اگر شرط 2 درست باشد، بلوک کد 2 اجرا می شود.
- اگر شرط 2 نادرست باشد، بلوک کد 3 اجرا می شود.

1st Condition is true

```
int number = 2;
if (number > 0) {
    // code
}
else if (number == 0){
    // code
}
else {
    //code
}
//code after if
```

2nd Condition is true

```
int number = 0;
if (number > 0) {
    // code
}
else if (number == 0){
    // code
}
else {
    //code
}
//code after if
```

All Conditions are false

```
int number = -2;
if (number > 0) {
    // code
}
else if (number == 0){
    // code
}
else {
    //code
}
//code after if
```

نکته: می توان از چند عبارت else if استفاده نمود اما فقط یک عبارت if و else مجاز است.

مثال 4: ساختار if...else...else if در C++

```
// Program to check whether an integer is positive, negative or zero

#include <iostream>
using namespace std;

int main() {

    int number;

    cout << "Enter an integer: ";
    cin >> number;

    if (number > 0) {
        cout << "You entered a positive integer: " << number << endl;
    }
}
```




```
else if (number < 0) {  
    cout << "You entered a negative integer: " << number << endl;  
}  
else {  
    cout << "You entered 0." << endl;  
}  
  
cout << "This line is always printed."  
  
return 0;  
}
```

خروجی 1

```
Enter an integer: 1  
You entered a positive integer: 1.  
This line is always printed.
```

خروجی 2



```
Enter an integer: -2
```

```
You entered a negative integer: -2.
```

```
This line is always printed.
```

خروجی 3

```
Enter an integer: 0
```

```
You entered 0.
```

```
This line is always printed.
```

در این برنامه، یک عدد از کاربر دریافت می کنیم. سپس با استفاده از ساختار `if...else...else if` بررسی می کنیم که عدد ورودی مثبت، منفی یا صفر است.

اگر عدد بزرگتر از 0 باشد، کد داخل بلوک `if` اجرا می شود. اگر عدد کمتر از 0 باشد، کد داخل بلوک `else if` اجرا می شود. در غیر این صورت، کد داخل بلوک `else` اجرا می شود.

ساختار if..else تو در تو (Nested)

گاهی اوقات، نیاز داریم یک دستور if را داخل دستور if دیگری قرار دهیم. این ساختار، یک دستور if تودرتو نامیده می شود.

تصور کنید که این کار مانند ایجاد لایه های مختلف از دستورات if است. یک دستور if بیرونی اولیه وجود دارد و در داخل آن، یک دستور if درونی دیگر قرار دارد. نحو (ساختار) آن به این صورت است:

```
// outer if statement
if (condition1)
{
    // statements

    // inner if statement
    if (condition2)
    {
        // statements
    }
}
```

نکات:

- می توانید با توجه به نیاز، دستورات `else if` و `else` را به دستور `if` داخلی اضافه کنید.
- دستور `if` داخلی را همچنین می توان در صورت وجود، درون دستورات `else if` یا `else if` بیرونی قرار داد.
- می توانید از چندین لایه از دستورات `if` به صورت تو در تو استفاده کنید.

مثال 5: ساختار `if` تو در تو در `C++`

```
// C++ program to find if an integer is positive, negative or zero
// using nested if statements
```

```
#include <iostream>
using namespace std;
```

```
int main() {
    int num;
```

```
    cout << "Enter an integer: ";
    cin >> num;
```



```
// outer if condition
if (num != 0) {

    // inner if condition
    if (num > 0) {
        cout << "The number is positive." << endl;
    }
    // inner else condition
    else {
        cout << "The number is negative." << endl;
    }
}
// outer else condition
else {
    cout << "The number is 0 and it is neither positive nor negative." << endl;
}

cout << "This line is always printed." << endl;

return 0;
}
```

خروجی 1

Enter an integer: 35
The number is positive.
This line is always printed.

خروجی 2

Enter an integer: -35
The number is negative.
This line is always printed.

خروجی 3

Enter an integer: 0
The number is 0 and it is neither positive nor negative.
This line is always printed.

در مثال بالا:

- یک عدد صحیح را به عنوان ورودی از کاربر دریافت می کنیم و آن را در متغیر num ذخیره می کنیم.
- سپس از یک عبارت if...else برای بررسی اینکه آیا num برابر با 0 نیست استفاده می کنیم.
 - اگر شرط درست باشد، عبارت if...else **داخلی** اجرا می شود.
 - اگر شرط غلط باشد، کد موجود در قسمت else **خارجی** اجرا می شود که عبارت "The number is 0 and it is neither positive nor negative." را چاپ می کند.
- عبارت if...else **داخلی** بررسی می کند که آیا عدد ورودی مثبت است یا خیر، یعنی آیا num بزرگتر از 0 است.
 - اگر شرط درست باشد، یک عبارت چاپ می کنیم که بگوییم عدد مثبت است.
 - اگر شرط غلط باشد، چاپ می کنیم که عدد منفی است.

نکته: همانطور که مشاهده می کنید، استفاده از چندین if...else تو در تو، منطق برنامه را پیچیده می کند. در صورت امکان، همیشه سعی کنید تا از تو در تو کردن (nesting) کردن بیش از حد if...else اجتناب کنید.

ساختار بدنه if..else با تنها یک عبارت اجرایی (Statement)

اگر بدنه if...else فقط یک عبارت داشته باشد، می توانید { } را در برنامه حذف کنید. به عنوان مثال، می توانید کد زیر را

```
int number = 5;

if (number > 0)
{
    cout << "The number is positive." << endl;
}
else
{
    cout << "The number is negative." << endl;
}
```

با کد



```
int number = 5;
```

```
if (number > 0)
```

```
    cout << "The number is positive." << endl;
```

```
else
```

```
    cout << "The number is negative." << endl;
```

خروجی هر دو برنامه یکسان خواهد بود.

نکته: اگرچه استفاده از { } زمانی که بدنه if...else فقط یک دستور دارد اجباری نیست، اما استفاده از { } به خوانایی کد شما کمک می کند.

موارد بیشتر درباره تصمیم گیری:

در شرایط خاص، یک عملگر سه گانه می تواند جایگزین یک عبارت if...else شود. برای بررسی بیشتر می توانید به سرفصل عملگر سه گانه (Ternary) در

C++ مراجعه کنید.



اگر نیاز به انتخاب بین بیش از یک گزینه بر اساس یک شرط آزمایشی داریم، می توان از عبارت switch استفاده کرد. برای بررسی بیشتر می توانید به سرفصل [ساختار switch در C++](#) مراجعه کنید.

برای یادگیری بیشتر به مثال های زیر مراجعه کنید:

[برنامه C++ برای بررسی زوج یا فرد بودن یک عدد](#)

[برنامه C++ برای بررسی اینکه آیا یک کاراکتر صدادار یا صامت است](#)

[برنامه C++ برای پیدا کردن بزرگترین عدد از بین سه عدد](#)

[سوال مسابقه کدکاپ 8 - نقطه ی گم شده](#)

تمرین های بخش ساختار شرطی if

1. مثال های حل شده در کلاس را مجدداً نوشته و نتایج را بررسی نمایید

2. برنامه ای بنویسید تا ریشه های معادله درجه دوم $ax^2 + bx + c = 0$ یافته و در خروجی با فرمت مناسب نمایش دهد

نکته: همانطور که در سرفصل مروری بر الگوریتم ها توضیح داده شد در ریاضیات برای محاسبه این معادله از رابطه زیر استفاده میشود:

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

3. برنامه ای بنویسید تا تاریخ تولد کاربر را به صورت **<روز : ماه : سال>** (YYYY:MM:DD) از کاربر دریافت نموده و سن کاربر را به صورت زیر نمایش دهد:

- به صورت YYYY:MM:DD

- تعداد روز های زندگی

- تعداد ساعت زندگی

برای مثال:

Input-> Year: 1386

Input-> Month: 5

Input-> Day: 20



Output-> 16 Year, 6 Month, 3 Day

Output-> 6029 Day

Output-> 144696 Hour

8. برنامه ای بنویسید تا یک کاراکتر را از کاربر دریافت نماید، نوع کاراکتر را بررسی و با توجه به اینکه کاراکتر وارد شده از حروف الفبا هست یا اعداد پیغام مناسب را نمایش دهید، برای مثال:

مثال 1:

Input-> Enter Value: G

Output-> Value Type: alphabet

مثال 2:

Input-> Enter Value: 3

Output-> Value Type: number

حلقه for در C++

در این بخش به بررسی حلقه for در زبان C/C++ خواهیم پرداخت

حلقه for در C++

در برنامه نویسی کامپیوتر، از حلقه ها برای تکرار یک بلوک کد استفاده می شود.

به عنوان مثال، فرض کنید می خواهیم یک پیام را 100 بار نمایش دهیم. به جای اینکه دستور چاپ را 100 بار بنویسیم، می توانیم از یک حلقه استفاده کنیم.

این فقط یک مثال ساده بود؛ با استفاده مؤثر از حلقه ها، می توانیم به کارایی و پیچیدگی بیشتری در برنامه های خود دست یابیم.

در C++ سه نوع حلقه وجود دارد:

- حلقه for
- حلقه while
- حلقه do...while

این سرفصل بر روی حلقه for در C++ تمرکز دارد. در سرفصل های بعدی با انواع دیگر حلقه ها آشنا خواهیم شد.

حلقه for در C++

ساختار حلقه for به صورت زیر است:

```
for (initialization; condition; update)
{
    // body of-loop
}
```

for (int i = 5; i <= 10; i++)

↑ ↑ ↑

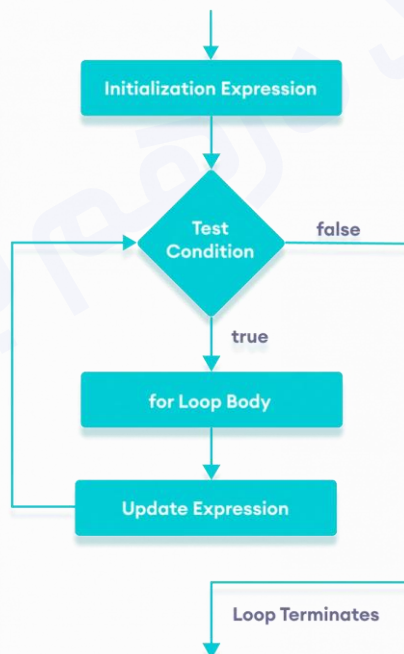
مقدار اولیه شرط برای بررسی بروزرسانی

در این عبارت:

- مقدار اولیه (initialization): مقداری که متغیرها را فقط یک بار اجرا می‌شود.
- شرط (condition): اگر درست باشد، بدنه حلقه اجرا می‌شود. اگر نادرست باشد، حلقه for پایان می‌یابد.

- بروزرسانی (update): مقدار متغیرهای مقداردهی شده را بروزرسانی می کند و سپس شرط را دوباره بررسی می کند.
برای یادگیری بیشتر در مورد شرط ها، سرفصل عملگرهای رابطه ای و منطقی در C++ را بررسی کنید.

فلوچارت حلقه for در C++





مثال 1: چاپ اعداد 1 تا 5

```
#include <iostream>

using namespace std;

int main() {
    for (int i = 1; i <= 5; ++i) {
        cout << i << " ";
    }
    return 0;
}
```

خروجی

1 2 3 4 5

اینجا نحوه عملکرد این برنامه نشان داده شده است

تکرار	متغیر	$i \leq 5$	عملیات
1st	$i = 1$	true	1 چاپ شده است و i به 2 افزایش پیدا کرده است
2nd	$i = 2$	true	2 چاپ شده است و i به 3 افزایش پیدا کرده است
3rd	$i = 3$	true	3 چاپ شده است و i به 4 افزایش پیدا کرده است
4th	$i = 4$	true	4 چاپ شده است و i به 5 افزایش پیدا کرده است
5th	$i = 5$	true	5 چاپ شده است و i به 6 افزایش پیدا کرده است
6th	$i = 6$	false	حلقه تمام میشود

مثال 2: نمایش یک متن به تعداد 5 بار

```
// C++ Program to display a text 5 times

#include <iostream>

using namespace std;

int main() {
    for (int i = 1; i <= 5; ++i) {
        cout << "Hello World! " << endl;
    }
    return 0;
}
```

خروجی

(صفحه بعد)

```
Hello World!  
Hello World!  
Hello World!  
Hello World!  
Hello World!
```

اینجا نحوه عملکرد این برنامه نشان داده شده است

تکرار	متغیر	$i \leq 5$	عملیات
1st	$i = 1$	true	"Hello World!" چاپ شده است و i به 2 افزایش پیدا کرده است
2nd	$i = 2$	true	"Hello World!" چاپ شده است و i به 3 افزایش پیدا کرده است
3rd	$i = 3$	true	"Hello World!" چاپ شده است و i به 4 افزایش پیدا کرده است



4th	$i = 4$	true	"Hello World!" چاپ شده است و i به 5 افزایش پیدا کرده است
5th	$i = 5$	true	"Hello World!" چاپ شده است و i به 6 افزایش پیدا کرده است
6th	$i = 6$	false	حلقه تمام میشود

مثال 3: یافتن مجموع n عدد طبیعی اول

```
// C++ program to find the sum of first n natural numbers
// positive integers such as 1,2,3,...n are known as natural numbers
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```



```
int num, sum;
sum = 0;

cout << "Enter a positive integer: ";
cin >> num;

for (int i = 1; i <= num; ++i) {
    sum += i;
}

cout << "Sum = " << sum << endl;

return 0;
}
```

خروجی

(صفحه بعد)

Enter a positive integer: 10

Sum = 55

در مثال بالا، دو متغیر num و sum داریم. متغیر sum با مقدار 0 و متغیر num با مقداری که کاربر وارد می کند، مقداردهی می شوند.

توجه: در این مثال از یک حلقه for استفاده شده است.

```
for(int i = 1; i <= num; ++i)
```

در این کد:

- `int i = 1`: متغیر i با مقدار اولیه 1 مقداردهی می شود.
- `i <= num`: این شرط تا زمانی که i کوچکتر یا مساوی num باشد حلقه را اجرا می کند.
- `++i`: با هر بار تکرار، مقدار i به اندازه 1 افزایش می یابد.

هنگامی که i به 11 می رسد، شرط نادرست می شود و sum برابر با مجموع $0 + 1 + 2 + \dots + 10$ خواهد بود.

حلقه for تو در تو (Nested for Loop)

حلقه for تو در تو به معنای قرار دادن یک حلقه داخل حلقه دیگر است. هر بار که حلقه بیرونی اجرا می شود، حلقه داخلی به طور کامل اجرا می شود. این راهی است برای تکرار وظایف درون وظایف در برنامه شما. نحو آن به صورت زیر است:

```
// outer for loop
for (initialization1; condition1; update1)
{
    // body of-loop

    // inner for loop
    for (initialization2; condition2; update2)
    {
        // body of-loop
    }
}
```

به مثال زیر توجه کنید:


```
// C++ program to demonstrate the use of Nested for loop.
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // Outer loop
```

```
    for (int i = 0; i < 4; i++) {
```

```
        // Inner loop
```

```
        for (int j = 0; j < 4; j++) {
```

```
            // Printing the value of i and j
```

```
            cout << "*" << " ";
```

```
        }
```

```
        cout << endl;
```

```
    }
```

```
}
```

```
* * * *  
* * * *  
* * * *  
* * * *
```

برنامه فوق از حلقه های تو در تو برای چاپ یک ماتریس 4×4 از علامت های ستاره (*) استفاده می کند. در اینجا، حلقه بیرونی (i) روی سطرها تکرار می شود و حلقه داخلی (j) روی ستون ها تکرار می شود. در هر تکرار، حلقه داخلی یک ستاره و یک فاصله چاپ می کند و همچنین بعد از چاپ هر سطر، یک خط جدید اضافه می شود.

چندین متغیر در حلقه for (Multiple Variable in for Loop)

در حلقه for می توانیم چند متغیر را به طور همزمان مقداردهی اولیه، بررسی، و بروزرسانی کنیم.

مثال 5: چندین متغیر در حلقه for



```
// C++ program to demonstrate the use of multiple variables in for loops
```

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    // initializing loop variable
```

```
    int m, n;
```

```
    // loop having multiple variable and updations
```

```
    for (m = 1, n = 1; m <= 5; m += 1, n += 2) {
```

```
        cout << "iteration " << m << endl;
```

```
        cout << "m is: " << m << endl;
```

```
        cout << "j is: " << n << endl;
```

```
    }
```

```
    return 0;
```

```
}
```

```
iteration 1
m is: 1
j is: 1
iteration 2
m is: 2
j is: 3
iteration 3
m is: 3
j is: 5
iteration 4
m is: 4
j is: 7
iteration 5
m is: 5
j is: 9
```

برنامه بالا از حلقه for با چندین متغیر (در اینجا m و n) استفاده می کند. این حلقه در هر تکرار هر دو متغیر را افزایش داده و بروزرسانی می کند و مقادیر مربوطه آن ها را چاپ می کند.

حلقه for مبتنی بر بازه (Ranged Based for Loop)

در C++11 یک حلقه for مبتنی بر بازه جدید معرفی شد تا با مجموعه هایی مانند آرایه ها و بردار ها کار کند. نحو آن به صورت زیر است:

```
for (variable : collection)
{
    // body of loop
}
```

for (**int n** : **int_array**)

متغیر عضو

مجموعه داده

در این عبارت:

- متغیر عضو (variable): برای تعریف یک متغیر که مقدار هر عنصر در بازه ارائه شده را می گیرد.
- مجموعه داده (collection): بازه ای را نشان می دهد که حلقه روی آن تکرار می شود.

در اینجا، حلقه for برای هر مقدار در مجموعه اجرا می شود و مقدار به متغیر اختصاص داده می شود.

مثال 6: حلقه for مبتنی بر بازه

```
#include <iostream>

using namespace std;

int main() {

    int num_array[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    for (int n : num_array) {
        cout << n << " ";
    }

    return 0;
}
```



```
1 2 3 4 5 6 7 8 9 10
```

در برنامه بالا، یک آرایه از نوع int به نام num_array تعریف و مقداردهی اولیه شده است. این آرایه 10 عنصر دارد. در اینجا، برای دسترسی به همه عناصر آرایه، از یک حلقه for مبتنی بر محدوده استفاده کرده ایم.

مثال 7: حلقه for مبتنی بر بازه

```
// C++ Program to demonstrate the use of range based for loop in C++
#include <iostream>
#include <vector>
using namespace std;

int main()
{
    int arr[] = { 10, 20, 30, 40, 50 };
```



```
cout << "Printing array elements: " << endl;
```

```
// range based for loop
```

```
for (int& val : arr) {
```

```
    cout << val << " ";
```

```
}
```

```
cout << endl;
```

```
cout << "Printing vector elements: " << endl;
```

```
vector<int> v = { 10, 20, 30, 40, 50 };
```

```
// range-based for loop for vector
```

```
for (auto& it : v) {
```

```
    cout << it << " ";
```

```
}
```

```
cout << endl;
```

```
return 0;
```

```
}
```



```
Printing array elements:
```

```
10 20 30 40 50
```

```
Printing vector elements:
```

```
10 20 30 40 50
```

برنامه فوق نحوه استفاده از یک حلقه for مبتنی بر محدوده را نشان می دهد. این حلقه از طریق هر عنصر در یک آرایه (arr در اینجا) و یک بردار (v در اینجا) عبور می کند و عناصر هر دو را چاپ می کند.

حلقه for_each (for_each Loop)

حلقه for_each در C++ یک تابع را به عنوان ورودی می گیرد که روی هر عنصر از یک مجموعه اجرا می شود. این حلقه در فایل هدر <algorithm> تعریف شده است، بنابراین برای عملکرد صحیح این حلقه، باید این فایل هدر را در برنامه خود قرار دهید. این حلقه به صورت زیر است:

for_each (InputIterator start_iter, InputIterator last_iter, Function fnc);

for_each (start_iter : last_iter : function)

عضو شروع

عضو پایان

تابع دستورات

در این عبارت:

- عضو شروع (start_iter): اشاره گر به موقعیت ابتدایی که از آنجا باید عملیات تابع اجرا شود.
- عضو پایان (last_iter): اشاره گر به موقعیت انتهایی که تا آن تابع باید اجرا شود.
- تابع دستورات (function): آرگومان سوم یک تابع یا شی تابع است که برای هر عنصر اجرا می شود.

مثال 8: حلقه for_each

(صفحه بعد)



```
// C++ Program to show use of for_each loop
```

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// function to print values passed as parameter in loop
```

```
void printValues(int i) { cout << i << " " << endl; }
```

```
int main()
```

```
{
```

```
    // initializing vector
```

```
    vector<int> v = { 1, 2, 3, 4, 5 };
```

```
    // iterating using for_each loop
```

```
    for_each(v.begin(), v.end(), printValues);
```

```
    return 0;
```

```
}
```



```
1
2
3
4
5
```

برنامه فوق از حلقه `for_each` برای تکرار هر عنصر در بردار (v) استفاده می کند، سپس تابع `printValues` را برای چاپ هر عنصر فراخوانی می کند. خروجی برای هر تکرار در خط بعدی نمایش داده می شود.

مثال 9: حلقه `for_each`

```
// C++ code to demonstrate the
// working of for_each loop
#include<iostream>
#include<vector>
```



```
#include<algorithm>
using namespace std;

// helper function 1
void printx2(int a)
{
    cout << a * 2 << " ";
}

int main()
{
    // initializing array
    int arr[5] = { 1, 5, 2, 4, 3 };

    cout << "Using Arrays:" << endl;

    // printing array using for_each
    // using function
```



```
cout << "Multiple of 2 of elements are : ";  
for_each(arr, arr + 5, printx2);  
  
cout << endl;  
  
// initializing vector  
vector<int> arr1 = { 4, 5, 8, 3, 1 };  
  
cout << "Using Vectors:" << endl;  
  
// printing array using for_each  
// using function  
cout << "Multiple of 2 of elements are : ";  
for_each(arr1.begin(), arr1.end(), printx2);  
  
cout << endl;  
  
}
```

Using Arrays:

Multiple of 2 of elements are : 2 10 4 8 6

Using Vectors:

Multiple of 2 of elements are : 8 10 16 6 2

در موارد استثناء، اگر تابع خطایی را اعلام کند یا هر یک از عملیات روی تکرارگر ها خطایی ایجاد کند، حلقه ی `for_each` نیز خطایی را اعلام کرده و حلقه را قطع خواهد کرد.

نکات:

- آرگومان های نا معتبر می تواند به رفتارهای غیرقابل تعریف منجر شود.
- `for_each` نمی تواند با اشاره گرهای آرایه کار کند (اشاره گر آرایه اندازه خود را نمی داند، حلقه `for_each` بدون دانستن اندازه آرایه با آرایه ها کار نمی کند).

حلقه for بینهایت (Infinite for Loop)

اگر شرط داخل یک حلقه for همیشه درست باشد، آن حلقه تا زمانی که حافظه پر شود به طور مداوم اجرا می شود. به عنوان مثال:

```
// infinite for loop
for (int i = 1; i > 0; i++)
{
    // block of code
}
```

در برنامه بالا، شرط همیشه درست است، بنابراین کد داخل حلقه به طور نامحدود اجرا می شود.

همچنین زمانی که هیچ پارامتری برای حلقه for داده نشود، به دلیل عدم وجود ورودی، به طور بی پایان تکرار می شود و به یک حلقه بی نهایت تبدیل می شود.

مثال 10: حلقه for بی نهایت

(صفحه بعد)



```
#include <iostream>
using namespace std;

int main()
{
    // skip Initialization, test and update conditions
    for (;;) {
        cout << "InfLoop" << endl;
    }
    // Return statement to show program compiles successfully safely
    return 0;
}
```

خروجی

(صفحه بعد)



InfLoop

InfLoop

•
•
•

infinite times

برای آشنایی بیشتر، نگاهی به این مثال ها بیندازید:

- برنامه C++ برای محاسبه مجموع اعداد طبیعی
- برنامه C++ برای محاسبه توان
- برنامه C++ برای پیدا کردن فاکتوریل
- برنامه C++ برای تولید جدول ضرب

در آموزش بعدی، حلقه های "while" و "do...while" را بررسی خواهیم کرد.

سوالات چالشی بخش حلقه for

1. مسابقه مسیرهای Δ AI و Δ Django – کافی نت رفقا

حلقه `do...while` در `C++`

در این بخش به بررسی حلقه `while` و `do...while` در زبان `C/C++` خواهیم پرداخت

حلقه while و do...while در C++

در برنامه نویسی کامپیوتر، از حلقه ها برای تکرار یک بلوک کد استفاده می شود.

به عنوان مثال، فرض کنید می خواهیم یک پیام را 100 بار نمایش دهیم. به جای اینکه دستور چاپ را 100 بار بنویسیم، می توانیم از یک حلقه استفاده کنیم.

این فقط یک مثال ساده بود؛ با استفاده مؤثر از حلقه ها، می توانیم به کارایی و پیچیدگی بیشتری در برنامه های خود دست یابیم.

در C++ سه نوع حلقه وجود دارد:

- حلقه for
- حلقه while
- حلقه do...while

در آموزش قبلی در مورد حلقه for در C++ یاد گرفتیم. در اینجا قصد داریم در مورد حلقه های while و do...while بیاموزیم.

حلقه while در C++

ساختار حلقه while به صورت زیر است:

```
while (condition)
{
    // body of the loop
}
```

while (i <= 10)



شرط برای بررسی

در اینجا،

- حلقه ی "while" یک شرط را بررسی می کند.
- اگر شرط درست باشد، کد داخل حلقه ی "while" اجرا می شود.



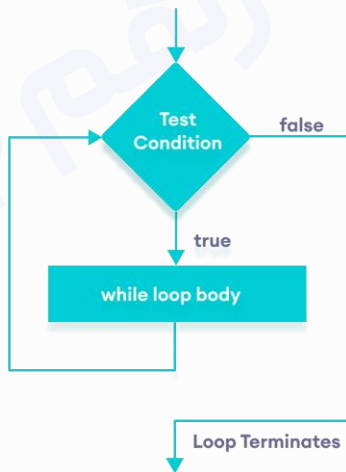
- شرط دوباره بررسی می شود.

- این فرایند تا زمانی که شرط نادرست شود، ادامه می یابد.

- زمانی که شرط نادرست باشد، حلقه پایان می یابد.

برای یادگیری بیشتر در مورد شرط ها، سرفصل عملگرهای رابطه ای و منطقی در C++ را بررسی کنید.

فلوچارت حلقه while در C++





مثال 1: چاپ اعداد 1 تا 5

```
// C++ Program to print numbers from 1 to 5
#include <iostream>

using namespace std;

int main() {
    int i = 1;

    // while loop from 1 to 5
    while (i <= 5) {
        cout << i << " ";
        ++i;
    }

    return 0;
}
```


1 2 3 4 5

اینجا نحوه عملکرد این برنامه نشان داده شده است

تکرار	متغیر	$i \leq 5$	عملیات
1st	$i = 1$	true	1 چاپ شده است و i به 2 افزایش پیدا کرده است
2nd	$i = 2$	true	2 چاپ شده است و i به 3 افزایش پیدا کرده است
3rd	$i = 3$	true	3 چاپ شده است و i به 4 افزایش پیدا کرده است
4th	$i = 4$	true	4 چاپ شده است و i به 5 افزایش پیدا کرده است
5th	$i = 5$	true	5 چاپ شده است و i به 6 افزایش پیدا کرده است



6th	i = 6	false	حلقه تمام میشود
-----	-------	-------	-----------------

مثال 2: جمع اعداد مثبت

```
// program to find the sum of positive numbers
// if the user enters a negative number, the loop ends
// the negative number entered is not added to the sum
```

```
#include <iostream>
using namespace std;
```

```
int main() {
    int number;
    int sum = 0;

    // take input from the user
```



```
cout << "Enter a number: ";
cin >> number;

while (number >= 0) {
    // add all positive numbers
    sum += number;

    // take input again if the number is positive
    cout << "Enter a number: ";
    cin >> number;
}

// display the sum
cout << "\nThe sum is " << sum << endl;

return 0;
}
```

```
Enter a number: 6
Enter a number: 12
Enter a number: 7
Enter a number: 0
Enter a number: -2

The sum is 25
```

در این برنامه، از کاربر خواسته می شود عددی را وارد کند، که در متغیر `number` ذخیره می شود. برای ذخیره مجموع اعداد، یک متغیر به نام `sum` تعریف کرده و آن را با مقدار ۰ مقداردهی اولیه می کنیم. حلقه `while` تا زمانی که کاربر عدد منفی وارد کند ادامه می یابد. در هر تکرار، عددی که کاربر وارد می کند به متغیر مجموع اضافه می شود. هنگامی که کاربر عدد منفی وارد می کند، حلقه پایان می یابد. در نهایت، مجموع کل نمایش داده می شود.

حلقه do...while در C++

حلقه do...while نوعی از حلقه while است که یک تفاوت مهم دارد: بدنه حلقه do...while یکبار قبل از بررسی شرط اجرا می شود.

ساختار حلقه do...while به صورت زیر است:

```
do
{
    // body of loop;
}
while (condition);
```

do {} while (**i <= 10**)

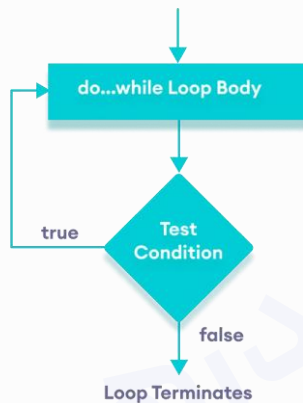
↑
شرط برای بررسی

در اینجا،

1. ابتدا بدنه حلقه اجرا می شود.
2. سپس شرط ارزیابی می شود.
3. اگر شرط درست باشد، بدنه حلقه داخل دستور do دوباره اجرا می شود.
4. شرط دوباره ارزیابی می شود.
5. اگر شرط درست باشد، بدنه حلقه داخل دستور do دوباره اجرا می شود.
6. این فرایند تا زمانی که شرط نادرست شود ادامه می یابد. سپس حلقه متوقف می شود.

فلوچارت حلقه while...do در C++

(صفحه بعد)



مثال 3: چاپ اعداد 1 تا 5

```
// C++ Program to print numbers from 1 to 5
#include <iostream>

using namespace std;

int main() {
    int i = 1;
```



```
// do...while loop from 1 to 5
```

```
do {
```

```
    cout << i << " ";
```

```
    ++i;
```

```
}
```

```
while (i <= 5);
```

```
return 0;
```

```
}
```

خروجی

1 2 3 4 5

اینجا نحوه عملکرد این برنامه نشان داده شده است

تکرار	متغیر	$i \leq 5$	عملیات
1st	$i = 1$	true	1 چاپ شده است و i به 2 افزایش پیدا کرده است
2nd	$i = 2$	true	2 چاپ شده است و i به 3 افزایش پیدا کرده است
3rd	$i = 3$	true	3 چاپ شده است و i به 4 افزایش پیدا کرده است
4th	$i = 4$	true	4 چاپ شده است و i به 5 افزایش پیدا کرده است
5th	$i = 5$	true	5 چاپ شده است و i به 6 افزایش پیدا کرده است
6th	$i = 6$	false	حلقه تمام میشود

```
// program to find the sum of positive numbers
// If the user enters a negative number, the loop ends
// the negative number entered is not added to the sum
#include <iostream>
using namespace std;

int main() {
    int number = 0;
    int sum = 0;

    do {
        sum += number;

        // take input from the user
        cout << "Enter a number: ";
        cin >> number;
```



```
}  
while (number >= 0);  
  
// display the sum  
cout << "\nThe sum is " << sum << endl;  
  
return 0;  
}
```

خروجی 1

```
Enter a number: 6  
Enter a number: 12  
Enter a number: 7  
Enter a number: 0  
Enter a number: -2
```

```
The sum is 25
```



در اینجا، حلقه do...while تا زمانی که کاربر یک عدد منفی وارد کند ادامه می یابد. هنگامی که عدد منفی شود، حلقه خاتمه می یابد و عدد منفی به متغیر sum اضافه نمی شود.

خروجی 2

```
Enter a number: -6
```

```
The sum is 0.
```

اگر کاربر عدد منفی وارد کند، بدنه حلقه do...while فقط یک بار اجرا می شود.

حلقه while بینهایت (Infinite while Loop) در C++

اگر شرط یک حلقه همیشه درست باشد، حلقه تا بینهایت اجرا می شود (تا زمانی که حافظه پر شود). برای مثال:

```
// infinite while loop
while(true) {
    // body of the loop
}
```

اینجا یک مثال از یک حلقه do...while بی انتها است:

```
// infinite do...while loop

int count = 1;

do {
    // body of loop
}
while(count == 1);
```

در برنامه بالا، شرط همیشه درست است. بنابراین، بدنه حلقه تا بی نهایت اجرا خواهد شد.

حلقه while در مقابل حلقه for

یک حلقه for معمولا زمانی استفاده می شود که تعداد تکرارها مشخص باشد. به عنوان مثال،



```
// This loop is iterated 5 times
for (int i = 1; i <=5; ++i)
{
    // body of the loop
}
```

در اینجا، ما می دانیم که حلقه for فقط 5 بار اجرا خواهد شد.

با این حال، حلقه های while و do...while معمولا زمانی استفاده می شوند که تعداد تکرارها نامشخص باشد. به عنوان مثال،

```
while (condition)
{
    // body of the loop
}
```

برای آشنایی بیشتر، نگاهی به این مثال ها بیندازید:

- [برنامه C++ برای چاپ سری فیبوناچی](#)



- برنامه C++ برای پیدا کردن GCD

- برنامه C++ برای پیدا کردن LCM

مجید درهم بخش



عبارت break در C++

در این بخش به بررسی عبارت break در زبان C/C++ خواهیم پرداخت

عبارت break در C++

در C++، هنگام مواجه شدن با عبارت break حلقه خاتمه می یابد. نحو عبارت break به صورت زیر است:

```
break;
```

break;



خروج از حلقه

قبل از اینکه در مورد دستور break یاد بگیرید، مطمئن شوید که در مورد موارد زیر اطلاعات دارید:

- [ساختار if...else، if...else و if...else های تو در تو در C++](#)
- [حلقه for در C++](#)
- [حلقه do...while در C++](#)

```
for (init; condition; update) {  
    // code  
    if (condition to break) {  
        break;  
    }  
    // code  
}
```

```
while (condition) {  
    // code  
    if (condition to break) {  
        break;  
    }  
    // code  
}
```

مثال 1: عبارت break با حلقه for

```
// program to print the value of i

#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        // break condition
        if (i == 3) {
            break;
        }
        cout << i << endl;
    }

    return 0;
}
```



```
1  
2
```

در کد بالا، از حلقه for برای چاپ مقدار i در هر تکرار استفاده شده است. در اینجا، به کد زیر توجه کنید:

```
if (i == 3) {  
    break;  
}
```

این یعنی زمانی که برابر با 3 شود، دستور break حلقه را متوقف می کند. بنابراین، خروجی شامل مقادیر بزرگتر یا مساوی 3 نیست.

نکته: دستور break معمولاً با دستورات تصمیم گیری استفاده می شود.

مثال 2: عبارت break با حلقه while

```
// program to find the sum of positive numbers
// if the user enters a negative numbers, break ends the loop
// the negative number entered is not added to sum

#include <iostream>
using namespace std;

int main() {
    int number;
    int sum = 0;

    while (true) {
        // take input from the user
        cout << "Enter a number: ";
        cin >> number;
```



```
// break condition
if (number < 0) {
    break;
}

// add all positive numbers
sum += number;
}

// display the sum
cout << "The sum is " << sum << endl;

return 0;
}
```

خروجی

(صفحه بعد)



```
Enter a number: 1
Enter a number: 2
Enter a number: 3
Enter a number: -5
The sum is 6.
```

در برنامه بالا، کاربر عددی را وارد می کند. حلقه while برای چاپ مجموع کل اعداد وارد شده توسط کاربر استفاده می شود. در اینجا، به کد زیر توجه کنید:

```
if(number < 0) {
    break;
}
```

این یعنی زمانی که کاربر عدد منفی وارد کند، دستور break حلقه را متوقف می کند و کدهای خارج از حلقه اجرا می شوند. حلقه while تا زمانی که کاربر عدد منفی وارد نکند، ادامه پیدا می کند.

break با حلقه تو در تو

هنگامی که از break در حلقه های تو در تو استفاده می شود، break فقط حلقه داخلی را خاتمه می دهد.

```
// using break statement inside
// nested for loop

#include <iostream>
using namespace std;

int main() {
    int number;
    int sum = 0;

    // nested for loops

    // first loop
    for (int i = 1; i <= 3; i++) {
```




```
// second loop
for (int j = 1; j <= 3; j++) {
    if (i == 2) {
        break;
    }
    cout << "i = " << i << ", j = " << j << endl;
}

return 0;
}
```

خروجی

(صفحه بعد)



```
i = 1, j = 1
```

```
i = 1, j = 2
```

```
i = 1, j = 3
```

```
i = 3, j = 1
```

```
i = 3, j = 2
```

```
i = 3, j = 3
```

در برنامه بالا، دستور break زمانی اجرا می شود که مقدار i برابر با ۲ شود. این دستور باعث خروج از حلقه داخلی شده و جریان برنامه به حلقه خارجی منتقل می شود.

بنابراین، مقدار $i = 2$ هرگز در خروجی نمایش داده نخواهد شد.

دستور break همچنین با دستور switch استفاده می شود. برای بررسی جزئیات به سرفصل [ساختار switch در C++](#) مراجعه کنید.

ادامه مطلب:

• [عبارت continue در C++](#)

عبارت continue در C++

در این بخش به بررسی عبارت continue در زبان C/C++ خواهیم پرداخت

عبارت continue در C++

در برنامه نویسی، از دستور continue برای رد شدن از تکرار جاری حلقه و رفتن کنترل برنامه به تکرار بعدی استفاده می شود. نحو دستور continue به صورت زیر است:

```
continue;
```

continue;

رد شدن از اجرای حلقه

قبل از اینکه در مورد دستور continue یاد بگیرید، مطمئن شوید که در مورد موارد زیر اطلاعات دارید:

- [ساختار if...else و if...else, if...else, if...else در C++](#)
- [حلقه for در C++](#)
- [حلقه do...while در C++](#)

کارکرد عبارت continue در C++

```
for (init; condition; update) {  
    // code  
    if (condition to continue) {  
        continue;  
    }  
    // code  
}
```

```
while (condition) {  
    // code  
    if (condition to continue) {  
        continue;  
    }  
    // code  
}
```

مثال 1: عبارت continue با حلقه for

در یک حلقه for، دستور continue از تکرار جاری رد شده و جریان کنترل به عبارت بروزرسانی می پرد.

```
// program to print the value of i

#include <iostream>
using namespace std;

int main() {
    for (int i = 1; i <= 5; i++) {
        // condition to continue
        if (i == 3) {
            continue;
        }

        cout << i << endl;
    }
}
```



```
return 0;
```

```
}
```

خروجی

```
1
```

```
2
```

```
4
```

```
5
```

در کد بالا، از حلقه for برای چاپ مقدار i در هر تکرار استفاده شده است. در اینجا، به کد زیر توجه کنید:

```
if (i == 3) {  
    continue;  
}
```



این به این معنی است که

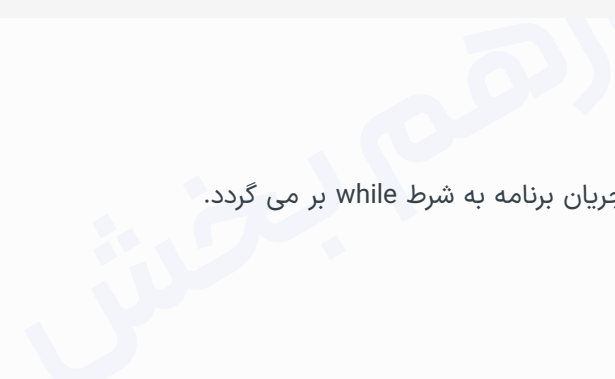
- وقتی i برابر با 3 است، عبارت continue تکرار فعلی را رد می کند و تکرار بعدی را شروع می کند.
- سپس، i به 4 تغییر می کند و شرط دوباره ارزیابی می شود.
- از این رو، 4 و 5 در دو تکرار بعدی چاپ می شوند.

نکته: دستور continue معمولا با دستورات تصمیم گیری استفاده می شود.

مثال 2: عبارت break با حلقه while

در یک حلقه while، دستور continue تکرار جاری را رد می کند و جریان برنامه به شرط while بر می گردد.

(صفحه بعد)





```
// program to calculate positive numbers till 50 only
// if the user enters a negative number,
// that number is skipped from the calculation

// negative number -> loop terminate
// numbers above 50 -> skip iteration

#include <iostream>
using namespace std;

int main() {
    int sum = 0;
    int number = 0;

    while (number >= 0) {
        // add all positive numbers
        sum += number;
```



```
// take input from the user
cout << "Enter a number: ";
cin >> number;

// continue condition
if (number > 50) {
    cout << "The number is greater than 50 and won't be calculated." << endl;
    number = 0; // the value of number is made 0 again
    continue;
}

// display the sum
cout << "The sum is " << sum << endl;

return 0;
}
```

```
Enter a number: 12
Enter a number: 0
Enter a number: 2
Enter a number: 30
Enter a number: 50
Enter a number: 56
The number is greater than 50 and won't be calculated.
Enter a number: 5
Enter a number: -3
The sum is 99
```

در برنامه بالا، کاربر عددی را وارد می کند. از حلقه while برای چاپ مجموع کل اعداد مثبت وارد شده توسط کاربر استفاده می شود، تا زمانی که اعداد وارد شده بزرگتر از 50 نباشند.

به استفاده از دستور continue توجه کنید.



```
if (number > 50){  
    continue;  
}
```

زمانی که کاربر عددی بزرگتر از 50 وارد کند، دستور continue از تکرار جاری رد می شود. سپس جریان برنامه به شرط حلقه while می رود. هنگامی که کاربر عددی کمتر از 0 وارد کند، حلقه خاتمه می یابد.

توجه: دستور continue برای حلقه های do...while نیز به همین صورت عمل می کند.

continue با حلقه تو در تو

هنگامی که از continue در حلقه های تو در تو استفاده می شود، از تکرار فعلی حلقه داخلی عبور می کند. به عنوان مثال،

(صفحه بعد)



```
// using continue statement inside
// nested for loop

#include <iostream>
using namespace std;

int main() {
    int number;
    int sum = 0;

    // nested for loops

    // first loop
    for (int i = 1; i <= 3; i++) {
        // second loop
        for (int j = 1; j <= 3; j++) {
            if (j == 2) {
```



```
        continue;
    }
    cout << "i = " << i << ", j = " << j << endl;
}

return 0;
}
```

خروجی

```
i = 1, j = 1
i = 1, j = 3
i = 2, j = 1
i = 2, j = 3
i = 3, j = 1
i = 3, j = 3
```



در برنامه بالا، زمانی که دستور `continue` اجرا می شود، تکرار جاری حلقه داخلی را رد می کند. و کنترل برنامه به عبارت بروزرسانی حلقه داخلی منتقل می شود.

بنابراین، مقدار $z = 2$ هرگز در خروجی نمایش داده نمی شود.

توجه: دستور `break` کل حلقه را خاتمه می دهد. با این حال، دستور `continue` فقط از تکرار فعلی صرف نظر می کند.

رایگان برای آموزش شخصی - کپی و بازنشر بدون مجوز ممنوع



ساختار switch در C++

در این بخش به بررسی ساختار switch..case در زبان C/C++ خواهیم پرداخت

ساختار switch در C++

عبارت سوئیچ به ما امکان می دهد تا یک بلوک کد را از بین چندین گزینه دیگر اجرا کنیم.

شما می توانید با دستور if...else نیز به نتیجه مشابهی برسید. با این حال، سینتکس عبارت سوئیچ برای خواندن و نوشتن بسیار راحت تر است.

ساختار دستور switch به صورت زیر است:

```
switch (expression)
{
    case constant1:
        // code to be executed if
        // expression is equal to constant1;
        break;

    case constant2:
        // code to be executed if
        // expression is equal to constant2;
```



```
break;  
.  
.  
.  
default:  
    // code to be executed if  
    // expression doesn't match any constant  
}
```

عبارت سوئیچ چگونه کار می کند؟

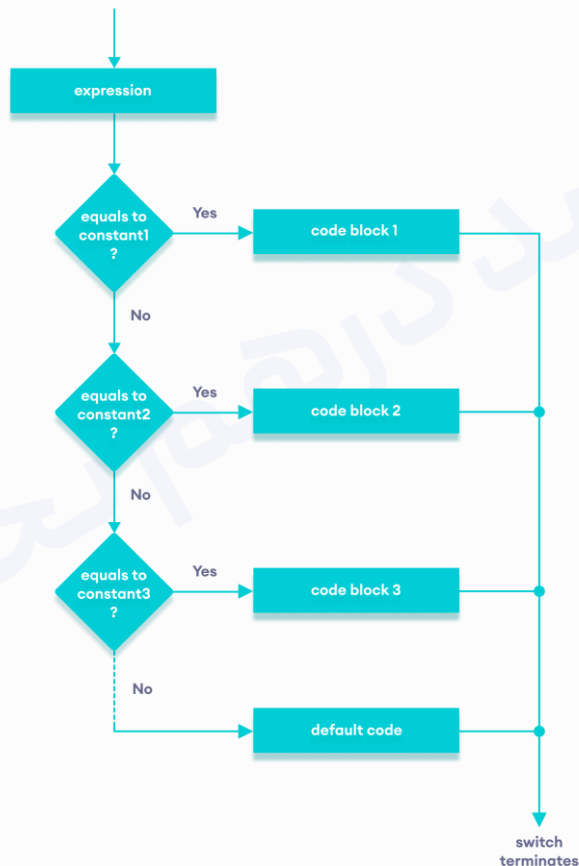
یکبار عبارت ارزیابی شده و با مقادیر هر برجسب مورد (case) مقایسه می شود.

- اگر مطابقت وجود داشته باشد، کد مربوطه بعد از برجسب مطابق اجرا می شود. برای مثال، اگر مقدار متغیر برابر با constant2 باشد، کد بعد از case: constant2 تا زمانی که به عبارت break برخورد کند اجرا می شود.
- اگر هیچ مطابقت وجود نداشته باشد، کد بعد از default: اجرا می شود.

توجه: ما می توانیم همین کار را با if...else..if نردبانی انجام دهیم. با این حال، نحو دستور سوئیچ تمیزتر و خواندن و نوشتن آن بسیار آسانتر

است.

فلوچارت دستور switch



مثال 1: ساخت ماشین حساب با دستور switch

```
// Program to build a simple calculator using switch Statement
#include <iostream>
using namespace std;

int main() {
    char oper;
    float num1, num2;
    cout << "Enter an operator (+, -, *, /): ";
    cin >> oper;
    cout << "Enter two numbers: " << endl;
    cin >> num1 >> num2;

    switch (oper) {
        case '+':
            cout << num1 << " + " << num2 << " = " << num1 + num2;
            break;
```



```
case '-':  
    cout << num1 << " - " << num2 << " = " << num1 - num2;  
    break;  
case '*':  
    cout << num1 << " * " << num2 << " = " << num1 * num2;  
    break;  
case '/':  
    cout << num1 << " / " << num2 << " = " << num1 / num2;  
    break;  
default:  
    // operator is doesn't match any case constant (+, -, *, /)  
    cout << "Error! The operator is not correct";  
    break;  
}  
  
return 0;  
}
```

خروجی 1

Enter an operator (+, -, *, /): +

Enter two numbers:

2.3

4.5

$2.3 + 4.5 = 6.8$

خروجی 2

Enter an operator (+, -, *, /): -

Enter two numbers:

2.3

4.5

$2.3 - 4.5 = -2.2$

خروجی 3

Enter an operator (+, -, *, /): *

Enter two numbers:

2.3

4.5

$2.3 * 4.5 = 10.35$

خروجی 4

Enter an operator (+, -, *, /): /

Enter two numbers:

2.3

4.5

$2.3 / 4.5 = 0.511111$



```
Enter an operator (+, -, *, /): ?  
Enter two numbers:  
2.3  
4.5  
Error! The operator is not correct.
```

در برنامه بالا، ما از دستور switch...case برای انجام محاسبات جمع، تفریق، ضرب و تقسیم استفاده می کنیم.

این برنامه چگونه کار میکند

1. ابتدا از کاربر می خواهیم که عملگر مورد نظر را وارد کند. این ورودی سپس در متغیر char به نام oper ذخیره می شود.
2. سپس از کاربر می خواهیم که دو عدد را وارد کند، که در متغیرهای float به نام های num1 و num2 ذخیره می شوند.
3. دستور switch سپس برای بررسی عملگر وارد شده توسط کاربر استفاده می شود:
a. اگر کاربر + را وارد کند، عمل جمع روی اعداد انجام می شود.
b. اگر کاربر - را وارد کند، عمل تفریق روی اعداد انجام می شود.



a. اگر کاربر * را وارد کند، عمل ضرب روی اعداد انجام می شود.

b. اگر کاربر / را وارد کند، عمل تقسیم روی اعداد انجام می شود.

c. اگر کاربر هر کاراکتر دیگری را وارد کند، کد پیش فرض (default) چاپ می شود.

توجه کنید که دستور break داخل هر بلوک case استفاده می شود. این کار باعث خاتمه یافتن دستور switch می شود.

اگر از دستور break استفاده نشود، تمام موارد بعد از case صحیح اجرا خواهند شد.

می توانید برای مطالعه بیشتر به بخش [برنامه ی C++ برای ساختن یک ماشین حساب ساده](#) مراجعه کنید.

رایگان برای آموزش شخصی - کپی و بازنشر بدون مجوز ممنوع



دستور goto در C++

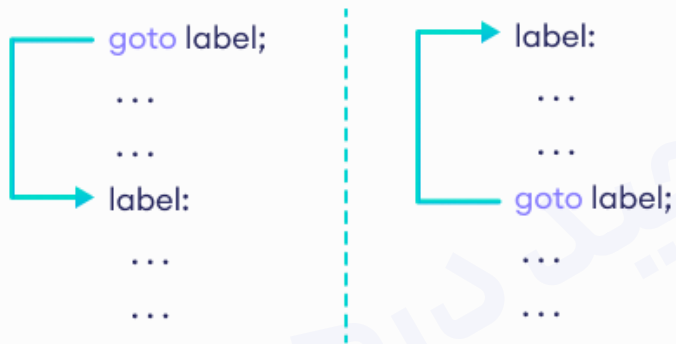
در این بخش به بررسی دستور goto در زبان C/C++ خواهیم پرداخت

دستور goto در C++

در برنامه نویسی C++ از دستور goto برای تغییر دادن روند اجرای معمولی برنامه با انتقال کنترل به بخش دیگری از برنامه استفاده می شود.
نحو دستور goto به صورت زیر است:

```
goto label;  
... ..  
... ..  
... ..  
label:  
statement;  
... ..
```

در نحو بالا، label یک شناسه (identifier) است. هنگامی که با `goto label;` برخورد می کنیم، کنترل برنامه به `label:` پرش پیدا کرده و کدهای پایین آن را اجرا می کند.



مثال: دستور goto

```
// This program calculates the average of numbers entered by the user.  
// If the user enters a negative number, it ignores the number and  
// calculates the average number entered before it.
```

```
# include <iostream>  
using namespace std;
```



```
int main()
{
    float num, average, sum = 0.0;
    int i, n;

    cout << "Maximum number of inputs: ";
    cin >> n;

    for(i = 1; i <= n; ++i)
    {
        cout << "Enter n" << i << ": ";
        cin >> num;

        if(num < 0.0)
        {
            // Control of the program move to jump:
            goto jump;
        }
    }
```



```
        sum += num;
    }

    jump:
    average = sum / i;
    cout << "\nAverage = " << average;
    return 0;
}
```

خروجی

Maximum number of inputs: 10

Enter n1: 2.3

Enter n2: 5.6

Enter n3: -5.6

Average = 3.95



شما می توانید هر برنامه C++ را بدون استفاده از دستور goto بنویسید و به طور کلی توصیه می شود از آنها استفاده نکنید.

دلایل اجتناب از دستور goto

دستور goto به شما امکان می دهد به هر قسمت از برنامه پرش کنید، اما منطق برنامه را پیچیده و درهم می کند.
در برنامه نویسی مدرن، دستور goto یک ساختار مضر و یک عمل برنامه نویسی بد در نظر گرفته می شود.
دستور goto را می توان در بیشتر برنامه های C++ با استفاده از دستورات break و continue جایگزین کرد.

06

توابع در C++

در این بخش به بررسی انواع توابع از پیش تعریف شده و نحوه ایجاد و توسعه توابع ساختار یافته در زبان C++ خواهیم پرداخت

توابع در C++

در این بخش به بررسی توابع در زبان C/C++ خواهیم پرداخت

توابع در C++

یک تابع، بلوکی از کد است که یک کار خاص را انجام می دهد.

فرض کنید ما نیاز به ایجاد یک برنامه برای رسم یک دایره و رنگ آمیزی آن داریم. برای حل این مشکل می توانیم دو تابع ایجاد کنیم:

- تابعی برای کشیدن دایره

- تابعی برای رنگ آمیزی دایره

تقسیم کردن یک مشکل پیچیده به بخش های کوچکتر باعث می شود برنامه ما به راحتی قابل درک و استفاده مجدد باشد.

دو نوع تابع وجود دارد:

1. **توابع کتابخانه استاندارد (Standard Library Functions):** از پیش در C++ تعریف شده اند

2. **تابع تعریف شده توسط کاربر (User-defined Function):** توسط کاربران ایجاد می شود

در این آموزش، ما بیشتر روی توابع تعریف شده توسط کاربر تمرکز خواهیم کرد. با این حال در سرفصلی جداگانه توابع کتابخانه استاندارد را نیز بررسی خواهیم کرد.

تابع تعریف شده توسط کاربر (User-defined Function)

C++ به برنامه نویس اجازه می دهد تا توابع دلخواه خود را تعریف کند.

یک تابع تعریف شده توسط کاربر، گروهی از کدها را برای انجام یک کار خاص جمع آوری می کند و به آن گروه از کدها یک نام (شناساگر) داده می شود.

هنگامی که تابع از هر قسمت از برنامه فراخوانده می شود، تمام کدهای تعریف شده در بدنه تابع اجرا می شوند.

اعلام تابع در C++

```
returnType functionName (parameter1, parameter2,...) {  
    // function body  
}
```

در اینجا یک مثال از اعلام تابع آورده شده است:

(صفحه بعد)



```
// function declaration
void greet()
{
    cout << "Hello World";
}
```

در این مثال:

- نام تابع (`greet()`) است
- نوع برگشتی تابع `void` است
- پرانتز های خالی به این معنی است که تابع هیچ پارامتری ندارد
- بدنه تابع درون `{}` نوشته شده است

توجه: در ادامه این آموزش در مورد `returnType` و `parameter` صحبت خواهیم کرد.



فراخوانی تابع

در برنامه بالا، ما تابعی به نام `greet()` تعریف کرده ایم. برای استفاده از تابع `greet()`، باید آن را فراخوانی کنیم.
در اینجا نحوه فراخوانی تابع `greet()` بالا آورده شده است.

```
int main() {  
  
    // calling a function  
    greet();  
  
}
```

تصویر زیر نحوه اجرا تابع را نشان می دهد.

(صفحه بعد)



```
#include<iostream>

void greet() {
    // code
}

int main() {
    ... ..
    greet();
    ... ..
}
```

function call

مثال 1: نمایش یک متن

```
#include <iostream>
using namespace std;

// declaring a function
void greet() {
    cout << "Hello there!";
}
```



```
int main() {  
  
    // calling the function  
    greet();  
  
    return 0;  
}
```

خروجی

Hello there!

پارامتر های تابع

همانطور که قبلا ذکر شد، یک تابع را می توان با پارامتر (آرگومان) اعلام کرد. پارامتر مقداری است که هنگام تعریف تابع passed می شود.



به عنوان مثال، تابع زیر را در نظر بگیرید:

```
void printNum(int num) {  
    cout << num;  
}
```

در اینجا، متغیر `int num` پارامتر تابع است.

ما هنگام فراخوانی تابع، مقداری را به پارامتر تابع پاس می دهیم.

```
int main() {  
    int n = 7;  
  
    // calling the function  
    // n is passed to the function as argument  
    printNum(n);  
  
    return 0;
```




```
// program to print a text

#include <iostream>
using namespace std;

// display a number
void displayNum(int n1, float n2) {
    cout << "The int number is " << n1;
    cout << "The double number is " << n2;
}

int main() {

    int num1 = 5;
    double num2 = 5.5;
```



```
// calling the function  
displayNum(num1, num2);  
  
return 0;  
}
```

خروجی

```
The int number is 5  
The double number is 5.5
```

در برنامه بالا، از تابعی استفاده کردیم که یک پارامتر عددی صحیح (int) و یک پارامتر ممیزدار (double) دارد. سپس num1 و num2 را به عنوان آرگومان پاس می دهیم. این مقادیر به ترتیب توسط پارامترهای تابع n1 و n2 ذخیره می شوند.

تصویر زیر نحوه اجرا تابع را نشان می دهد.

(صفحه بعد)

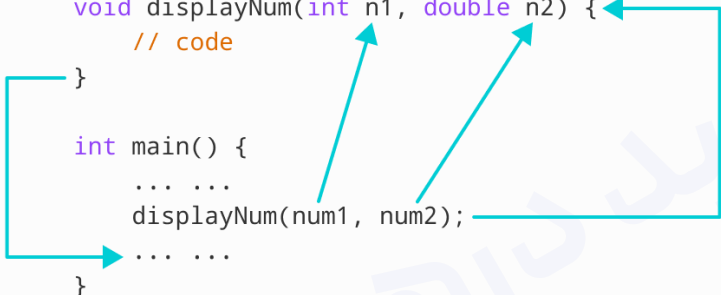


```
#include<iostream>
```

```
void displayNum(int n1, double n2) {  
    // code
```

```
}  
  
int main() {  
    ... ..  
    displayNum(num1, num2);  
    ... ..  
}
```

function
call



نکته: نوع آرگومان هایی که هنگام فراخوانی تابع ارسال می شوند باید با پارامترهای متناظر تعریف شده در اعلام تابع مطابقت داشته باشد.

عبارت بازگشتی

در برنامه های بالا، از void در اعلام تابع استفاده کرده ایم. به عنوان مثال:



```
void displayNumber() {  
    // code  
}
```

این بدان معناست که تابع هیچ مقداری را بر نمی گرداند.

بازگردانی یک مقدار از یک تابع نیز امکان پذیر است. برای این منظور، باید returnType تابع را در هنگام اعلام آن مشخص کنیم.

سپس، از عبارت بازگشتی می توان برای بازگرداندن یک ارزش از یک تابع استفاده کرد.

برای مثال:

```
int add (int a, int b) {  
    return (a + b);  
}
```

در اینجا به جای void، نوع داده int داریم. این بدان معنی است که تابع یک مقدار عدد صحیح (int) بر می گرداند.



کد `return (a + b);` مجموع دو پارامتر را به عنوان ارزش تابع باز می گرداند.

عبارت بازگشتی نشان می دهد که تابع به پایان رسیده است. هیچ کدی بعد از `return` در داخل تابع اجرا نمی شود.

مثال 3: جمع دو عدد

```
// program to add two numbers using a function
```

```
#include <iostream>
```

```
using namespace std;
```

```
// declaring a function
```

```
int add(int a, int b) {
```

```
    return (a + b);
```

```
}
```

```
int main() {
```



```
int sum;
```

```
// calling the function and storing
```

```
// the returned value in sum
```

```
sum = add(100, 78);
```

```
cout << "100 + 78 = " << sum << endl;
```

```
return 0;
```

```
}
```

خروجی

100 + 78 = 178

در برنامه بالا، از تابع add() برای پیدا کردن مجموع دو عدد استفاده می شود.

ما در هنگام فراخوانی تابع، دو عدد صحیح 100 و 78 را به آن وارد می کنیم.

مقدار برگشتی تابع را در متغیر sum ذخیره می کنیم و سپس آن را چاپ می کنیم.

```
#include<iostream>

int add(int a, int b) {
    return (a + b);
}

int main() {
    int sum;

    sum = add(100, 78);
    ... ..
}
```

function call

توجه کنید که sum یک متغیر از نوع int است. دلیل این امر این است که نوع مقدار بازگشتی تابع add() از نوع int است.

نمونه تابع (Function Prototype)

در C++، کد اعلام تابع باید قبل از فراخوانی تابع نوشته شود. با این حال، اگر بخواهیم تابعی را بعد از فراخوانی آن تعریف کنیم، باید از prototype

تابع استفاده کنیم. برای مثال:



```
// function prototype
```

```
void add(int, int);
```

```
int main() {
```

```
    // calling the function before declaration.
```

```
    add(5, 3);
```

```
    return 0;
```

```
}
```

```
// function definition
```

```
void add(int a, int b) {
```

```
    cout << (a + b);
```

```
}
```

در کد بالا، نمونه تابع به صورت زیر است:

```
void add(int, int);
```




این کد اطلاعاتی در مورد نام تابع و پارامترهای آن را برای کامپایلر فراهم می کند. به همین دلیل است که ما می توانیم قبل از تعریف تابع، از کد برای فراخوانی آن استفاده کنیم.

نحو (سینتکس) نمونه تابع به صورت زیر است:

```
returnType functionName(dataType1, dataType2, ...);
```

مثال 4: پروتوتایپ تابع در C++

```
// using function definition after main() function
// function prototype is declared before main()
#include <iostream>

using namespace std;

// function prototype
int add(int, int);
```



```
int main() {  
    int sum;  
  
    // calling the function and storing  
    // the returned value in sum  
    sum = add(100, 78);  
  
    cout << "100 + 78 = " << sum << endl;  
  
    return 0;  
}  
  
// function definition  
int add(int a, int b) {  
    return (a + b);  
}
```




$$100 + 78 = 178$$

برنامه بالا تقریباً مشابه مثال 3 است. تنها تفاوت این است که در اینجا، تابع بعد از فراخوانی تابع تعریف شده است. به همین دلیل است که ما در این مثال از یک نمونه اولیه تابع استفاده کرده ایم.

مزایای استفاده از توابع تعریف شده توسط کاربر

- توابع باعث می شوند کد قابل استفاده مجدد باشد. ما می توانیم آنها را یک بار تعریف کنیم و چندین بار از آنها استفاده کنیم.
- توابع برنامه را آسانتر می کنند زیرا هر کار کوچک به یک تابع تقسیم می شود.
- توابع باعث افزایش خوانایی کد می شوند.

توابع کتابخانه ای C++

توابع کتابخانه ای توابع داخلی (built-in) در برنامه نویسی C++ هستند. برنامه نویسان می توانند با فراخوانی مستقیم توابع از توابع کتابخانه ای استفاده کنند. آنها نیازی به نوشتن خود توابع ندارند. برخی از توابع کتابخانه ای رایج در C++ عبارتند از `sqrt()`، `abs()`، `isdigit()` و غیره.



برای استفاده از توابع کتابخانه ای، معمولا باید فایل هدر را که در آن این توابع کتابخانه ای تعریف شده اند را include کنیم.
برای مثال، برای استفاده از توابع ریاضی مانند sqrt() و abs()، باید فایل هدر cmath را include کنیم.

مثال 5: برنامه C++ برای پیدا کردن ریشه یک عدد

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double number, squareRoot;

    number = 25.0;

    // sqrt() is a library function to calculate the square root
    squareRoot = sqrt(number);
```



```
cout << "Square root of " << number << " = " << squareRoot;
```

```
return 0;
```

```
}
```

خروجی

Square root of 25 = 5

این برنامه از تابع کتابخانه `sqrt()` برای محاسبه ریشه دوم یک عدد استفاده می کند.

اعلام تابع `sqrt()` در فایل هدر `cmath` تعریف شده است. به همین دلیل است که برای استفاده از تابع `sqrt()` باید از کد `#include <cmath>` استفاده کنیم.

مطالب مرتبط:

- [توابع کتابخانه استاندارد C++](#)
- [انواع توابع تعریف شده توسط کاربر در C++](#)

انواع توابع تعریف شده توسط کاربر در C++

در این بخش به بررسی انواع توابع تعریف شده توسط کاربر در زبان C/C++ خواهیم پرداخت

انواع توابع تعریف شده توسط کاربر در C++

برای درک بهتر آرگومان ها و مقدار برگشتی در توابع، توابع تعریف شده توسط کاربر را می توان به چهار دسته تقسیم کرد:

- تابع بدون آرگومان و بدون مقدار برگشتی
- تابع بدون آرگومان ولی با مقدار برگشتی
- تابع با آرگومان ولی بدون مقدار برگشتی
- تابع با آرگومان و با مقدار برگشتی

به عنوان مثال، فرض کنید می خواهیم یک عدد را چک کنیم که اول است یا نه. این مسئله در زیر با ایجاد تابع تعریف شده توسط کاربر به چهار روش

مختلف حل می شود که در بالا ذکر شد.

مثال 1: بدون آرگومان ورودی و بدون مقدار برگشتی



```
#include <iostream>
using namespace std;

void prime();

int main()
{
    // No argument is passed to prime()
    prime();
    return 0;
}

// Return type of function is void because value is not returned.
void prime()
{

    int num, i, flag = 0;
```




```
int num, i, flag = 0;

cout << "Enter a positive integer enter to check: ";
cin >> num;

for(i = 2; i <= num/2; ++i)
{
    if(num % i == 0)
    {
        flag = 1;
        break;
    }
}

if (flag == 1)
{
    cout << num << " is not a prime number.";
}
```



```
else
{
    cout << num << " is a prime number.";
}
}
```

در برنامه بالا، تابع prime() از تابع main() بدون هیچ آرگومانی فراخوانی می شود.
تابع prime() یک عدد صحیح مثبت را از کاربر می گیرد و بررسی می کند که آیا آن عدد اول است یا خیر.
از آنجایی که نوع برگشتی prime() برابر void است، هیچ مقداری از تابع برگردانده نمی شود.

مثال 2: بدون آرگومان ورودی ولی با مقدار برگشتی

```
#include <iostream>
using namespace std;

int prime();
```



```
int main()
{
    int num, i, flag = 0;

    // No argument is passed to prime()
    num = prime();
    for (i = 2; i <= num/2; ++i)
    {
        if (num%i == 0)
        {
            flag = 1;
            break;
        }
    }

    if (flag == 1)
    {
        cout<<num<<" is not a prime number.";
    }
}
```



```
}  
else  
{  
    cout<<num<<" is a prime number."  
}  
return 0;  
}  
  
// Return type of function is int  
int prime()  
{  
    int n;  
  
    printf("Enter a positive integer to check: ");  
    cin >> n;  
  
    return n;  
}
```



در برنامه بالا، تابع `prime()` از تابع `main()` بدون هیچ آرگومانی فراخوانی می شود.

تابع `prime()` یک عدد صحیح مثبت را از کاربر دریافت می کند. از آنجایی که نوع برگشتی تابع برابر `int` است، عدد ورودی کاربر را به تابع `main()` فراخوانده بر می گرداند.

سپس، اینکه آیا عدد اول است یا خیر، در خود تابع `main()` بررسی شده و روی صفحه چاپ می شود.

مثال 3: با آرگومان ورودی ولی بدون مقدار برگشتی

```
#include <iostream>
using namespace std;

void prime(int n);

int main()
{
    int num;
    cout << "Enter a positive integer to check: ";
```



```
cin >> num;
```

```
// Argument num is passed to the function prime()
```

```
prime(num);
```

```
return 0;
```

```
}
```

```
// There is no return value to calling function. Hence, return type of function is void. */
```

```
void prime(int n)
```

```
{
```

```
int i, flag = 0;
```

```
for (i = 2; i <= n/2; ++i)
```

```
{
```

```
    if (n%i == 0)
```

```
    {
```

```
        flag = 1;
```

```
        break;
```

```
    }
```



```
}  
  
if (flag == 1)  
{  
    cout << n << " is not a prime number.";  
}  
else {  
    cout << n << " is a prime number.";  
}  
}
```

در برنامه بالا، ابتدا یک عدد صحیح مثبت از کاربر درخواست می شود که در متغیر num ذخیره می شود.
سپس، num به تابع prime() فرستاده می شود، جایی که بررسی می شود که آیا عدد اول است یا خیر و چاپ می شود.
از آنجایی که نوع برگشتی prime() برابر void است، هیچ مقداری از تابع برگردانده نمی شود.

مثال 4: با آرگومان ورودی و با مقدار برگشتی



```
#include <iostream>
using namespace std;

int prime(int n);

int main()
{
    int num, flag = 0;
    cout << "Enter positive integer to check: ";
    cin >> num;

    // Argument num is passed to check() function
    flag = prime(num);

    if(flag == 1)
        cout << num << " is not a prime number.";
    else
        cout << num << " is a prime number.";
```




```
    return 0;
}

/* This function returns integer value. */
int prime(int n)
{
    int i;
    for(i = 2; i <= n/2; ++i)
    {
        if(n % i == 0)
            return 1;
    }

    return 0;
}
```

در برنامه بالا، یک عدد صحیح مثبت از کاربر خواسته می شود و در متغیر num ذخیره می شود.
سپس، num به تابع prime() فرستاده می شود، جایی که بررسی می شود که آیا عدد اول است یا خیر.





از آنجایی که نوع برگشتی `prime()` برابر `int` است، عدد 1 یا 0 به تابع فراخواننده `main()` برگردانده می شود. اگر عدد اول باشد، 1 برگردانده می شود. در غیر این صورت، 0 برگردانده می شود.

در تابع `main()`، مقدار برگشتی 1 یا 0 در متغیر `flag` ذخیره می شود و متن مربوطه روی صفحه چاپ می شود.

کدام روش بهتر است؟

هر چهار برنامه بالا خروجی یکسانی می دهند و همه از نظر فنی برنامه های درستی هستند.

هیچ قانون سخت و سریعی در مورد اینکه کدام روش باید انتخاب شود وجود ندارد.

روش خاص بسته به موقعیت و نحوه حل مسئله انتخاب می شود.

سرفصل های مرتبط:

- [توابع در C++](#)
- [حلقه for در C++](#)



سربارگذاری توابع (Function Overloading) در C++

در این بخش به بررسی سربارگذاری توابع در زبان C++ خواهیم پرداخت

سربارگذاری توابع (Function Overloading) در C++

در C++، دو تابع می توانند نام یکسانی داشته باشند، به شرطی که تعداد و/یا نوع آرگومان های (arguments) ارسالی متفاوت باشد. این توابع که نام یکسان اما آرگومان های متفاوتی دارند به عنوان توابع سربار (overloaded functions) شناخته می شوند. برای مثال:

```
// same name different arguments
int test() { }
int test(int a) { }
float test(double a) { }
int test(int a, double b) { }
```

در اینجا، هر 4 تابع، توابع سربار (overloaded functions) هستند.

توجه داشته باشید که انواع برگشتی (return types) همه این 4 تابع یکسان نیستند. توابع سربار ممکن است انواع برگشتی متفاوتی داشته باشند یا نداشته باشند، اما باید آرگومان های متفاوتی داشته باشند. برای مثال:

(صفحه بعد)



```
// Error code
int test(int a) { }
double test(int b){ }
```

در اینجا، هر دو تابع نام یکسان، نوع یکسان و تعداد آرگومان های یکسان دارند. بنابراین، کامپایلر خطا مرتبط را نمایش خواهد داد.

مثال 1: بارگذاری اضافی با استفاده از انواع مختلف پارامتر

```
// Program to compute absolute value
// Works for both int and float

#include <iostream>
using namespace std;

// function with float type parameter
```



```
float absolute(float var){
    if (var < 0.0)
        var = -var;
    return var;
}

// function with int type parameter
int absolute(int var) {
    if (var < 0)
        var = -var;
    return var;
}

int main() {

    // call function with int type parameter
    cout << "Absolute value of -5 = " << absolute(-5) << endl;
```



```
// call function with float type parameter
```

```
cout << "Absolute value of 5.5 = " << absolute(5.5f) << endl;
```

```
return 0;
```

```
}
```

خروجی

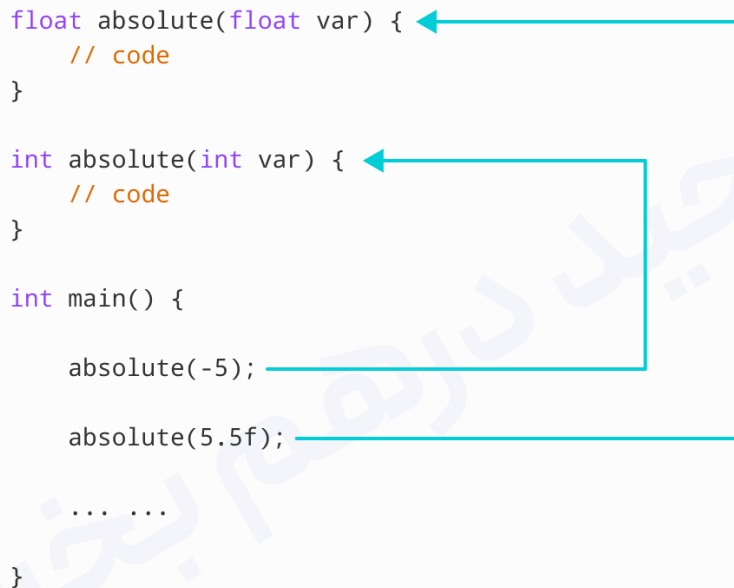
Absolute value of -5 = 5

Absolute value of 5.5 = 5.5

به تصویر زیر دقت کنید



```
float absolute(float var) {  
    // code  
}  
  
int absolute(int var) {  
    // code  
}  
  
int main() {  
    absolute(-5);  
    absolute(5.5f);  
    ...  
}
```



در این برنامه، ما تابع `absolute()` را سربارگذاری می کنیم (overload). بدین صورت بر اساس نوع پارامتری که در هنگام فراخوانی تابع ارسال می شود، تابع مربوطه فراخوانده می شود.



```
// Error code
int test(int a) { }
double test(int b){ }
```

در اینجا، هر دو تابع نام یکسان، نوع یکسان و تعداد آرگومان های یکسان دارند. بنابراین، کامپایلر خطا مرتبط را نمایش خواهد داد.

مثال 2: بارگذاری اضافی با استفاده از تعداد پارامترهای مختلف

```
#include <iostream>
using namespace std;

// function with 2 parameters
void display(int var1, double var2) {
    cout << "Integer number: " << var1;
    cout << " and double number: " << var2 << endl;
}
```



```
// function with double type single parameter
```

```
void display(double var) {  
    cout << "Double number: " << var << endl;  
}
```

```
// function with int type single parameter
```

```
void display(int var) {  
    cout << "Integer number: " << var << endl;  
}
```

```
int main() {
```

```
    int a = 5;
```

```
    double b = 5.5;
```

```
    // call function with int type parameter
```

```
    display(a);
```



```
// call function with double type parameter
```

```
display(b);
```

```
// call function with 2 parameters
```

```
display(a, b);
```

```
return 0;
```

```
}
```

خروجی

Integer number: 5

Float number: 5.5

Integer number: 5 and double number: 5.5

در اینجا، تابع `display()` سه بار با آرگومان های مختلف فراخوانده می شود. بسته به تعداد و نوع آرگومان های ارسال شده، تابع `display()` مربوطه

فراخوانده می شود.



```
void display(int var1, double var2) {  
    // code  
}  
  
void display(double var) {  
    // code  
}  
  
void display(int var) {  
    // code  
}  
  
int main() {  
    int a = 5;  
    double b = 5.5;  
  
    display(a);  
    display(b);  
    display(a, b);  
  
    ... ..  
}
```



نوع بازگشتی همه این توابع یکسان است، اما این الزاما برای سربارگذاری توابع ضروری نیست.

توجه: در C++، بسیاری از توابع کتابخانه استاندارد سربارگذاری شده اند. برای مثال، تابع `sqrt()` می تواند `int`، `float`، `double` و غیره را به عنوان پارامتر بگیرد. این امر امکان پذیر است زیرا تابع `sqrt()` سربارگذاری شده است.

سرفصل های مرتبط:

- بازنویسی توابع (Function Overriding) در C++
- سربارگذاری عملگر (Operator Overloading) در C++
- سربارگذاری تابع سازنده (Constructor Overloading) در C++

راید درهم بخش

آرگومان های پیش فرض (پارامترها) در برنامه نویسی C++

در این بخش به بررسی آرگومان های پیش فرض در زبان C++ خواهیم پرداخت

آرگومان های پیش فرض (پارامترها) در برنامه نویسی C++

در برنامه نویسی C++، ما می توانیم برای پارامترهای تابع، مقادیر پیش فرض را تعریف کنیم.

اگر تابعی با آرگومان های پیش فرض فراخوانده شود بدون اینکه آرگومانی به آن بدهیم، از مقادیر پیش فرض پارامترها استفاده می شود.

با این حال، اگر هنگام فراخوانی تابع، آرگومان هایی به آن داده شوند، آرگومان های پیش فرض نادیده گرفته می شوند.

نحوه عملکرد آرگومان های پیش فرض

برای درک بهتر عملکرد آرگومان های پیش فرض به تصویر زیر توجه نمایید:

(صفحه بعد)



Case 1: No argument is passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp();
    ...
}

void temp(int i, float f) {
    // code
}
```

Case 2: First argument is passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp(6);
    ...
}

void temp(int i, float f) {
    // code
}
```

Case 3: All arguments are passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp(6, -2.3);
    ...
}

void temp(int i, float f) {
    // code
}
```

Case 4: Second argument is passed

```
void temp(int = 10, float = 8.8);

int main() {
    ...
    temp(3.4);
    ...
}

void temp(int i, float f) {
    // code
}
```




ما می توانیم نحوه عملکرد آرگومان های پیش فرض را با استفاده از تصویر بالا به صورت زیر ارزیابی کنیم:

1. هنگامی که `temp()` فراخوانده می شود، هر دو پارامتر پیش فرض توسط تابع استفاده می شوند.
2. هنگامی که `temp(6)` فراخوانده می شود، آرگومان اول 6 می شود در حالی که از مقدار پیش فرض برای پارامتر دوم استفاده می شود.
3. هنگامی که `temp(6, -2.3)` فراخوانده می شود، هر دو پارامتر پیش فرض با `i = 6` و `f = -2.3` جایگزین می شوند.
4. هنگامی که `temp(3.4)` پاس داده می شود، تابع به روشی ناخواسته رفتار می کند زیرا آرگومان دوم بدون پاس دادن آرگومان اول قابل پاس دادن نیست.
بنابراین، 3.4 به عنوان آرگومان اول پاس داده می شود. از آنجایی که آرگومان اول به صورت `int` تعریف شده است، مقدار واقعی که منتقل می شود 3 است.

مثال: آرگومان پیشفرض

```
#include <iostream>
using namespace std;

// defining the default arguments
```



```
void display(char = '*', int = 3);
```

```
int main() {
```

```
    int count = 5;
```

```
    cout << "No argument passed: ";
```

```
    // *, 3 will be parameters
```

```
    display();
```

```
    cout << "First argument passed: ";
```

```
    // #, 3 will be parameters
```

```
    display('#');
```

```
    cout << "Both arguments passed: ";
```

```
    // $, 5 will be parameters
```

```
    display('$', count);
```

```
    return 0;
```



```
}
```

```
void display(char c, int count) {  
    for(int i = 1; i <= count; ++i)  
    {  
        cout << c;  
    }  
    cout << endl;  
}
```

خروجی

No argument passed: ***

First argument passed: ###

Both arguments passed: \$\$\$\$

نحوه عملکرد این برنامه به صورت زیر است:

1. تابع display بدون دریافت آرگومان فراخوانی می شود. در این حالت، display از هر دو پارامتر پیش فرض $c = '*'$ و $n = 1$ استفاده می کند.

2. تابع display با تنها یک آرگومان فراخوانی می شود. در این حالت، کاراکتر اول به # تغییر می کند. دومین پارامتر پیش فرض $n = 1$ همچنان حفظ می شود.

3. تابع display با دو آرگومان فراخوانی می شود. در این حالت، از آرگومان های ورودی استفاده شده و از مقادیر پیش فرض صرف نظر می شود.

ما همچنین می توانیم پارامترهای پیش فرض را مستقیماً درون تعریف تابع قرار دهیم. برنامه زیر معادل برنامه بالا است.

```
#include <iostream>
using namespace std;

// defining the default arguments
void display(char c = '*', int count = 3) {
    for(int i = 1; i <= count; ++i) {
        cout << c;
    }
}
```



```
    cout << endl;
}

int main() {
    int count = 5;

    cout << "No argument passed: ";
    // *, 3 will be parameters
    display();

    cout << "First argument passed: ";
    // #, 3 will be parameters
    display('#');

    cout << "Both argument passed: ";
    // $, 5 will be parameters
    display('$', count);
}
```



```
return 0;
```

```
}
```

نکات مهم

1. هنگامی که مقدار پیش فرض را برای یک پارامتر تعریف می کنیم، همه پارامترهای بعدی نیز باید دارای مقادیر پیش فرض باشند. برای مثال:

```
// Invalid
```

```
void add(int a, int b = 3, int c, int d);
```

```
// Invalid
```

```
void add(int a, int b = 3, int c, int d = 4);
```

```
// Valid
```

```
void add(int a, int c, int b = 3, int d = 4);
```

2. اگر آرگومان های پیش فرض را در تعریف تابع به جای نمونه اولیه تابع تعریف می کنیم، تابع باید قبل از فراخوانی تابع تعریف شود.

```
// Invalid code

int main() {
    // function call
    display();
}

void display(char c = '*', int count = 5) {
    // code
}
```

در کد بالا برای رفع خطا باید تابع display را بالاتر از تابع main تعریف کنیم، و یا از پروتوتایپ تابع برای آن استفاده نموده و آرگومان پیشفرض را تنها در پروتوتایپ آن قرار دهیم.



سرفصل های مرتبط:

- انواع توابع تعریف شده توسط کاربر در C++

مجید درهم بخش



توابع درون-خطی (inline) در C++

در این بخش به بررسی توابع درون-خطی (inline) در زبان C/C++ خواهیم پرداخت

توابع درون-خطی (inline) در C++

در C++، ما می توانیم یک تابع را به عنوان inline اعلام کنیم. این کار باعث می شود که تابع در زمان کامپایل در محل فراخوانی تابع کپی شود و ممکن است اجرای برنامه را سریعتر کند.

قبل از ادامه یادگیری این سرفصل، مطمئن شوید که سرفصل [توابع در C++](#) را مرور کرده اید.

توابع درون-خطی (inline) در C++

برای ایجاد یک تابع درون-خطی، از کلیدواژه inline استفاده می کنیم. برای مثال:

```
inline returnType functionName(parameters) {  
    // code  
}
```

به استفاده از کلیدواژه inline قبل از تعریف تابع دقت کنید.



inline returnType functionName (parameters)

↑
درون-خطی

↑
نوع خروجی

↑
نام تابع

↑
ورودی های تابع

مثال: توابع inline

```
#include <iostream>
using namespace std;

inline void displayNum(int num) {
    cout << num << endl;
}

int main() {
    // first function call
```



```
displayNum(5);
```

```
// second function call
```

```
displayNum(8);
```

```
// third function call
```

```
displayNum(666);
```

```
return 0;
```

```
}
```

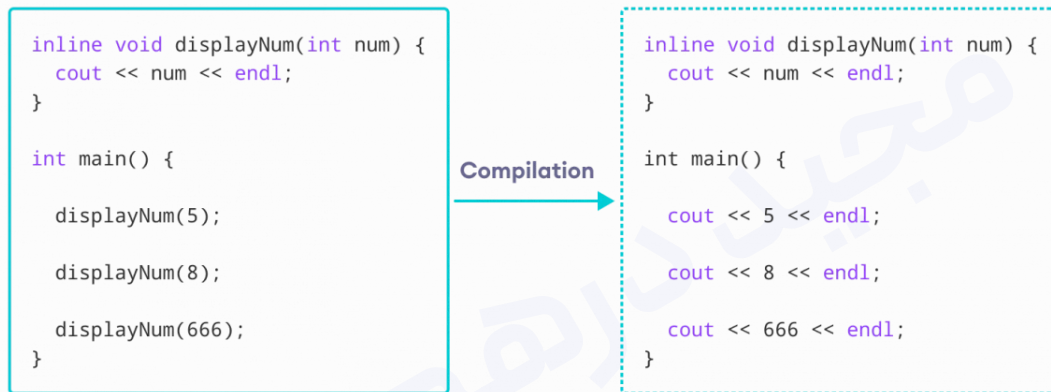
خروجی:

5

8

666

این برنامه به این شکل کار می کند:



در اینجا، ما یک تابع درون-خطی به نام `displayNum()` ایجاد کردیم که یک عدد صحیح به عنوان پارامتر می گیرد. سپس 3 بار تابع را در تابع `main()` با آرگومان های مختلف فراخوانی کردیم. هر بار که `displayNum()` فراخوانده می شود، کامپایلر کد تابع را در محل آن فراخوانی کپی می کند.

بازگشت (Recursion) در C++

در این بخش به بررسی توابع بازگشتی در زبان C/C++ خواهیم پرداخت

بازگشت (Recursion) در C++

یک تابع که خود را صدا می زند به عنوان تابع بازگشتی شناخته می شود. این تکنیک به بازگشت (Recursion) معروف است.

کارکرد بازگشت در C++

```
void recurse()  
{  
    ... ..  
    recurse();  
    ... ..  
}  
  
int main()  
{  
    ... ..  
    recurse();  
    ... ..  
}
```

شکل زیر نحوه عملکرد بازگشت (Recursion) را با فراخوانی مکرر خودش نشان می دهد.

```
void recurse() {  
    ... ..  
    recurse();  
    ... ..  
}  
  
int main() {  
    ... ..  
    recurse();  
    ... ..  
}
```

recursive call

function call

بازگشت (Recursion) تا زمانی که برخی شرایط برآورده شود ادامه می یابد.

برای جلوگیری از بازگشت بی نهایت، می توان از دستور if ... Else (یا رویکرد مشابه) استفاده کرد که در آن یک شاخه فراخوانی بازگشتی را انجام می دهد و دیگری انجام نمی دهد.

مثال 1: فاکتوریل یک عدد با استفاده از بازگشت



```
// Factorial of n = 1*2*3*...*n
```

```
#include <iostream>
```

```
using namespace std;
```

```
int factorial(int);
```

```
int main() {
```

```
    int n, result;
```

```
    cout << "Enter a non-negative number: ";
```

```
    cin >> n;
```

```
    result = factorial(n);
```

```
    cout << "Factorial of " << n << " = " << result;
```

```
    return 0;
```

```
}
```



```
int factorial(int n) {  
    if (n > 1) {  
        return n * factorial(n - 1);  
    } else {  
        return 1;  
    }  
}
```

خروجی:

```
Enter a non-negative number: 4  
Factorial of 4 = 24
```

کارکرد برنامه فاکتوریل با بازگشت



```
int main() {  
    ...  
    result = factorial(n);  
    ...  
}  
  
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}  
  
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}  
  
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}  
  
int factorial(int n) {  
    if (n > 1)  
        return n * factorial(n-1);  
    else  
        return 1;  
}
```

n = 4
4 * 6 = 24
is returned

n = 3
3 * 2 = 6
is returned

n = 2
2 * 1 = 2
is returned

n = 1
1 is returned



همانطور که می بینیم، تابع `factorial()` خود را فراخوانی می کند. با این حال، در طول هر فراخوانی، مقدار `n` را 1 کاهش داده ایم. هنگامی که `n` کمتر از 1 باشد، تابع `factorial()` در نهایت خروجی را بر می گرداند.

مزایا و معایب بازگشت

در زیر مزایا و معایب استفاده از بازگشت (Recursion) در C++ آمده است.

مزایای بازگشت (Recursion) در C++

- باعث می شود کد ما کوتاه تر و تمیزتر شود.
- بازگشت (Recursion) در مسائل مربوط به ساختارهای داده و الگوریتم های پیشرفته مانند پیمایش گراف و درخت الزامی است.

معایب بازگشت (Recursion) در C++

- این کار در مقایسه با یک برنامه تکرار شونده (iterative) فضای پشته (stack) زیادی را اشغال می کند.
- از زمان پردازنده بیشتری استفاده می کند.
- ممکن است اشکال زدایی آن نسبت به یک برنامه تکرار شونده معادل دشوارتر باشد.

مطالب مرتبط:

- [برنامه C++ برای محاسبه توان با استفاده از بازگشت](#)
- [برنامه C++ برای محاسبه فاکتوریل یک عدد با استفاده از بازگشت](#)

07

آرایه و رشته ها در C++

در این بخش به بررسی نحوه کار با آرایه ها و رشته ها در زبان C++

میپردازیم

آرایه هادر C++

در این بخش به بررسی آرایه ها در زبان C/C++ خواهیم پرداخت

آرایه هادر C++

در C++، یک آرایه متغیری است که می تواند مقادیر زیادی از یک نوع داده را ذخیره کند. برای مثال،

فرض کنید کلاسی دارای 27 دانش آموز باشد و بخواهیم نمرات همه آنها را ذخیره کنیم. به جای ایجاد 27 متغیر جداگانه، می توانیم به سادگی یک آرایه ایجاد کنیم:

```
double grade[27];
```

در اینجا، grade یک آرایه است که می تواند حداکثر 27 عنصر از نوع double را در خود جای دهد.

در C++، اندازه و نوع آرایه ها بعد از اعلام آن قابل تغییر نیستند.

اعلام آرایه در C++

```
dataType arrayName[arraySize];
```




data_type array_name [array_size];

↑
نوع داده

↑
نام آرایه

↑
سایز آرایه

برای مثال:

```
int x[6];
```

در اینجا،

- int - نوع داده ای که در آرایه ذخیره می شود
- x - نام آرایه
- 6 - اندازه آرایه

دسترسی به عناصر در آرایه C++

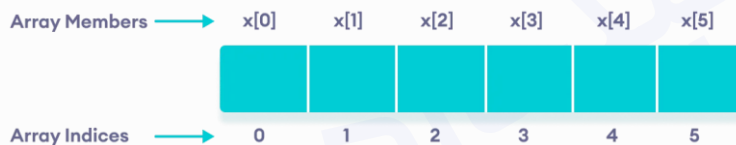
در C++، به هر عنصر در یک آرایه، عددی اختصاص داده می شود. این عدد، «شاخص آرایه» (index) نامیده می شود. ما می توانیم با استفاده از این

شاخص ها به عناصر یک آرایه دسترسی پیدا کنیم. رایگان برای آموزش شخصی - کپی و بازنشر بدون مجوز ممنوع

// syntax to access array elements

array[index];

آرایه ی x که قبلا دیدیم را در نظر بگیرید.



چند نکته برای یادآوری:

- شاخص های آرایه از 0 شروع می شوند. به این معنی که x[0] اولین عنصری است که در شاخص 0 ذخیره شده است.
- اگر اندازه یک آرایه n باشد، آخرین عنصر در شاخص (n-1) ذخیره می شود. در این مثال، x[5] آخرین عنصر است.
- عناصر یک آرایه دارای آدرس های متوالی هستند.

برای مثال، فرض کنید آدرس شروع x[0] برابر با 1000 باشد.

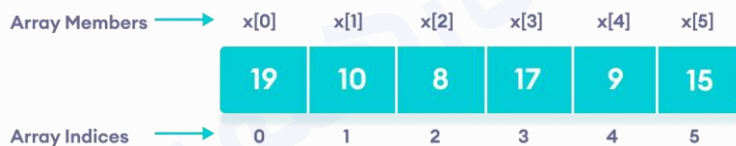
سپس، آدرس عنصر بعدی x[1] برابر با 1004، آدرس x[2] برابر با 1008 و به همین ترتیب خواهد بود.

در اینجا، اندازه هر عنصر به اندازه 4 افزایش می یابد. زیرا اندازه int برابر 4 بایت است.

مقداردهی اولیه آرایه ها در C++

در C++، امکان مقداردهی اولیه به یک آرایه در هنگام اعلام آن وجود دارد. برای مثال:

```
// declare and initialize an array
int x[6] = {19, 10, 8, 17, 9, 15};
```



روش دیگری برای مقداردهی اولیه آرایه در هنگام اعلام:

```
// declare and initialize an array
int x[] = {19, 10, 8, 17, 9, 15};
```

در اینجا، اندازه آرایه را ذکر نکرده ایم. در چنین مواقعی، کامپایلر به طور خودکار اندازه را محاسبه می کند.

آرایه با عضوهای خالی در C++

در C++، اگر یک آرایه دارای اندازه n باشد، می توانیم تا n عنصر را در آرایه ذخیره کنیم. با این حال، اگر کمتر از n عنصر ذخیره کنیم چه اتفاقی می افتد؟

برای مثال،

```
// store only 3 elements in the array  
int x[6] = {19, 10, 8};
```

در اینجا اندازه آرایه x برابر با 6 است. با این حال، ما آن را تنها با 3 عنصر مقداردهی اولیه کرده ایم. در چنین مواردی، کامپایلر مقادیر تصادفی را به مکان های باقی مانده اختصاص می دهد. این مقدار تصادفی اغلب به سادگی 0 است.



چگونه عناصر آرایه را مقداردهی و چاپ کنیم؟

```
int mark[5] = {19, 10, 8, 17, 9}

// change 4th element to 9
mark[3] = 9;

// take input from the user
// store the value at third position
cin >> mark[2];

// take input from the user
// insert at ith position
cin >> mark[i-1];

// print first element of the array
cout << mark[0];
```



```
// print ith element of the array
cout >> mark[i-1];
```

مثال 1: نمایش عناصر آرایه

```
#include <iostream>
using namespace std;

int main() {

    int numbers[5] = {7, 5, 6, 12, 35};

    cout << "The numbers are: ";

    // Printing array elements
    // using range based for loop
```



```
for (int n : numbers) {  
    cout << n << " ";  
}  
  
cout << "\nThe numbers are: ";  
  
// Printing array elements  
// using traditional for loop  
for (int i = 0; i < 5; ++i) {  
    cout << numbers[i] << " ";  
}  
  
return 0;  
}
```

خروجی



```
The numbers are: 7 5 6 12 35
```

```
The numbers are: 7 5 6 12 35
```

در اینجا، ما از یک حلقه for برای تکرار از $i = 0$ تا $i = 4$ استفاده کرده ایم. در هر تکرار، `numbers[i]` را چاپ کرده ایم.

ما دوباره از یک حلقه for مبتنی بر محدوده برای چاپ عناصر آرایه استفاده کردیم. برای کسب اطلاعات بیشتر در مورد این حلقه، [سرفصل حلقه های مبتنی بر بازه در C++](#) را بررسی کنید.

مثال 2: دریافت ورودی از کاربر و ذخیره آن در یک آرایه

```
#include <iostream>
using namespace std;

int main() {

    int numbers[5];
```




```
cout << "Enter 5 numbers: " << endl;
```

```
// store input from user to array
```

```
for (int i = 0; i < 5; ++i) {  
    cin >> numbers[i];  
}
```

```
cout << "The numbers are: ";
```

```
// print array elements
```

```
for (int n = 0; n < 5; ++n) {  
    cout << numbers[n] << " ";  
}
```

```
return 0;
```

```
}
```

Enter 5 numbers:

11

12

13

14

15

The numbers are: 11 12 13 14 15

دوباره، ما از یک حلقه for برای تکرار از $i = 0$ تا $i = 4$ استفاده کردیم. در هر تکرار، ما ورودی را از کاربر گرفتیم و آن را در `numbers[i]` ذخیره کردیم. سپس، از یک حلقه for دیگر برای چاپ تمام عناصر آرایه استفاده کردیم.

خارج از محدوده آرایه در C++

اگر آرایه ای با اندازه 10 تعریف کنیم، این آرایه شامل عناصری از اندیس 0 تا 9 خواهد بود.

با این حال، اگر سعی کنیم به عنصری با اندیس 10 یا بیشتر از 10 دسترسی پیدا کنیم، منجر به رفتار تعریف نشده ای می شود.



این نوع خطا زمانی محتمل است که از یک حلقه for معمولی استفاده کنیم و به عنصر آرایه با استفاده از عملگر [] دسترسی پیدا کنیم.



توجه: برای غلبه بر این نوع خطا، بهتر است از حلقه for مبتنی بر بازه استفاده کنیم که در آن نیازی به چنین عملگر هایی برای دسترسی به عناصر آرایه نیست.

مثال 3: نمایش مجموع و میانگین عناصر آرایه با استفاده از حلقه for

```
#include <iostream>
using namespace std;

int main() {

    // initialize an array without specifying size
    double numbers[] = {7, 5, 6, 12, 35, 27};

    double sum = 0;
```



```
double count = 0;
double average;

cout << "The numbers are: ";

// print array elements
// use of range-based for loop
for (const double &n : numbers) {
    cout << n << " ";

    // calculate the sum
    sum += n;

    // count the no. of array elements
    ++count;
}

// print the sum
```



```
cout << "\nTheir Sum = " << sum << endl;

// find the average
average = sum / count;
cout << "Their Average = " << average << endl;

return 0;
}
```

خروجی

```
The numbers are: 7 5 6 12 35 27
Their Sum = 92
Their Average = 15.3333
```

در این برنامه:

1. ما یک آرایه double به نام numbers را بدون مشخص کردن اندازه آن مقداردهی اولیه کرده ایم. همچنین سه متغیر double دیگر به نام های

sum، count و average اعلام کردیم.

در اینجا، $sum = 0$ و $count = 0$.

2. سپس از یک حلقه for مبتنی بر محدوده برای چاپ عناصر آرایه استفاده کردیم. در هر تکرار حلقه، عنصر آرایه جاری را به sum اضافه می کنیم.

3. همچنین مقدار count را در هر تکرار 1 افزایش می دهیم تا بتوانیم اندازه آرایه را در انتهای حلقه for بدست آوریم.

4. پس از چاپ تمام عناصر، مجموع و میانگین همه اعداد را چاپ می کنیم. میانگین اعداد با $average = sum / count$ بدست می آید.

سرفصل های مرتبط:

- [آرایه های چند بعدی در C++](#)
- [اشاره گر و آرایه در C++](#)
- [بردار در C++](#)
- [آرایه std در C++](#)

وارد کردن آرایه به یک تابع در برنامه نویسی C++

در این بخش به بررسی نحوه وارد کردن و استفاده از یک آرایه به توابع در زبان C/C++ خواهیم

پرداخت

وارد کردن آرایه به یک تابع در برنامه نویسی C++

در C++، ما می توانیم آرایه ها را به عنوان آرگومان به یک تابع منتقل کنیم. همچنین، می توانیم آرایه ها را از یک تابع برگردانیم. قبل از اینکه در مورد انتقال آرایه ها به عنوان آرگومان تابع یاد بگیرید، مطمئن شوید که در مورد آرایه های C++ و توابع C++ دانش دارید.

سینتکس

سینتکس برای انتقال یک آرایه به یک تابع به شرح زیر است:

```
returnType functionName(dataType arrayName[]) {  
    // code  
}
```

returnType functionName (dataType arrayName[]);

نوع خروجی

نام تابع

نوع آرایه ورودی

آرایه ورودی

مثال 1: انتقال آرایه یک بعدی به یک تابع

```
// C++ Program to display marks of 5 students
#include <iostream>
using namespace std;

const int ARRAY_SIZE = 5;

// declare function to display marks
// take a 1d array as parameter
void display(int m[]) {
    cout << "Displaying marks: " << endl;

    // display array elements
    for (int i = 0; i < ARRAY_SIZE; ++i) {
        cout << "Student " << i + 1 << ": " << m[i] << endl;
    }
}
```



```
int main() {  
  
    // declare and initialize an array  
    int marks[ARRAY_SIZE] = {88, 76, 90, 61, 69};  
  
    // call display function  
    // pass array as argument  
    display(marks);  
  
    return 0;  
}
```

خروجی

```
Displaying marks:  
Student 1: 88
```



Student 2: 76

Student 3: 90

Student 4: 61

Student 5: 69

در اینجا،

1. زمانی که تابعی را با ارسال یک آرایه به عنوان آرگومان فراخوانی می کنیم، تنها از نام آرایه استفاده می شود.

```
display(marks);
```

در اینجا، آرگومان marks نشان دهنده آدرس حافظه اولین عنصر آرایه marks[5] است.

2. با این حال، به پارامتر تابع `display(int m[])` توجه کنید.

```
void display(int m[])
```

در اینجا، ما فقط انتظار یک آرایه از اعداد صحیح را داریم. C++ برای صرفه جویی در حافظه و زمان، ارسال یک آرایه به تابع را به این روش

مدیریت می کند.



انتقال آرایه چند بعدی به یک تابع

همچنین می توان آرایه های چند بعدی را به عنوان آرگومان به تابع ارسال کرد. برای مثال،

مثال 2: انتقال آرایه چند بعدی به یک تابع

```
// C++ Program to display the elements of two
// dimensional array by passing it to a function

#include <iostream>
using namespace std;

// define a function
// pass a 2d array as a parameter
void display(int n[][2]) {
    cout << "Displaying Values: " << endl;
```



```
for (int i = 0; i < 3; ++i) {  
    for (int j = 0; j < 2; ++j) {  
        cout << "num[" << i << "][" << j << "]: " << n[i][j] << endl;  
    }  
}  
  
int main() {  
  
    // initialize 2d array  
    int num[3][2] = {  
        {3, 4},  
        {9, 5},  
        {7, 1}  
    };  
  
    // call the function
```



```
// pass a 2d array as an argument
```

```
display(num);
```

```
return 0;
```

```
}
```

خروجی

Displaying Values:

num[0][0]: 3

num[0][1]: 4

num[1][0]: 9

num[1][1]: 5

num[2][0]: 7

num[2][1]: 1



در برنامه بالا، تابعی به نام نمایش (display) تعریف کرده ایم. این تابع یک آرایه دو بعدی از اعداد صحیح را به عنوان آرگومان (argument) می پذیرد و عناصر آرایه را چاپ می کند.

هنگام فراخوانی تابع، فقط نام آرایه دو بعدی را به عنوان آرگومان تابع نمایش (display(num)) ارسال می کنیم.

توجه: دقت داشته باشید که لازم نیست تعداد ردیف ها در آرایه مشخص شود. با این حال، تعداد ستون ها همیشه باید مشخص شود. به همین دلیل است که از `int n[2]` استفاده کرده ایم.

ما همچنین می توانیم آرایه هایی با بیش از دو بعد را به عنوان آرگومان تابع ارسال کنیم.

برگرداندن آرایه از یک تابع در C++

ما همچنین می توانیم یک آرایه را از تابع برگردانیم. با این حال، آرایه واقعی برگردانده نمی شود. در عوض، آدرس اولین عنصر آرایه با کمک اشاره گرها برگردانده می شود.

ما در آموزش های بعدی در مورد برگرداندن آرایه از یک تابع خواهیم آموخت.



آرایه های چند بعدی در C++

در این بخش به بررسی آرایه های چند بعدی در زبان C/C++ خواهیم پرداخت

آرایه های چند بعدی در C++

در C++، ما می توانیم یک آرایه از آرایه ها ایجاد کنیم، که به عنوان یک آرایه چند بعدی شناخته می شود. برای مثال:

```
int x[3][4];
```

در اینجا، x یک آرایه دو بعدی است. این آرایه می تواند حداکثر ۱۲ عنصر را در خود جای دهد.

می توانیم این آرایه را به عنوان جدولی با ۳ سطر تصور کنیم و هر سطر دارای ۴ ستون است، همانطور که در زیر نشان داده شده است.

	Col 1	Col 2	Col 3	Col 4
Row 1	x[0][0]	x[0][1]	x[0][2]	x[0][3]
Row 2	x[1][0]	x[1][1]	x[1][2]	x[1][3]
Row 3	x[2][0]	x[2][1]	x[2][2]	x[2][3]

آرایه های سه بعدی نیز به شکل مشابهی کار می کنند. برای مثال:

```
float x[2][4][3];
```

این آرایه x می تواند حداکثر 24 عنصر را در خود جای دهد.

ما می توانیم تعداد کل عناصر موجود در آرایه را با ضرب ابعاد آن به دست آوریم:

$$2 \times 4 \times 3 = 24$$

مقداردهی اولیه آرایه های چند بعدی

مانند یک آرایه معمولی، ما می توانیم یک آرایه چند بعدی را به بیش از یک روش مقداردهی کنیم.

1. مقداردهی آرایه دو بعدی

```
int test[2][3] = {2, 4, 5, 9, 0, 19};
```

روش بالا ترجیح داده نمی شود. روش بهتر برای مقداردهی این آرایه با عناصر مشابه آرایه در زیر آمده است:



```
int test[2][3] = { {2, 4, 5}, {9, 0, 19}};
```

این آرایه دارای ۲ سطر و ۳ ستون است، به همین دلیل ما دو ردیف از عناصر با ۳ عنصر در هر ردیف داریم.

	Col 1	Col 2	Col 3
Row 1	2	4	5
Row 2	9	0	19

2. مقداردهی آرایه سه بعدی

```
int test[2][3][4] = {3, 4, 2, 3, 0, -3, 9, 11, 23, 12, 23,  
2, 13, 4, 56, 3, 5, 9, 3, 5, 5, 1, 4, 9};
```

روش بالا راه خوبی برای مقداردهی اولیه یک آرایه سه بعدی نیست. روش بهتر برای مقداردهی اولیه این آرایه عبارت است از:



```
int test[2][3][4] = {  
    { {3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 23, 2} },  
    { {13, 4, 56, 3}, {5, 9, 3, 5}, {5, 1, 4, 9} }  
};
```

به ابعاد این آرایه سه بعدی توجه کنید.

بعد اول مقدار ۲ دارد. بنابراین، دو عنصری که بعد اول را تشکیل می دهند عبارتند از:

Element 1 = { {3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 23, 2} }

Element 2 = { {13, 4, 56, 3}, {5, 9, 3, 5}, {5, 1, 4, 9} }

بعد دوم مقدار ۳ دارد. توجه کنید که هر کدام از عناصر بعد اول، خود دارای سه عنصر هستند:

{3, 4, 2, 3}, {0, -3, 9, 11} and {23, 12, 23, 2} for Element 1.

{13, 4, 56, 3}, {5, 9, 3, 5} and {5, 1, 4, 9} for Element 2.

در نهایت، در هر یک از عناصر بعد دوم، چهار عدد صحیح وجود دارد.

{3, 4, 2, 3}
{0, -3, 9, 11}
... ..
... ..

مثال 1: آرایه های دو بعدی

```
// C++ Program to display all elements  
// of an initialised two dimensional array  
  
#include <iostream>  
using namespace std;  
  
int main() {
```



```
int test[3][2] = {{2, -5},
                  {4, 0},
                  {9, 1}};
```

```
// use of nested for loop
```

```
// access rows of the array
```

```
for (int i = 0; i < 3; ++i) {
```

```
    // access columns of the array
```

```
    for (int j = 0; j < 2; ++j) {
```

```
        cout << "test[" << i << "][" << j << "] = " << test[i][j] << endl;
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```



```
test[0][0] = 2
test[0][1] = -5
test[1][0] = 4
test[1][1] = 0
test[2][0] = 9
test[2][1] = 1
```

در مثال بالا، یک آرایه دو بعدی از نوع `int` به نام `test` ایجاد کردیم که دارای 3 سطر و 2 ستون است.

در اینجا، از حلقه های تو در تو `for` برای نمایش عناصر آرایه استفاده کرده ایم.

- حلقه بیرونی با `i` که از 0 تا 2 تغییر می کند به سطر های آرایه دسترسی پیدا می کند.

- حلقه داخلی با `z` که از 0 تا 1 تغییر می کند به ستون های آرایه دسترسی پیدا می کند.

در نهایت، عناصر آرایه را در هر تکرار چاپ می کنیم.

مثال 2: دریافت ورودی برای آرایه های دو بعدی



```
#include <iostream>
using namespace std;

int main() {
    int numbers[2][3];

    cout << "Enter 6 numbers: " << endl;

    // Storing user input in the array
    for (int i = 0; i < 2; ++i) {
        for (int j = 0; j < 3; ++j) {
            cin >> numbers[i][j];
        }
    }

    cout << "The numbers are: " << endl;

    // Printing array elements
```




```
for (int i = 0; i < 2; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        cout << "numbers[" << i << "][" << j << "]: " << numbers[i][j] << endl;  
    }  
}  
  
return 0;  
}
```

خروجی

Enter 6 numbers:

1
2
3
4
5
6



The numbers are:

numbers[0][0]: 1

numbers[0][1]: 2

numbers[0][2]: 3

numbers[1][0]: 4

numbers[1][1]: 5

numbers[1][2]: 6

در اینجا، از یک حلقه **for** تو در تو برای گرفتن ورودی آرایه دو بعدی استفاده کرده ایم. هنگامی که تمام ورودی ها گرفته شد، از یک حلقه **for** تو در تو دیگر برای چاپ اعضای آرایه استفاده کرده ایم.

مثال 3: آرایه های سه بعدی

```
// C++ Program to Store value entered by user in  
// three dimensional array and display it.
```



```
#include <iostream>
using namespace std;

int main() {
    // This array can store upto 12 elements (2x3x2)
    int test[2][3][2] = {
        {
            {1, 2},
            {3, 4},
            {5, 6}
        },
        {
            {7, 8},
            {9, 10},
            {11, 12}
        }
    };
};
```



```
// Displaying the values with proper index.
```

```
for (int i = 0; i < 2; ++i) {  
    for (int j = 0; j < 3; ++j) {  
        for (int k = 0; k < 2; ++k) {  
            cout << "test[" << i << "][" << j << "][" << k << "] = " << test[i][j][k] << endl;  
        }  
    }  
}  
  
return 0;  
}
```

خروجی

```
test[0][0][0] = 1  
test[0][0][1] = 2  
test[0][1][0] = 3
```



```
test[0][1][1] = 4
test[0][2][0] = 5
test[0][2][1] = 6
test[1][0][0] = 7
test[1][0][1] = 8
test[1][1][0] = 9
test[1][1][1] = 10
test[1][2][0] = 11
test[1][2][1] = 12
```

مفهوم کلی چاپ عناصر یک آرایه سه بعدی مشابه مفهوم چاپ یک آرایه دو بعدی است.

با این حال، از آنجایی که ما با سه بعد کار می کنیم، به جای دو حلقه for از یک حلقه for تو در تو با مجموعاً سه حلقه استفاده می کنیم:

- حلقه بیرونی از $i == 0$ تا $i == 1$ به بعد اول آرایه دسترسی پیدا می کند
- حلقه میانی از $z == 0$ تا $z == 2$ به بعد دوم آرایه دسترسی پیدا می کند
- حلقه درونی ترین از $k == 0$ تا $k == 1$ به بعد سوم آرایه دسترسی پیدا می کند

همانطور که می بینیم، پیچیدگی آرایه با افزایش ابعاد به صورت تصاعدی افزایش می یابد.

مطالب مرتبط:

- برنامه C++ برای جمع دو ماتریس با استفاده از آرایه های چند بعدی
- برنامه C++ برای ضرب دو ماتریس با استفاده از آرایه های چند بعدی

رشته‌های (String) C++

در این بخش به بررسی رشته‌ها در زبان C++ خواهیم پرداخت

رشته های (String) C++

رشته مجموعه ای از کاراکترها است. دو نوع رشته به طور معمول در C++ استفاده می شود:

- رشته هایی که شی از کلاس string هستند (کلاس رشته کتابخانه استاندارد C++)
- رشته های C (رشته های به سبک C)

رشته های C

در برنامه نویسی C، مجموعه کاراکترها به صورت آرایه ذخیره می شوند. این مورد در برنامه نویسی C++ نیز پشتیبانی می شود. از همین رو به آنها رشته های C گفته می شود.

رشته های C آرایه هایی از نوع char هستند که با یک کاراکتر تهی (null character) خاتمه می یابند، یعنی \0 (مقدار ASCII کاراکتر تهی 0 است).

چگونه یک رشته C تعریف کنیم؟

```
char str[] = "C++";
```


در کد بالا، str یک رشته است و حاوی 4 کاراکتر می باشد.

اگرچه "C++" دارای سه کاراکتر است، اما کاراکتر تهی \0 به طور خودکار به انتهای رشته اضافه می شود.

روش های جایگزین برای تعریف یک رشته

```
char str[4] = "C++";
```

```
char str[] = {'C', '+', '+', '\0'};
```

```
char str[4] = {'C', '+', '+', '\0'};
```

شبیه آرایه ها، ضروری نیست از کل فضای اختصاص داده شده برای رشته استفاده شود. برای مثال:

```
char str[100] = "C++";
```

مثال 1: رشته C++ برای خواندن یک کلمه



```
#include <iostream>
using namespace std;

int main()
{
    char str[100];

    cout << "Enter a string: ";
    cin >> str;
    cout << "You entered: " << str << endl;

    cout << "\nEnter another string: ";
    cin >> str;
    cout << "You entered: "<<str<<endl;

    return 0;
}
```

Enter a string: C++

You entered: C++

Enter another string: Programming is fun.

You entered: Programming

توجه کنید که در مثال دوم، به جای "Programming is fun"، فقط "Programming" نمایش داده می شود.

این به این دلیل است که عملگر استخراج >> مانند scanf() در C عمل می کند و فضای خالی " " را به عنوان یک کاراکتر پایان دهنده در نظر می گیرد.

مثال 1: رشته C++ برای خواندن یک خط متن

```
#include <iostream>
using namespace std;
```



```
int main()
{
    char str[100];
    cout << "Enter a string: ";
    cin.get(str, 100);

    cout << "You entered: " << str << endl;
    return 0;
}
```

خروجی

```
Enter a string: Programming is fun.
You entered: Programming is fun.
```

برای خواندن متن حاوی فضای خالی، می توان از تابع `cin.get` استفاده کرد. این تابع دو آرگومان می گیرد.

آرگومان اول نام رشته (آدرس اولین عنصر رشته) و آرگومان دوم حداکثر اندازه آرایه است.

در برنامه بالا، `str` نام رشته و 100 حداکثر اندازه آرایه است. برای آموزش شخصی - کپی و بازنشر بدون مجوز ممنوع



شی String در C++

در C++، شما همچنین می توانید برای نگهداری رشته ها یک شی رشته ایجاد کنید. برخلاف استفاده از آرایه های char اشیاء رشته طول ثابتی ندارند و می توانند حسب نیاز شما گسترش یابند.

مثال 3: رشته C++ با استفاده از نوع داده رشته

```
#include <iostream>
using namespace std;

int main()
{
    // Declaring a string object
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
}
```



```
cout << "You entered: " << str << endl;  
return 0;  
}
```

خروجی

```
Enter a string: Programming is fun.  
You entered: Programming is fun.
```

در این برنامه، ابتدا یک رشته با نام str تعریف می شود. سپس، رشته از کاربر درخواست می شود. به جای استفاده از تابع `cin>>` یا `cin.get()`، می توانید خط متنی وارد شده را با استفاده از `getline()` دریافت کنید. تابع `getline()` استریم ورودی را به عنوان پارامتر اول که `cin` و `str` به عنوان مکان خطی است که باید ذخیره شود، می گیرد.

پاس دادن رشته به تابع

رشته ها به روشی مشابه آرایه ها به یک تابع منتقل می شوند.



```
#include <iostream>

using namespace std;

void display(char *);
void display(string);

int main()
{
    string str1;
    char str[100];

    cout << "Enter a string: ";
    getline(cin, str1);

    cout << "Enter another string: ";
    cin.get(str, 100, '\n');
```





```
    display(str1);  
    display(str);  
    return 0;  
}  
  
void display(char s[])  
{  
    cout << "Entered char array is: " << s << endl;  
}  
  
void display(string s)  
{  
    cout << "Entered string is: " << s << endl;  
}
```

خروجی





Enter a string: Programming is fun.

Enter another string: Really?

Entered string is: Programming is fun.

Entered char array is: Really?

در برنامه فوق، از کاربر خواسته می شود دو رشته را وارد کند. این رشته ها به ترتیب در str و str1 ذخیره می شوند، جایی که str یک آرایه از کاراکترها و str1 یک شی رشته است.

سپس، ما دو تابع display() داریم که خروجی رشته را روی صفحه نمایش می دهند.

تنها تفاوت بین این دو تابع پارامتر آن ها است.

تابع display() اول آرایه کاراکتر را به عنوان پارامتر می گیرد، در حالی که تابع دوم رشته را به عنوان پارامتر می گیرد.

این فرآیند به عنوان سربارگذاری تابع (Function Overloading) شناخته می شود.

سرفصل های مرتبط:

- کلاس String در C++

- cstring در C++



- تبدیل رشته به عدد اعشاری (float/double) و بالعکس در C++

- تبدیل رشته به عدد صحیح (int) و بالعکس در C++

مجید درهم بخش



کلاس String در C++

در این بخش به بررسی کلاس String در زبان C++ خواهیم پرداخت



کلاس String در C++

در C++، از کلاس رشته برای نمایش دادن توالی ای از کاراکترها به عنوان یک شی از کلاس استفاده می شود. با درج کردن هدر <string> در فایل خود، می توانیم به توابع مختلف کلاس رشته دسترسی پیدا کنیم.

```
#include <string>
```

توابع کلاس رشته

توابع رایجی که با کلاس رشته استفاده می شوند عبارتند از:

تابع	توضیحات
find()	پیدا کردن اندیس زیر رشته در رشته



پیدا کردن آخرین اندیس یک زیر رشته در رشته	<code>rfind()</code>
اضافه کردن به آخر رشته	<code>append()</code>
قرار دادن درون رشته	<code>insert()</code>
پاک کردن کاراکترها از رشته	<code>erase()</code>
جایگزین کردن بخش هایی از رشته	<code>replace()</code>
مقایسه کردن دو رشته	<code>compare()</code>

مثال 1: پیدا کردن زیر رشته در رشته داده شده

تابع `find()` رشته را برای اندیس زیر رشته مشخص شده جستجو می کند و موقعیت اولین کاراکتر از اولین تطابق را بر می گرداند.

تابع `rfind()` رشته را برای آخرین اندیس زیر رشته مشخص شده جستجو می کند و موقعیت اولین کاراکتر از آخرین تطابق را بر می گرداند.

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    string str = "Hello world, wonderful world!";
    cout << "String: " << str << endl;

    // find the first occurrence
    size_t first_occurrence = str.find("world");

    // find the last occurrence
    size_t last_occurrence = str.rfind("world");

    if (first_occurrence != string::npos) {
```



```
cout << "First occurrence: 'world' found at position: " << first_occurrence << endl;
cout << "Last occurrence: 'world' found at position: " << last_occurrence << endl;
}
else {
    cout << "'world' not found" << endl;
}

return 0;
}
```

خروجی

```
String: Hello world, wonderful world!
First occurrence: 'world' found at position: 6
Last occurrence: 'world' found at position: 23
```

خط کد (first_occurrence != string::npos) بررسی می کند که آیا متغیر first_occurrence حاوی یک موقعیت معتبر است که زیر

رشته world در داخل رشته اصلی پیدا شده باشد. رایگان برای آموزش شخصی - کپی و بازنشر بدون مجوز ممنوع



اگر **world** در رشته پیدا شود، **first_occurrence** روی موقعیت اولین باری که پیدا شده است تنظیم می شود.

اگر **world** پیدا نشود، روی یک مقدار ثابت خاص **string::npos** تنظیم می شود، که نشان می دهد زیر رشته در رشته اصلی پیدا نشده است.

توجه: هر دو تابع به حروف حساس هستند. بنابراین، اگر رشته ما **Hello World, wonderful world!** باشد، تابع **find()** عدد ۲۳ را به عنوان اولین باری که زیر رشته پیدا شده است برمیگرداند.

مثال 2: الحاق به یک رشته

میتوانیم با استفاده از تابع **append()** رشته ای را به یک رشته موجود اضافه کنیم. برای مثال:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
```




```
string str = "Hello world,";
cout << "Before: " << str << endl;

//append the string
str.append(" Have a good day!");

cout << "After: " << str << endl;

return 0;
}
```

خروجی

```
Before: Hello world,
After: Hello world, Have a good day!
```

مثال 3: درج رشته در موقعیت مشخص

با استفاده از تابع `insert()` می توانیم یک رشته را در هر موقعیت دلخواهی درج کنیم. برای مثال،

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    string str = "Hello world, world!";
    cout << "Before: " << str << endl;

    // insert "beautiful" at the 13th index
    str.insert(13, " beautiful");

    cout << "After: " << str << endl;
```



```
return 0;
```

```
}
```

خروجی

Before: Hello world, world!

After: Hello world, beautifulworld!

در اینجا، زیر رشته **beautiful** را در سیزدهمین اندیس رشته قرار داده ایم. توجه داشته باشید که این کار فضایی را در انتهای رشته اضافه شده اضافه نمی کند.

مثال 4: پاک کردن N کاراکتر از موقعیت داده شده

```
#include <iostream>
#include <string>
using namespace std;
```



```
int main() {  
  
    string str = "Hello world, beautiful world!";  
    cout << "Before: " << str << endl;  
  
    // replace two characters with "Earth"  
    // starting from the seventh index  
    str.replace(6, 2, "Earth");  
  
    cout << "After: " << str << endl;  
  
    return 0;  
}
```

خروجی

Before: Hello world, beautiful world!

After: Hello w beautiful world!

در اینجا، با استفاده از تابع `erase()`، پنج کاراکتر را از شروع اندیس هفتم حذف کردیم.

مثال 5: جایگزینی N کاراکتر در یک رشته

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    string str = "Hello world, beautiful world!";
    cout << "Before: " << str << endl;

    // replace two characters with "Earth"
    // starting from the seventh index
    str.replace(6, 2, "Earth");
```



```
cout << "After: " << str << endl;
```

```
return 0;
```

```
}
```

خروجی

Before: Hello world, beautiful world!

After: Hello Earthrld, beautiful world!

اینجا، دو کاراکتر (wo) را با Earth جایگزین کرده ایم. ما می توانیم با استفاده از تابع **replace()** هر طول رشته ای را با هر طول رشته دیگری جایگزین کنیم.

مثال 6: مقایسه حروف الفبایی رشته ها

برای بدست آوردن روابط واژه نگاری بین رشته ها، از تابع **compare()** استفاده می کنیم.



تابع **compare()** در کلاس رشته C++ یک عدد صحیح بر می گرداند که نشان دهنده رابطه واژه نگاری بین رشته های مقایسه شده است. این تابع موارد زیر را بر می گرداند:

- 0 - اگر رشته های مقایسه شده برابر باشند.
- < 0 - اگر رشته فراخواننده دارای طول کمتری از رشته مقایسه شده باشد.
- > 0 - اگر رشته فراخواننده دارای طول بیشتری از رشته مقایسه شده باشد.

برای مثال،

```
#include <iostream>
#include <string>
using namespace std;

int main() {

    string str1 = "Hello world";
    string str2 = "Hello world";
    string str3 = "Hello";
```



```
string str4 = "Hello world, What's Up?";
```

```
cout << "String 1: " << str1 << endl;
```

```
cout << "String 2: " << str2 << endl;
```

```
cout << "String 3: " << str3 << endl;
```

```
cout << "String 4: " << str4 << endl;
```

```
// compare the strings
```

```
cout << "Comparing String 1 and String 2: " << str1.compare(str2) << " (Equal)" << endl;
```

```
cout << "Comparing String 1 and String 3: " << str1.compare(str3) << " (String 1 is Longer)" << endl;
```

```
cout << "Comparing String 1 and String 4: " << str1.compare(str4) << " (String 1 is Smaller)" << endl;
```

```
return 0;
```

```
}
```

خروجی

(صفحه بعد)



```
String 1: Hello world
String 2: Hello world
String 3: Hello
String 4: Hello world, What's Up?
Comparing String 1 and String 2: 0 (Equal)
Comparing String 1 and String 3: 6 (String 1 is Longer)
Comparing String 1 and String 4: -12 (String 1 is Smaller)
```

در اینجا،

- `str1.compare(str2)` مقدار 0 را بر می گرداند زیرا `str1` و `str2` برابر هستند.
- `str1.compare(str3)` عددی مثبت را بر می گرداند زیرا `str1` از نظر لغوی بزرگتر از `str3` است (حروف بیشتری دارد).
- `str1.compare(str4)` عددی منفی را بر می گرداند زیرا `str1` از نظر لغوی کمتر از `str4` است.

08

ساختارها و داده‌های شمارشی در C++

در این بخش به بررسی انواع ساختارها و تعریف انواع داده منحصر به فرد خواهیم پرداخت

ساختارها (Structures) در C++

در این بخش به بررسی نحوه کار با ساختارها در زبان C/C++ خواهیم پرداخت

ساختارها (Structures) در ++C

یک ساختار (structure) مجموعه ای از متغیرها با انواع داده ای متفاوت و توابع عضو (member functions) تحت یک نام واحد است.

این مفهوم مشابه با مفهوم کلاس (class) است، زیرا هر دو مجموعه ای از داده ها با انواع داده ای متفاوت را نگه می دارند.

فرض کنید می خواهید اطلاعاتی در مورد یک فرد را ذخیره کنید: `first_name`، `last_name`، `age` و `salary`.

شما به راحتی می توانید متغیرهای مختلفی مانند `first_name`، `last_name`، `age` و `salary` را برای ذخیره سازی جداگانه این اطلاعات ایجاد کنید.

با این حال، ممکن است در آینده بخواهید اطلاعات مربوط به چندین نفر را ذخیره کنید.

حالا برای هر اطلاعات مربوط به هر فرد، باید متغیرهای متفاوتی ایجاد کنید: `first_name1`، `last_name1`، `age1`، `salary1`، `first_name2`،

`last_name2`، `age2`، `salary2` و ...

می توانید تصور کنید که کد چقدر بزرگ و درهم به نظر می رسد. علاوه بر این، از آنجایی که بین متغیرها (اطلاعات) ارتباطی وجود ندارد، مدیریت آن

ها کار دشواری خواهد بود.



رویکرد بهتر این است که مجموعه ای از تمام اطلاعات مرتبط را تحت یک نام واحد، مانند **Person**، داشته باشید و از آن برای هر فرد استفاده کنید.

حالا کد بسیار تمیزتر، خواناتر و کارآمدتر به نظر می رسد.

این مجموعه از تمام اطلاعات مرتبط تحت یک نام واحد **Person**، یک ساختار است.

چگونه در برنامه نویسی C++ یک ساختار (Structure) تعریف کنیم؟

کلمه کلیدی struct نوعی ساختار را به همراه یک شناسه (نام ساختار) تعریف می کند.

سپس، داخل آکولاد می توانید یک یا چند عضو (اعضا همان متغیرهایی هستند که داخل آکولاد تعریف می شوند) برای آن ساختار تعریف کنید. برای

مثال:

```
struct Person
{
    string first_name;
    string last_name;
    int age;
    float salary;
};
```



در این مثال، ساختار Person تعریف شده است که دارای چهار عضو می باشد: first_name، last_name، age و salary.

زمانی که یک ساختار تعریف می شود، حافظه خاصی برای آن اختصاص داده نمی شود.

تعریف ساختار تنها نقش یک الگو برای ایجاد متغیرها را دارد. می توانید آن را به عنوان یک نوع داده در نظر بگیرید.

زمانی که یک عدد صحیح را به شکل زیر تعریف می کنید:

```
int foo;
```

کلمه کلیدی `int` مشخص می کند که متغیر `foo` تنها می تواند عناصر صحیح را نگه دارد. به طور مشابه، تعریف ساختار تنها مشخص می کند که یک متغیر از نوع ساختار چه ویژگی هایی را می تواند داشته باشد.

توجه: فراموش نکنید که اعلان ساختار را با سمی کالن (;) به پایان برسانید.

تعریف متغیر از نوع ساختار

وقتی ساختاری به نام Person مانند بالا تعریف کردید، می توانید یک متغیر ساختاری به این شکل تعریف کنید:

(صفحه بعد)



```
Person bill;
```

در اینجا، یک متغیر ساختاری به نام bill تعریف شده است که از نوع ساختار Person است. فقط زمانی که متغیر ساختاری تعریف می شود، حافظه مورد نیاز توسط کامپایلر تخصیص داده می شود.

دسترسی به اعضای یک ساختار

برای دسترسی به اعضا یک متغیر ساختاری از عملگر نقطه (.) استفاده می شود. فرض کنید می خواهید به عضو age متغیر ساختاری bill دسترسی پیدا کنید و مقدار 50 را به آن اختصاص دهید. می توانید این کار را با استفاده از کد زیر انجام دهید:

```
bill.age = 50;
```

مثال: ساختار در C++

(صفحه بعد)



```
// Program to assign data to members of a structure variable
```

```
#include <iostream>
using namespace std;
```

```
struct Person
{
    string first_name;
    string last_name;
    int age;
    float salary;
};
```

```
int main()
{
    Person p1;

    cout << "Enter first name: ";
```




```
cin >> p1.first_name;
cout << "Enter last name: ";
cin >> p1.last_name;
cout << "Enter age: ";
cin >> p1.age;
cout << "Enter salary: ";
cin >> p1.salary;

cout << "\nDisplaying Information." << endl;
cout << "First Name: " << p1.first_name << endl;
cout << "Last Name: " << p1.last_name << endl;
cout << "Age: " << p1.age << endl;
cout << "Salary: " << p1.salary;

return 0;
}
```



```
Enter first name: Jane
Enter last name: Smith
Enter age: 27
Enter salary: 10000

Displaying Information.
First Name: Jane
Last Name: Smith
Age: 27
Salary: 10000
```

در اینجا ساختاری به نام Person تعریف شده است که دارای چهار عضو به نام های first_name، last_name، age و salary می باشد.

در تابع main یک متغیر ساختاری به نام p1 تعریف می شود. سپس از کاربر خواسته می شود اطلاعات وارد کند و اطلاعات وارد شده توسط کاربر نمایش داده می شود.



توابع عضو در ساختارهای C++

در C++، ساختارها نیز می توانند توابع عضو داشته باشند.

این توابع عضو مشابه توابع معمولی هستند اما در محدوده یک ساختار تعریف می شوند. آن ها می توانند مستقیماً به اعضای داده ساختار دسترسی پیدا کنند و آن ها را دستکاری کنند.

ما می توانیم یک تابع عضو را با تعریف تابع در داخل تعریف ساختار اعلام کنیم.

```
struct Person {  
    string first_name;  
    string last_name;  
    int age;  
    float salary;  
  
    // member function to display information about the person  
    void displayInfo() {  
        cout << "First Name: " << first_name << endl;  
        cout << "Last Name: " << last_name << endl;  
    }  
};
```



```
    cout << "Age: " << age << endl;  
    cout << "Salary: " << salary << endl;  
}  
};
```

در این مثال، ساختار Person شامل یک تابع عضو به نام displayInfo() است که اطلاعات مربوط به شخص را نمایش می دهد.

بیایید به یک مثال نگاه کنیم.

```
#include <iostream>  
using namespace std;  
  
struct Person {  
    string first_name;  
    string last_name;  
    int age;  
    float salary;
```



```
// member function to display information about the person
```

```
void display_info() {  
    cout << "First Name: " << first_name << endl;  
    cout << "Last Name: " << last_name << endl;  
    cout << "Age: " << age << endl;  
    cout << "Salary: " << salary << endl;  
}  
};
```

```
int main() {  
    Person p1;  
  
    cout << "Enter first name: ";  
    cin >> p1.first_name;  
    cout << "Enter last name: ";  
    cin >> p1.last_name;  
    cout << "Enter age: ";  
    cin >> p1.age;
```



```
cout << "Enter salary: ";
cin >> p1.salary;

// display information using member function
cout << "\nDisplaying Information." << endl;
p1.display_info();

return 0;
}
```

خروجی

```
Enter first name: Jane
Enter last name: Smith
Enter age: 27
Enter salary: 10000

Displaying Information.
```



First Name: Jane

Last Name: Smith

Age: 27

Salary: 10000

سرفصل های مرتبط:

- چگونه یک ساختار را به توابع وارد کنیم؟
- چگونه از اشاره گرها با ساختارها استفاده کنیم؟

ساختارها (Structures) و توابع در C++

در این بخش به بررسی ارتباط ساختارها و توابع در زبان C/C++ خواهیم پرداخت

اشاره گر به ساختار (Pointers to Structure) در C++

در این بخش به بررسی ارتباط اشاره گر ها با ساختار ها در زبان C/C++ خواهیم پرداخت

داده های شمارشی (Enumeration) در C++

در این بخش به بررسی نحوه کار با داده های شمارشی در زبان C/C++ خواهیم پرداخت

09

اشاره گر و مرجع در ++C

در این بخش به بررسی نحوه کنترل حافظه و ارتباط با آدرس های حافظه توسط اشاره گر و مرجع خواهیم پرداخت

اشاره گر و آرایه در C++

در این بخش به بررسی نحوه کار با اشاره گر و آرایه و ارتباط آن ها با یکدیگر در زبان C/C++ خواهیم پرداخت

بردار در C++

در این بخش به بررسی بردار ها و نحوه کار با آن ها در زبان C++ خواهیم پرداخت

آرایه STD در C++

در این بخش به بررسی آرایه موجود در فضای نام std در زبان C/C++ خواهیم پرداخت

پایان دوره مقدماتی

سوالات ارزیابی پایانی دوره مقدماتی C++

به سوالات ارسال شده پاسخ داده و پاسخنامه را ارسال نمایید